$\sqrt{Eurocomp}$ LGP-21 Subroutine Handbook

Equinox Deschanel

June 2026

"Our need will be the real creator."

— Plato, *The Republic*

"The enemy of art is the absence of limitations."

— Orson Welles

Contents

1 Notes on the Translation

1.1 Number representation

Introduction to fractional fixed-point architecture

The LGP-21's number representation is very different from 21st century computing platforms. It is not at all unique; many early computers used it, including the IBM 701[3], the ILLAC I[4], the Elliot 405[5], the RPC-4000[10], and the LGP-21's predecessor, the LGP-30[6]. This architecture is known as the IAC architecture, and was first proposed by John von Neumann for EDSAC[1].

A fixed-point fractional representation treats all numbers as fractions lying between -1 and $+1$. As von Neumann pointed out[2], the multiplication of 2 n-bit integers results in a value that may require $2n$ bits to represent it, which could easily overflow a register, but the multiplication of *fractions* always results in a number no larger, and usually smaller, than both multiplicands (e.g., $0.5 \times 0.5 = 0.25$). The answer still requires $2n$ bits to represent, but the extra bits end up in the *least* significant digits, preventing overflow from occurring.

The design uses a combined register pair to contain the $2n$ bits; on the LGP-21, the accumulator receives the most significant half, and a hidden register not available to the programmer gets the least significant half. To reduce complexity, the LGP-21 simply throws this second half away. Since these are the least significant digits, the (unscaled) error introduced is 2^{-31} or less.

This advantage in multiplication is offset by a problem this creates with division: fractional multiplication *shrinks* numbers, but fractional division (potentially) *expands* them.

In a fractional fixed-point architecture, dividing two numbers requires that the absolute magnitude of the divisor has to be *strictly greater than* the absolute magnitude of the dividend.

$$|\text{Divisor}| > |\text{Dividend}|$$

Getting this wrong means that the theoretical quotient is ≥ 1.0, and such a value cannot physically fit in the accumulator. The architecture maintains the *hardware* binary point just after the sign bit, so a value ≥ 1.0 results in an overflow.

Von Neumann wrote that "*[t]he programmer must see to it* [emphasis mine - ED] that the left-shift [scale factor] is sufficiently large to ensure this inequality."burks1946preliminary[2] The cognitive load of remembering where the *implied* binary point is (via the scaling factor) vs. the *physical* binary point (implemented by the hardware) is placed solely on the programmer.

Modern embedded systems use similar notation (referred to as *Q-notation* or *fixed-point scaling*) to define this manual placement of the implicit binary point within a computer word, and in fact, the original version of this document specifically refers to the scaling factor as q.

LGP-21 Fractional Fixed-point Implementation

The LGP-21 treats all 31-bit words strictly as fractions ranging from -1 to $+1$, with the physical binary point fixed immediately to the right of the (left-most) sign bit. To work with integers or mixed numbers, the programmer must mentally shift this binary point to the right by n bits (as indicated by the @n notation).

For example, a value at texttt@3 allocates 3 bits for the integer portion of a number (allowing values up to 7.999999) and the remaining 27 bits for the fraction. As noted above, the programmer is *entirely responsible* for tracking this scale factor across operations, and must manually shift, multiply, or divide intermediate values as needed to maintain the desired scale factor.

Addition/Subtraction Pitfalls

Addition and subtraction require that the scale factors (`@n`) of both operands be identical before the operation is executed. The LGP-21 treats every 31-bit word as a raw fixed-point fraction. If you add two numbers with different q factors without shifting them appropriately, the machine will blindly align their bits and perform the operation, smashing incommensurate units of completely different mathematical weights into the same columns.

For example, if you attempt to add 1.0 scaled at `@1` to 1.0 scaled at `@4` without normalizing them first:

	1.0 at `@1` →	`0 100000...`	(Hardware sees 2^{29})
+	1.0 at `@4` →	`0 000100...`	(Hardware sees 2^{26})
	Raw Sum →	`0 100100...`	(Hardware sees $2^{29} + 2^{26}$)

Depending on which scale factor you *think* the result is in, your answer is now either 1.125 (if you assume the result is `@1`) or 9.0 (if you assume `@4`). The machine will not throw an error, trigger an overflow, or warn you; it will simply calculate perfectly represented garbage because the binary point in the two numbers is in *two different places*.

It is vital to understand that the LGP-21 hardware itself does not know about scale factors and has no way to check them or track them. Scale factor changes occur during multiplication and division: multiplication *adds* the scale factors, and division *subtracts* them. After a multiplication or division, the result must be shifted left or right to ensure that the result conforms to the scale factor desired.

As an example, if an `@1` value were to be divided by an `@9` one, the resulting scale factor would be -8, meaning that to return it to an `@1` value, the result would need to be shifted right *8* bits, and for an `@9` result, it would need to be shifted right *17*(!) bits.

In the subroutines documented below, I've carried over the original `@n` notation where it was used, most often for referring to values in storage (e.g., θ@9) or the accumulator (accumulator`@1`).

1.2 Document organization

First, the code described in this document spans a wide range of functions. Some of it really is subroutines, some of it what we'd call functions (subroutines that return a specific value or result), and some is what we'd call a program, utility. or monitor. The original document simply calls everything a suroutine, and what the document calls a "calling sequence" is how the code is executed. This means that some "calling sequences" really are a set of machine instructions that perform a subroutine call, while others are a series of operations that the machine operator must perform to load and start the program.

Because the actual function of the code is highly variable, I've chosen to refer to the individual programs as appropriate, whether a function, subroutine, utility, etc. "Calling sequence" is altered as appropriate for the kind of program in question; it may be termed "Execution" or "Usage" instead.

The document is organized in broad section, with the broadly broken into mathematical functions, a pair of floating-point calculators, input and output support code, utilities, and a grab-bag of miscellaneous other algorithms and utilities: sorts, searches, data maintenance/backup, and programming tools. (The tool that automates self-modifying code loops is particularly engaging!)

Code names

The programs do not have names! This is probably because the name was pretty arbitrary as far as the machine was concerned – you can't load a program by name, etc.; it's all simply reference material for humans. As such the names are assigned to organize the programs, from broad functional groups down to exactly which program is which.

The first character of the name establishes which broad area a program falls into: `B` is mathematical, `D` is matrixes, `J` is input/output, and so on. The second character refines this: `B1` is trig functions; `B3` is exponential/log functions; `B4` is roots. There is no rule as to how the refinement is assigned other than personal taste.

The numeric part of the name refines the classification further. `B1` is trig; `B1-10` is sine and cosine, `B1-11` is arctangent, `B1-12` is arcsin/arccos. Within that numeric classification, we have a point number that differentiates implementations. So `B1-10.0` (we number the point classifications from `0`) is mathematics (`B`), trigonometry (`1`), sine/cosine (`-10`), first implementation (`.1`). As a sample, `B1-10.0` is the *first* sine and cosine implementation, which accepts degrees; `B1-10.1` is the *second* sine and cosine implementation, which accepts radians.

These break up reasonably well into sections and subsections, so I've done that to make it easier to navigate the table of contents. I have not reordered any of the programs to ensure that any cross-referencing between the German and English versions of the document were easy to do.

1.3 LGP-21 coding conventions

1.3.1 Address notation

Subroutine addresses in the original text are written as `L` or, for offsets into program code, `L+n`.

Addresses in the main program are written as α or $\alpha + n$.

The LGP-21 is too primitive to have anything like a linkage editor, so it's the programmer/operator's responsibility to use the LGP-21's Program Input Routine[1] to load subroutine code and automatically relocate it as it's loaded. The usage of `L` for the base address of code is to remind the reader that they will need to relocate these library functions by hand and use the new base addresses in their code to make subroutine calls work properly.

1.3.2 Calling sequences

Because the LGP-21 libraries were written by multiple programmers, and there was no oversight body to enforce a One True Way of doing things, there is a Cambrian explosion of alternatives. Very capable programmers not only coded their algorithms, but cleverly spaced their storage so that the data was under the disk's read head when an instruction wanted to access memory.[2]

Fortunately, none of those programming tricks are used here[3], but the "let a million flowers bloom" philosophy does show itself in the multiple ways there are to call subroutines.

Accumulator-only parameters

One calling convention streamlines the call as much as possible by loading the parameter into the accumulator, setting the return address in a `U` instruction in the subroutine at an agreed-upon offset, and then jumping to the subroutine:

Location	**Instruction**	**Address**	**Description**
$\alpha - 1$	`B`	`N`	Bring N into the accumulator.
α	`R`	`L+`*`offset`*	Store the return address in the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to the subroutine entry.

In this example a `B` loads the argument into the accumulator, but any instruction that gets the appropriate value into the accumulator is just as valid. The `R` saves the return address (α+2) in the subroutine, and the main program's `U` to the start of the subroutine calls it. Execution of the main program resumes just after the `U` to `L` at $\alpha + 2$.

[1] or Rhys Weatherly's `lgp21-tools` assembler, which can include and relocate subroutines.

[2] See the story of Mel[?], who originally wrote for the LGP-30; his code was famous for using instructions as data, and exploiting the weird corners of the instruction set to do things like have loops with no exit which nevertheless eventually exit.

[3] As far as I know. Heaven knows what's actually lurking in the library code itself.

This means that calling a subroutine implies that code *must* be modified for a return to work: specifically, a `U` (unconditional jump) instruction in the subroutine must be altered to set the return address. Multiple calls to a subroutine therefore will clobber any previous return address.

This means that re-entrant code is not natively supported by the hardware. Programmers must ensure that they carefully manage the hierarchy of subroutine calls to prevent a second call to a subroutine, directly or indirectly, before it returns.

Inlined parameters

A second calling convention, with longer, in-memory parameter lists, looks like this:

Location	Instruction	Address	Description
$\alpha - 1$	`E`	`0000`	Exit from the Interpretive System.
α	`R`	`L`	Modify subroutine instruction with key word address.
$\alpha + 1$	`U`	`L`	Unconditional branch to subroutine entry.
$\alpha + 2$	…	…	A fixed number of parameter words
$\alpha + n$	—	—	Return to main program.

This call looks exceedingly strange if you're used to the previous calling convention. If we were doing exactly the same operation as before (altering a `U` instruction to set the return, then jumping to the subroutine), we'd just jump right back, so that can't be right, and it isn't.

This convention uses the `R` instruction to load the address of the *parameters* and store it into a `B` instruction at the start of the subroutine instead. Jumping to the subroutine loads the address of the parameters ($\alpha + 2$) into the accumulator, allowing the subroutine to add an offset to this address to set it to $\alpha + n$, which can then be stored locally in a `U` instruction to do the actual return to the caller, and to have a pointer to the parameters as well!

Deep Wizardry: Floating-Point Parameters

A third, even more astounding calling sequence passes a multi-word floating-point[4] number to a subroutine as an argument.

Location	Instruction	Address	Description
$\alpha - 1$	`B`	`N`	Bring word 1 of floating-point number N.
α	`R`	`L+offset`	Store the return address ($\alpha + 2$) inside the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to the subroutine entry.

[4]While the LGP-21 lacks floating-point *hardware*, it relies entirely on a software-driven floating-point math *interpreter*. See Section **??**.

Now wait a minute, I hear you say. A floating-point number is *three* words long. That cannot possibly fit in the single-word hardware accumulator—what kind of trickery is this?

You're right. The full floating-pint number doesn't get loaded; only its first word does. But the `B` instruction itself at cell $\alpha - 1$ now contains the exact piece of metadata the subroutine needs: *the starting memory address `N` of the floating-point block*[5].

This explains why the manual repeatedly states that "loading this value into the accumulator by any means" is acceptable for some subroutines, but does *not* say that for this one[6]. The `B` here is essential to the program working. It relies on the code modified by the `R` instruction to find and read its caller:

1. The subroutine picks up the return address ($\alpha + 2$) deposited by the `R` instruction into cell `L+offset`.
2. It subtracts 3 from that value to locate cell $\alpha - 1$ (the `B` instruction).
3. It modifies the next instruction, a `B`, to set its address to $\alpha - 1$.
4. The modified `B` instruction loads the literal 32-bit instruction word stored at $\alpha - 1$...that contains the address of the floating-point number.
5. It strips away the `B` opcode bits using a bitmask, leaving behind only the raw address `N`.
6. Now the subroutine has the address of the first of the three words making up the floating-point number, and can proceed.

The value loaded into the accumulator at step $\alpha - 1$ is *entirely ignored*. The `B` instruction's sole purpose in this calling sequence is to serve as an *executable static data pointer*, which the subroutine finds and dissects to find word 2 (`N+1` of the floating-point parameter.

1.4 Repeated/redundant information

Because this is a reference manual meant for use by people who were casually programmers and principally something else entirely, this manual fairly consistently repeats things like the standard `B/R/U` instruction sequence used to call a function as noted in **??**. I have tried to condense this where possible.

Some subroutines use unusual parameterization (such as patching the values of words in the subroutine itself!), or are actually programs to be loaded and run, and I have retained all of the setup information and/or programming information as it was originally written in these cases.

[5] My reaction was "You are *kidding* me" with a significant expletive titre.

[6] Gun, see foot.

1.5 Acknowledgements and disclaimers

Thanks in particular to David Lovett of Usagi Electric for resurrecting a real, physical LGP-21 and starting me down this road; Rhys Weatherly for the lgp21-tools, and Paul Kimpel for the wonderful LGP-21 web emulator that has given me such a lot of pleasure learning the machine.

Thanks also to the many people whose names I don't know[7], who have tirelessly made sure that important historical documents like the original German version of this manual and the programs that go with it have been preserved. This document is my small contribution to this historical preservation effort, and I am immensely grateful to those giants upon whose shoulders I stand.

This work *is* a translation, and I cannot guarantee I have been completely accurate, whether from misunderstanding or transcription error. I've done my best to be sure my interpretation of what's documented here is correct, but I have not run or tested all of the programs referenced, and I can't make any guarantees as to their fitness and accuracy, nor can I be sure that the document I've translated here is in fact 100 percent accurate itself! I have made corrections where errors were obvious, but I may have missed things.

To that end, this document will be up on GitHub; please feel free to submit PRs for corrections and errata there if you find anything that needs a fix. I intend to CC license this document such that it can be freely built upon and distributed, but that credit for the basic work is retained.

[7] Send me patches via GitHub so I can credit you, please!

2 Trigonometric functions

2.1 Sine-Cosine 1

B1-10.0

PURPOSE

`B1-10.0` calculates the sine or cosine of an angle value in degrees. If the angle supplied is greater than 360°, the value is reduced to an angle between 0° and 360° before the function value is calculated. A 9th-degree polynomial is used for approximation.

`B1-10.0` uses the standard LGP-21 calling sequence. The angle argument must supplied in the accumulator@9. The function value, including its sign, will be in the accumulator@1 when the subroutine returns.

Memory usage

Memory for program	1 track
Buffer usage	Sectors 2, 4, 5, 6, 7, 45 of track 63

Input

The input variable for B1-10.0 is an angle in degrees, θ, for which the function value is to be calculated. θ must be in the accumulator@9 when the jump to the subroutine occurs. The angle is reduced to an angle between 0° and 360° by the subroutine before evaluation.

Calling Sequence

Different calling sequences are provided for sine and cosine. Only one value (either sine or cosine) can be calculated per subroutine call.

Sine

Location	Instruction	Address	Description
$\alpha - 1$	`B`	θ	; Load θ@9 into the accumulator
α	`R`	$L + 49$	; Store return address at end of sub
$\alpha + 1$	`U`	L	; call the sine routine

This is a standard function call, with the argument passed in the accumulator and the result returned in it.

Cosine

Location	Instruction	Address	Description
$\alpha - 1$	`B`	θ	; Load θ@9 into the accumulator
α	`R`	`L+49`	; Store return address at end of sub
$\alpha + 1$	`U`	`L+4`	; call the cosine routine

The calling sequence for cosine is exactly the same as for sine, except the subroutine is entered at the origin address of the `B1-10.0` subroutine plus 4 words.

Output

When the subroutine returns, the properly-signed function value is in the accumulator@1.

OPERATION

- **Accuracy:** The value is accurate to within 5 x 10^{-7}.
- **Time required:** approximately 775 ms.
- **Program input:** the tape contains the program in two versions:
  - Relocatable, hexadecimal. Valid for J1-10.0 (section 8.1).
  - Relocatable, decimal, in coding sheet format.
- **Required I/O devices:** none.
- **Overflow:** ignored on entry and not reset on exit.

2.2 Sine-Cosine 2

B1-10.1

PURPOSE

`B1-10.1` calculates the sine or cosine of an angle θsupplied in radians. The absolute value of the argument must not exceed 7.9999999. Before calculating the function, the subroutine reduces the angle θ to an angle β whose absolute value in radians is $\leq$1.75. A 14th-degree Taylor series is used to calculate the function.

The program `B1-10.1` is a standard LGP-21 subroutine. The angle argument must be in the accumulator when the subroutine is called. The function value is in the accumulator@1 when the function returns.

Memory usage

Memory for program	1 track
Buffer usage	Sectors 8 and 49 of track 63

Input

The input variable for B1-10.1 is an angle θ, whose function value is to be calculated. θ must be in the accumulator@3 and have a value <= 7.9999999 when the call to the subroutine occurs. The subroutine reduces the angle to an equivalent angle β with an absolute value <= 1.75.

Calling Sequence

Sine

Location	Instruction	Address	Description
$\alpha - 1$	`B`	θ	; Load θ@3 into the accumulator
α	`R`	`L+16`	; Store return address at end of sub
$\alpha + 1$	`U`	`L`	; call the sine routine

This is a standard LGP-21 function call; the argument is loaded into the accumulator before the call, and the result is returned in the accumulator.

Cosine

Location	Instruction	Address	Description
$\alpha - 1$	`B`	θ	; Load θ@3 into the accumulator
α	`R`	`L+16`	; Store return address at end of sub
$\alpha + 1$	`U`	`L+45`	; call the cosine routine

The calling sequence is exactly the same as for the sine, except that we jump to the cosine entry point to the subroutine, 45 words past the start of the subroutine.

Output

On return, the function value with the appropriate sign is in the accumulator@1.

OPERATION

- **Accuracy:** The error in radian measure is less than 8 x 10^{-9}.
- **Time required:** approx. 1200 ms.
- **Program format:** the tape contains the program in two ways:
  - Relocatable, hexadecimal, valid for J1-10.0 (section 8.1).
  - Relocatable, decimal, in coding sheet format.
- **Required input/output devices:** None.
- **Overflow:** Ignored and not reset.

2.3 Arctangent 1

B1-11.0

PURPOSE

`B1-11.0` calculates the arctangent of any number less than 512. The returned value is signed and expressed in degrees. A 15th-degree polynomial is used to approximate the function.

`B1-11.0` is a standard LGP-21 subroutine. The argument must be in the accumulator; the arctangent value is in the accumulator on return to the main program. [Translator note: see section below on result scaling.]

This is a standard LGP-21 function call: the argument is loaded into the accumulator before the call, and the result is returned in the accumulator.

Memory usage

Memory for program	1 track
Buffer usage	Sectors 4, 5, 6, 7, 8, 9, 10, 13, 50, and 51 of track 63

Input

The input variable for `B1-11.0` is the value N@9 whose arctangent is to be computed. This value must be loaded into the accumulator before the subroutine is called.

Calling sequence

Location	Instruction	Address	Description
$\alpha - 1$	`B`	N	; Load N@9 into the accumulator
α	`R`	`L+51`	; Store return address at end of sub
$\alpha + 1$	`U`	`L`	; call the arctangent routine

This is a standard function call. Load the argument into the accumulator before the call; the value is returned in the accumulator.

Output

Upon returning to the main program, the signed principal value of the arctangent in degrees is in the accumulator@9. The absolute value will be between 0° and 89.90°.

OPERATION

- **Accuracy:** The absolute error is no greater than 5 x 10^{-7} degrees.
- **Time required:** 950 ms.

- **Program format -** the tape contains the program in two ways:
  - Arbitrarily relocatable hexadecimal, suitable for J1-10.0 (section 8.1).
  - arbitrarily relocatable decimal, in coding sheet format.
- **Required input/output devices:** None.
- **Overflow:** Neither considered at input nor reset at output.

REMARKS

A slight modification to the subroutine call can be made to allow larger input values whose arctangents are closer to 90° (absolute) by changing the contents of memory locations $\alpha + 56$ and $\alpha + 59$ to rescaled values of 1 and 2.

Memory location	Contents (old)	Contents (new)
`L+56`	1@9	1@q
`L+59`	2@9	2@q

The argument passed in the accumulator must be rescaled as well to the same q scaling value when jumping to the subroutine.

Translator's notes

Arctangent calculations near 90 degrees

The q value above is an LGP-21 fixed-point scaling factor[8], allowing the routine to process much larger numbers and yield accurate results for angles close to 90°. Unlike modern mathematical functions, which seamlessly handle scaling or accept floating-point infinities, or which permit passing more than one parameter to a subroutine(!), this function requires the programmer to adjust the input boundary parameters by patching the constants inside the function itself.

In the default configuration (@9), the maximum value the machine can physically represent is $2^9 - 1$ or 511.999999. As an angle approaches 90°, its tangent approaches infinity. Since the arctangent is the inverse function, this means that your application cannot use the @9 constants if the input values need to be larger than 511; calculating with the @9 constants will overflow.

Changing the scaling of the patched values (and of the incoming parameter!) to a larger scaling factor q, of 10 or greater, expands the range of the input. We still cannot represent mathematical infinity, but we can process much larger input values, allowing us to get significantly closer to a true 90° output.

[8] see 1.1

Warnings

The subroutine does not validate that the q value used for the patchable constants and your input argument match! If you patch the internal constants at $\alpha + 56$ and $\alpha + 59$ to use a custom scale factor (say, @12) , you must scale your input argument to that exact same scale factor before calling the routine. Cross-scaled parameters and constants will silently yield erroneous results and possible overflow conditions.

Because the word size is a fixed 31 bits, increasing q to accommodate larger integers directly reduces the number of bits available to the fractional side of the word. If you push q too high, you will lose precision on return values for smaller input values.

2.4 Arctangent of a Vector 1

B1-11.1

PURPOSE

The program `B1-11.1` calculates θ such that, given x and y, the following holds:

$$y = \sqrt{x^2 + y^2} \sin\theta \tag{1}$$

$$x = \sqrt{x^2 + y^2} \cos\theta \tag{2}$$

The x and y values may be positive or negative; the function operates correctly in all quadrants of the cartesian plane. The values x and y must be given at the same q. The result represents a partial result that can be converted into degrees or degrees of arc.

`B1-11.1` uses a standard LGP-21 subroutine call, but the y value must be stored in the subroutine's designated parameter storage, and x must be in the accumulator prior to the call. Upon returning to the main program, the result will be in the accumulator@1.

Memory usage

Memory required for program	2 tracks, 17 sectors
Buffer usage	None

Input

The input variables for the program are the values x and y , which must have the same q. y must be stored in the subroutine, and x must be in the accumulator when the subroutine is called.

Calling Sequence

Location	**Instruction**	**Address**	**Description**
$\alpha - 3$	`B`	Y	; Load Y@q into the accumulator
$\alpha + 2$	`C`	`L+34`	; save into subroutine parameter storage
$\alpha + 1$	`B`	`X`	; load X@q into the accumulator
α	`R`	`L+10`	; set return address
$\alpha + 1$	`U`	`L`	; call the arctangent routine

Store y in the designated parameter storage, `L+34` (any operation that puts it there is fine). Load x into the accumulator (again, any operation accomplishing this is fine). Remember that x and y must be scaled to the same q value.

Output

On return from the subroutine, the accumulator@1 will contain the result. To convert to degrees, multiply this value by 180; to convert to radians, multiply the value by π.

Operation

- **Accuracy:** No error exceeds +4 x 10^{-8} in the partial result.
- **Time required:** approx. 1160 ms.
- **Program format -** the tape contains the program in two versions:
  - Relocatable hexadecimal, valid for J1-10.0 (section 8.1).
  - Relocatable decimal, in coding sheet format.
- No input/output devices are required.
- **Overflow:** Ignored at subroutine call, and not reset at exit.

2.5 arcsin/arccos 1

B1-12.0

PURPOSE

The program `B1-12.0` calculates either the arcsine or the arccosine of a given number from -1 $\leq N \leq$ 1. The signed return value is expressed in degrees. A 7th-degree polynomial is used for approximation.

`B1-12.0` is a standard LGP-21 subroutine. When jumping to the subroutine, the argument must be in the accumulator, and the return address must be stored in the subroutine. Upon returning to the main program, the desired function value is in the accumulator@9.

Memory usage

Program storage	2 track, 32 sectors
Buffer usage	Sectors 12, 16, 18, 19, 20, 21, 23, 24, 28 of Track 63

Input

The value N@1 , with -1 $\leq N \leq$ 1 must be in the accumulator when the function is called.

Calling Sequence

Arcsin

Location	Instruction	Address	Description
$\alpha - 1$	`B`	N	; Load N@1 into the accumulator
α	`R`	`L+21`	; set return address
$\alpha + 1$	`U`	`L`	; call the arcsin routine

Load N into the accumulator, set the return address at `L+21` , and jump to `L` to compute the arcsin. Mainline execution continues after the jump.

Arccos

Location	Instruction	Address	Description
$\alpha - 1$	`B`	N	; Load N@1 into the accumulator
α	`R`	`L+21`	; set return address
$\alpha + 1$	`U`	`L+211`	; call the arcsin routine

Load N into the accumulator, set the return address at `L+21` , and jump to `L+211` to compute the arccos. Mainline execution continues after the jump.

Output

Output is returned in the accumulator@9 in radians; the value of the function is - $\pi/2$ to $\pi/2$ for arcsin, and 0 to π for arccos.

Notes

Since calculating the arcsine or arccosine requires calculating a square root, a program for this has been included in this subroutine. It can be used independently with the following calling sequence:

```
α − 1   B   value    ; load value with an even q
                     ; into the accumulator
                     ; (see translator notes)
α       R   L+150    ; set return address
α + 1   U   L+100    ; call the square root routine
```

The square root is returned in the accumulator. Note that the q value of the result is *half* of the argument's q value.

Translator's note

There's a lot hiding inside that throwaway "the q value of the result is half of the argument's q value". You may also be wondering why the q value of the argument *must* be even, especially since we're doing all this math in @1's and @9's.

This all goes back, again, to the fractional fixed-point math of the LGP-21. In modern floating-point math, the hardware (or software) manages all the exponents for you, but on the LGP-21, something interesting happens when we take the square root of a scaled number.

If you have a value X scaled to factor q, its internal integer representation in the machine is actually $X = x \cdot 2^q$. When you take the square root of that internal machine representation, look at what happens to the exponent:

$$\sqrt{X} = \sqrt{x \cdot 2^q} = \sqrt{x} \cdot 2^{q/2} \tag{3}$$

If your scale factor q is an odd number (like the standard @1 or the @9 used elsewhere in these library functions), then $q/2$ is a fraction (4.5)! You cannot shift a binary word by 4.5 bits, meaning that you can't make the result of the square root operation commensurate to any q.

Forcing the programmer to provide an input with an even q (like @2, @8, or @18) means that the resulting scale factor after the square root function finishes will be an integer (@1, @4, or @9, respectively) and that the result will be valid.

3 Logarithmic Functions

3.1 Exponential Function 1

B3-10.0

PURPOSE

The program `B3-10.0` calculates the exponential function K^x, where K = 2, e, or 10 and $x : -1 \leq x \leq 1$. `B3-10.0` expects x in the accumulator, and returns K in the accumulator.

Memory usage

Program storage	63 sectors
Buffer usage	None

Input

The input value x must be scaled to $-1 \leq x \leq 1$ and have a q @1.

Calling Sequence

Each of the exponential functions has a separate entry point into the subroutine:

2^x

```
α − 1   B   N      ; load N@1 into the accumulator
α       R   L+9    ; set return address
α + 1   U   L      ; call the 2^x routine
```

e^x

```
α − 1   B   N      ; load N@1 into the accumulator
α       R   L+9    ; set return address
α + 1   U   L+2    ; call the ex routine
```

10^x

```
α − 1   B   N      ; load N@1 into the accumulator
α       R   L+9    ; set return address
α + 1   U   L+3    ; call the 10^x routine
```

Each of the entry points is a standard LGP-21 subroutine call; mainline execution resumes directly following the unconditional jump to the subroutine.

Output

On return, the function value is in the accumulator@4.

Notes

- **Accuracy:** No error is greater than 5 x 10^{-8}.
- **Time required:** approx. 775 ms.
- **Program format -** the tape contains the program in two versions:
  - Relocatable hexadecimal, valid for J1-10.0 (section 8.1)
  - Relocatable decimal, in coding sheet format.
- **Required input/output devices:** None.
- **Overflow:** The program neither checks overflow upon input nor resets it.

Translator notes

While the input to these functions is strictly restricted to @1, the routine returns the final calculated value in the accumulator scaled to @4. This choice of scale factor represents a best-case compromise with the LGP-21's fixed-point architecture.

Because the maximum possible input value to any of these functions is 1.0, the maximum possible outputs are predetermined:

$$\begin{aligned} 2^1 &= 2.0 \\ e^1 &= 2.718281828\ldots \\ 10^1 &= 10.0 \end{aligned}$$

Recalling that the representation of the largest possible value for any of these functions as an integer is 10, or 1010_2 — four bits in binary — this means we need to have four bits in front of the binary point, but not more, to represent the maximum value of 10.0: a q of @4.

If we instead chose to return the data at a standard library scale like @9, this would throw away bits on the right side of the binary point to provide space to on the left-hand side of the binary point to represent values much larger that we would ever need. If, on the other hand, we chose to use @1, this would limit the return value's maximum to 1.99, causing an overflow on 2^1 and most positive e^x and 10^x calculations.

By standardizing the output to @4, `B3-10.0` allows the maximum possible value (10.0) for any of these functions to be returned while preserving an optimal 26 bits of precision for the fractional portion of the result for smaller numbers.

3.2 Log 1

B3-11.0

PURPOSE

The program `B3-11.0` calculates the logarithm of a positive number to the base 2, e, or 10. A 7th-degree polynomial is utilized as the approximation method. `B3-11.0` uses an altered multi-argument LGP-21 function call sequence. The function argument must reside in the accumulator, but the two words immediately following the jump to the subroutine must contain the q of the argument and the desired base of the logarithm. The return value is in the accumulator on return.

STORAGE REQUIREMENTS

Program Space: 1 track, 58 words
Temporary Storage: None

INPUT

The required input consists of:

1. N: The positive number whose logarithm is to be calculated.
2. Q: The q factor of N.
3. K: The desired base of the logarithm.

N must be a positive number greater than zero ($N > 0$) and must reside in the accumulator with a positive q factor when branching to the subroutine.
The q factor must lie within the range $0 \leq q \leq 31$. In the calling sequence, Q is stored in the sector portion of a `Z` instruction that immediately follows the branch instruction.
K, the desired base of the logarithm, is also stored in the sector portion of a `Z` instruction.

- For $\log_2 N$, $K =$ `00`
- For $\log_e N$, $K =$ `01`
- For $\log_{10} N$, K can take any value other than `00` or `01`.

The `Z` instruction containing K is the final entry in the calling sequence.

CALLING SEQUENCE

```
α − 1   B   N       ; load N@1 into the accumulator
α       R   L+24    ; set return address
α + 1   U   L       ; call the log routine
                    ; The following parameter constants
                    ; are effectively scaled at @29
α + 2   Z   00Q     ; The q value of N
α + 3   Z   00K     ; Desired base K
```

As usual, any instruction sequence leaving N in the accumulator prior to the call is fine as long as it leaves N there with a positive q. The instruction at location α stores the address $(\alpha + 2)$ of the `Z` instruction containing Q into the subroutine's internal tracking mechanism, giving the subroutine the address of Q and K.[9].

Execution resumes following the second parameter constant upon return from the subroutine.

OUTPUT

Upon the return branch to the main program, the calculated logarithm to the desired base resides in the accumulator@6.

OPERATING DETAILS

1. **Accuracy:**
   Any potential error is less than 3×10^{-8}.

2. **Execution time:**
   Approximately 1350 ms.

3. **Program input:**
   The program tape contains the routine in two formats:

   - Relocatable hexadecimal, valid for J1-10.0 (section 8.1)
   - Relocatable decimal, in coding sheet format.

4. **Required input/output devices:**
   None.

5. **Overflow:**
   Overflow is neither checked upon entry to the subroutine nor reset upon exit.

[9]Translator's note: the subroutine figures out the return address by taking the address supplied by the `R` instruction and adding 2 to skip over the two parameter words. It then updates a `U` instruction in *itself* to jump back to the proper return address

6. **Error Stops:**

Memory Location	Meaning and Remedy
`main program+8`	Argument N is zero or negative. The accumulator contains the value $N-$(`1@30`). To continue, deposit a corrected value, and jump manually via the console to resume execution.

Translator's Notes

To a modern programmer accustomed to software exceptions and terminal logs, or one not quite as modern who's used to core dumps, the behavior of `B3-11.0` upon encountering an invalid argument ($N \leq 0$) might seem strange. The program halting at location `L+8` is clear enough; that leaves execution realtively close to where the error occurred. But why set the accumulator to $N - $(`1@30`)?

`1@30` denotes the integer value 1 scaled to a q factor of 30, which places a single `1` bit at Bit 30 (representing a quantitative value of 2^{-30}). This is the next-to-last usable bit in the word.

Let's say a programmer accidentally passed a negative fraction; for example, $N = -0.5$ at a q of 1. This value natively resides in the accumulator as:

`11000000000000000000000000000000`

When the error trap, occurs, the program subtracts 1@30 from the argument and leaves it in the accumulator. Subtracting a bit so low in the word forces a massive binary borrow chain to propagate all the way up to the first available `1` bit at the MSB side:

```
  11000000000000000000000000000000  (N = -0.5@1)
- 00000000000000000000000000000010  (1@30)
------------------------------------
  10111111111111111111111111111110  (Result remaining in Accumulator)
```

Remember that the LGP-21 doesn't have a big console with a lot of lights; it just has a small oscilloscope tube with a custom mask that displays the accumulator, the current address, and the instruction to be executed, painting a visual representation of the bits. A `0` bit is drawn at the baseline, while a `1` bit is drawn higher. The LGP-21 runs slowly enough that this does a pretty good job os showing the machine status continuously.

When this subroutine executes its error stop, the oscilloscope will be displaying a static image of the current contents of the accumulator. Reading from left to right (Bit 0 to Bit 31), the operator would see:

- A single **HIGH** dash at the far left edge (Bit 0, confirming a negative sign).
- A single **LOW** dash immediately following it (Bit 1, dropped to 0 by the borrow).

- An unbroken, massive **wall of 29 HIGH dashes** stretching across the screen (Bits 2 through 30, flipped to 1).
- A final **LOW** dash at the absolute right margin (Bit 31, remaining 0).

This striking visual pattern, a solid block of high dashes bracketed by a low one on the left, serves as an immediate, non-textual diagnostic flag. A seasoned operator could glance at the oscilloscope screen from across the room and instantly diagnose that the logarithm routine had been fed an illegal negative vector, making it unnecessary to print or punch memory contents to debug the issue.

4 Root Extraction

4.1 Square Root 1

B4-10.0

PURPOSE

The program `B4-10.0` calculates the square root of a positive number. The number must reside in the accumulator at an even q factor. The program utilizes Newton's method to solve the equation:

$$0 = x^2 - N \tag{4}$$

by means of successive approximations:

$$x_{i+1} = x_i + \frac{1}{2}\left(\frac{N}{x_i} - x_i\right) \tag{5}$$

The program operates as a subroutine and is invoked via a calling sequence in the main program. Upon branching to the subroutine, the argument must reside in the accumulator, and the return address must be stored within the subroutine. Upon return to the main program, the calculated square root is held in the accumulator.

STORAGE REQUIREMENTS

Program Space: 51 words
Temporary Storage: None

INPUT

The input variable for "Square Root 1" is N, the positive number whose square root is to be calculated.
N must reside in the accumulator with an even q factor when branching to the subroutine.

CALLING SEQUENCE

Location	Instruction	Address	Description
$\alpha - 1$	`B`	`L`	Bring N into the accumulator at an even q.
α	`R`	`L+50`	Store the return address in the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to the subroutine entry.

This function uses a standard LGP-21 function call. The only notable factor is that N must have an even q.[10]

[10] See for why.

OUTPUT

Upon the return branch to the main program, the square root of the argument N resides in the accumulator at $q_N/2$. For example, if N is `@24`, then $\sqrt{N}$ will be `@12`.

OPERATING DETAILS

1. **Accuracy:**
   No error is greater than 2^{-30}.

2. **Execution Time:**
   Approximately 775 ms.

3. **Program input:**
   The program tape contains the routine in two formats:

   - Relocatable hexadecimal, valid for J1-10.0 (section 8.1)
   - Relocatable decimal, in coding sheet format.

4. **Required Input/Output Units:**
   None.

5. **Overflow:**
   Overflow is ignored and is not reset upon return.

6. **Error Stops:**

Memory Location	Meaning and Remedy
`L+24`	N is negative. After pressing the START button, the accumulator is cleared, and control returns directly to the main program.

4.2 n-th Root

B4-11.0

PURPOSE

The program `B4-11.0` calculates the n-th root of any non-negative value. The exponent n must be an integer such that $2 \leq n \leq 20$. The results are calculated using an iterative method according to the formula:

$$y_{i+1} = y_i + \frac{1}{n}\left(\frac{A}{y_i^{n-1}} - y_i\right) \tag{6}$$

where y_i represents the i-th approximation of the root.
`B4-11.0` uses a calling sequence similar to **??**, except it has only one secondary parameter. The number A whose root is to be taken is loaded into the accumulator with an appropriate scale factor (see below), and the degree of the root n is stored in a `Z` instruction following the `R` that calls the subroutine. On return to the main program, the desired root resides in the accumulator@q/n.

STORAGE REQUIREMENTS

Program Space: 1 track (64 words)
Temporary Storage: Sectors 01, 02, 04, 05, 44, 53, 54, and 60 of track 63.

INPUT

The input variables for `B4-11.0` are A, the non-negative value whose root is to be calculated, and n, the integer specifying the degree of the root ($2 \leq n \leq 20$). Upon branching to the subroutine, A must reside in the accumulator scaled to a q factor that is exactly divisible by n.[11]

CALLING SEQUENCE

Location	Instruction	Address	Description
$\alpha - 1$	`B`	A	Bring A into accumulator at a q divisible by n.
α	`R`	`L+16`	Store the parameter address in the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to the subroutine entry.
$\alpha + 2$	`Z`	`08nn`	Parameter n encoded as a decimal sector address.

The instruction at location $\alpha - 1$ brings the value A into the accumulator at a q factor that is divisible by n. (Any instruction sequence that achieves this result is permissible.)

[11] This is similar to the requirement that a number whose square root is being taken is divisible by two; however, n only needs to divide A's q factor exactly to be acceptable. As an example, if we had A@12, then the square, cube, fourth, and sixth roots of A could all be taken, as all are even divisors of 12.

The instruction at location α stores the address (α+2) containing the parameter instruction for n within the subroutine, and execution resumes at $\alpha + 3$.[12]

OUTPUT

Upon the return branch to the main program, the n-th root of the argument resides in the accumulator scaled to a q factor of q_A/n. For example, if $n = 3$ and $q_A = 12$, the result $\sqrt[3]{A}$ will have a q of `@4`.

OPERATING DETAILS

1. **Accuracy:**
   The result is guaranteed accurate to eight decimal places.

2. **Execution time:**
   Between 2 and 60 seconds. Execution times are significantly longer when calculating high-degree roots of small values.

3. **Required input/output devices:**
   None.

4. **Overflow:**
   Overflow is ignored upon entry and is not reset upon exit.

5. **Error stops:**

Memory Location	Meaning and Remedy
`L+61`	The requested root is an imaginary value (i.e., an even-degree root of a negative number). Pressing the START button clears the accumulator and returns execution directly to the main program.

[12] The subroutine takes the parameter address supplied via the `R` instruction, recalculates the return address, and adjusts its own code to properly return.

5 Matrix Operations

5.1 Matrix Programs 1

The program `D1-10.0` is a collection of five routines, all of which utilize the Floating-Point Interpretive System 1 (`H1-11.0`). This suite consists of the following programs:

Program	SC Number	Title
`D1-11.0`	2044	Matrix Inversion 1
`D1-12.0`	2045	Matrix-Vector Multiplication 1
`D1-13.0`	2046	Matrix Multiplication 1
`D1-14.0`	2047	Matrix Addition/Subtraction 1
`D1-15.0`	2048	Matrix Transposition 1

TRANSLATOR NOTE

The 6.1 floating-point interpreter provides a software floating-point implementation for the LGP-21, and is used extensively in the matrix routines.

5.2 Matrix Inversion 1

D1-11.0

PURPOSE

`D1-11.0` utilizes the Floating-Point Interpretive System 1 (`H1-11.0`) to replace the elements of a square matrix A with the elements of its inverse, $A^{-1} = B$. The input data must consist of the elements of a square, non-singular matrix whose rank is greater than 1, formatted as floating-point values. The total number of matrix elements must not exceed the available memory space.

"Matrix Inversion 1" operates as a subroutine invoked via a calling sequence in the main program. Prior to executing the calling sequence, an explicit exit from the Interpretive System must be performed. Furthermore, before branching to the subroutine, the starting address of the Interpretive System must be stored within the subroutine's internal tracking workspace. The calling sequence must supply both the rank of the matrix and its starting memory address. Upon return to the main program, the calculated inverse matrix occupies the exact memory locations originally held by the input matrix.

STORAGE REQUIREMENTS

Program Storage: 2 tracks (128 words)
Temporary Storage: None

INPUT

The input variables are the individual elements of an $N \times N$ matrix, stored sequentially in a row-major configuration (i.e., the first element of the second row immediately follows the final element of the first row). These data values must comply with the floating-point format dictated by Interpretive System 1. The matrix must be non-singular.

Although storing the original matrix elements inherently requires only N^2 memory locations, a continuous block of N^2+N sequential memory cells starting at base address M is strictly required for execution workspace.[13]

Prior to executing the branch to the subroutine, the starting address of the Interpretive System **??** must be stored into memory location `L+163`, where `L` represents the starting address of the Matrix Inversion code. This initialization can be performed manually by the operator immediately after loading the subroutine tape, or it may be handled programmatically by initialization instructions within the main program.

CALLING SEQUENCE

The calling sequence must provide the following parameters:

[13]The matrix inversion is implemented using a Gauss-Jordan elimination algorithm that requires a one-column working vector to track pivoting operations and column swaps as it performs the inversion in-place.

1. N: The rank (dimension) of the matrix, scaled at $q = 1$, with $N > 1$.
2. M: The starting memory address of the matrix, specified in decimal track/sector notation.

Both parameters N and M are packed into a single parameter word. If N is greater than 15, the packed configuration requires hexadecimal entry format.

Location	**Instruction**	**Address**	**Description**
$\alpha - 1$	`E`	`0000`	Exit from the Interpretive System.
α	`R`	`L+14`	Modify subroutine instruction with key word address.
$\alpha + 1$	`U`	`L`	Unconditional branch to subroutine entry.
$\alpha + 2$	…	…	Packed parameter word: N`@15` and M`@29`.[14]
$\alpha + 3$	—	—	Return to main program.

The `E` instruction at location $\alpha - 1$ is only required if the preceding instruction executed by the main program was an interpretive pseudo-instruction. The instruction at location α copies the address of the packed parameter word ($\alpha+2$) into the subroutine's entry loop.

The instruction at location $\alpha + 1$ executes an unconditional branch to `L`, the base address of the subroutine. Location $\alpha+2$ holds the packed descriptor word detailing N at a q of 15 and M at a q of 29. Location $\alpha + 3$ contains the next instruction of the main program, which receives control upon subroutine exit.

OUTPUT

The result of Matrix Inversion 1 is the inverted $N \times N$ matrix B, stored in row-major order starting at memory location M. The elements are held in continuous sequential memory locations such that element b_{21} immediately follows element b_{1n}, and so forth.

OPERATING DETAILS

1. **Program Input:**
   The subroutine tape is provided in two distribution formats:
   a) arbitrarily relocatable, hexadecimal format, compatible with `J1-10.0`; and
   b) arbitrarily relocatable, decimal coding sheet format.
2. **Required Input/Output Units:**
   None.
3. **Overflow:**
   Overflow does not disrupt the entry phase and is not reset upon subroutine exit.

4. **Error Stops:**

Memory Location	Meaning and Remedy
`L+759`	The matrix is singular. Mathematical inversion is impossible. Execution stops; do not attempt to bypass or resume.

REMARK

While Matrix Inversion 1 heavily depends on the Interpretive System during calculation, an exit sequence is executed internally prior to returning control to the main program; control returns to the main program in native hardware execution mode.

TRANSLATOR'S NOTES

Gauss-Jordan elimination is probably the most sophisticated algorithm usable for this purpose on the LGP-21, but is has some pitfalls. Because it performs a larger total number of arithmetic operations than, say, Gaussian elimination with back-substitution, it is even more vulnerable to subtractive cancellation and catastrophic loss of significance.

Subtractive cancellation occurs when you subtract two nearly identical numbers. The leading bits, which match, cancel each other out and leave zeros.

For instance, consider two numbers that match out to 25 bits:

$$\begin{array}{lll} 1.01101101101101101101101\,10 & 101 & \times 2^e \\ 1.01101101101101101101101\,10 & 011 & \times 2^e \\ \hline 0.00000000000000000000000\,00 & 010 & \times 2^e \end{array}$$

After the subtraction, the only remaining non-zero significant bit are the last two. The floating-point interpreter will normalize this result by shifting it left, which shifts in zeroes from the right side of the word. Because the original numbers only had 30 bits of precision, those newly-added lower bits are *garbage*! You now have a value where the error is as large as the signal itself.

In a classic Gauss-Jordan implementation, you aren't just clearing elements below the pivot; you are simultaneously clearing elements *above* the pivot to force the left side of the matrix to be a pure identity matrix. This requires roughly $\frac{n^3}{2}$ multiplications and subtractions compared to the $\frac{n^3}{3}$ required by Gaussian elimination. Every extra row operation is another opportunity for a subtractive cancellation event to add errors to your data.

If a pivot element becomes very small (usually due to an earlier subtractive cancellation), dividing the rest of the row by that tiny pivot scales up any existing truncation errors dramatically. When that row is subsequently subtracted from all the other rows, that amplified noise propagates through the entire matrix.

?? uses a couple of techniques to help prevent this:

- **Partial Pivoting**: The routine scans down the current column to find the element with the largest absolute magnitude and swaps that row to the

pivot position. This ensures that you are always dividing by the largest possible number, keeping the multipliers less than or equal to 1.0, which drastically decreases error amplification.

- **The L+759 Hard Stop**: If the largest available pivot is zero (or dangerously close to it under the interpreter's precision bounds), the routine realizes that it has hit a singular or ill-conditioned matrix where subtractive cancellation has wiped out all meaningful data. Rather than returning a useless result, it hits the brakes and halts.

5.3 Matrix-Vector Multiplication 1

D1-12.0

PURPOSE

This program uses the Floating-Point Interpretive System 1 (6.1). `D1-12.0` multiplies a matrix by a column vector and stores the resulting product vector in a designated set of memory locations.

The input consists of a matrix with i rows and j columns, and a column vector containing j elements. All data must be supplied in floating-point format. The matrix does not need to be square. The dimensions i and j must each be less than 64. The total number of elements across the matrix, vector, and product vector must not exceed the available memory space.

"Matrix-Vector Multiplication 1" operates as a subroutine invoked via a calling sequence in the main program. H1-11.0 must be memory-resident, but should be shut down via an explicit exit from the Interpretive System before the subroutine is called.

The calling sequence must supply:

1. the starting address of the Interpretive System,
2. the dimensions and starting address of the matrix,
3. the starting address of the column vector, and
4. the starting address for the destination product vector. Upon return to the main program, the calculated product vector resides in the specified memory cells.

STORAGE REQUIREMENTS

Program Storage: 1 track, 32 sectors (96 words)
Temporary Storage: None

INPUT

The input variables are:

1. The elements of a matrix with i rows and j columns, stored sequentially in row-major order (i.e., the first element of the second row immediately follows the last element of the first row).
2. The elements of a column vector stored in consecutive memory cells. The number of elements in the vector must equal the number of columns (j) in the matrix.

The data must be formatted in a style compatible with Floating-Point Interpretive System 1. The dimensions i and j must each be strictly less than 64, though they do not need to be equal. The cumulative number of elements from the matrix, input vector, and output vector must not exceed available memory capacity.

CALLING SEQUENCE

The following information must be supplied by the calling sequence in the exact order specified below:

1. F: The starting address of the Interpretive System in decimal track/sector notation.
2. M: The starting address of the matrix in decimal track/sector notation.
3. The dimensions of the matrix, with row count i formatted as a decimal track address and column count j formatted as a decimal sector address.
4. V: The starting address of the column vector in decimal track/sector notation.
5. P: The starting address for the destination product vector in decimal track/sector notation.

A sample calling sequence is illustrated below, where `L` denotes the starting address of this subroutine.

Location	**Instruction**	**Address**	**Description**
$\alpha - 1$	`XE`	`0000`	Exit from the Interpretive System.
α	`R`	L_0	Store the Interpretive System entry address in subroutine.
$\alpha + 1$	`U`	L_0	Unconditional branch to subroutine entry.
$\alpha + 2$	`Z`	F_0	Starting address of the Interpretive System.
$\alpha + 3$	`Z`	`M`	Starting address of the matrix.
$\alpha + 4$	`Z`	`iijj`	Matrix dimensions: rows (`ii`) and columns (`jj`).
$\alpha + 5$	`Z`	`V`	Starting address of the column vector.
$\alpha + 6$	`Z`	`P`	Starting address of the destination product vector.
$\alpha + 7$	—	—	Return to main program.

The `E` instruction at location $\alpha - 1$ is only mandatory if the immediately preceding instruction was an Interpretive System pseudo-instruction. The instruction at location α copies the address of the Interpretive System's key word parameter $(\alpha + 2)$ into the subroutine setup loop.
The instruction at location $\alpha + 1$ executes an unconditional branch to `L`, the base address of the subroutine. Locations $\alpha + 2$ through $\alpha + 6$ function as inline data parameters defining the operational addresses and dimensions. Location $\alpha + 7$ contains the first instruction of the main program to be executed after exiting the subroutine.

OUTPUT

The individual elements of the calculated product vector are saved sequentially in consecutive memory cells, beginning at address P.

OPERATING DETAILS

1. **Program Input:**
   The subroutine tape is distributed in two formats:
   a) arbitrarily relocatable, hexadecimal format, compatible with `J1-10.0`; and
   b) arbitrarily relocatable, decimal coding sheet format.

2. **Required Input/Output Units:**
   None.

3. **Overflow:**
   Overflow is ignored during the execution initialization phase and is not reset upon subroutine exit.

NOTES

The subroutine relies heavily on the Interpretive System for processing all floating-point vector math. However, an exit sequence from the interpreter is automatically executed prior to returning control to the main program; control is handed back to the main program in native hardware execution mode.

TRANSLATOR'S NOTE

This is the first function to use the extended calling sequence that looks like this, documented at **??**.

Speculating slightly, because I have not seen/dumped the code in question, this is my best guess on how it works.

- The `R` instruction in the main program stores the return address ($\alpha + 2$) into L_0. Note that the current address + 2 is the start of the parameter list to be passed to the subroutine.
- The `U` instruction at $\alpha + 1$ jumps to L_0.
- A `B` instruction at L_0 loads $\alpha + 2$, the parameter list's address into the accumulator.
- The subroutine adds 5 to the accumulator to get the actual return address (the word following the parameter list).
- The subroutine then stores the return address into a `U` instruction that it will use for the return to the main program.

This lets the `B` instruction at L_0 do double duty; it serves to save the "return address" (actually the parameter list address for the call) and to set up the calculation of the real return address by loading it into the accumulator.

5.4 Matrix Multiplication 1

D1-13.0

PURPOSE

The program utilizes the Floating-Point Interpretive System 1 (`H1-11.0`). `D1-13.0` calculates the product matrix C resulting from the matrix equation:

$$AB = C \tag{7}$$

The product of an $n \times m$ matrix A and an $m \times p$ matrix B is an $n \times p$ matrix C. The input consists of the individual elements of both matrices A and B, which must be supplied in floating-point format. The matrix dimensions must be strictly less than 64.[15] The cumulative number of elements across matrices A, B, and the output matrix C must not exceed the available memory space.

"Matrix Multiplication 1" operates as a subroutine invoked via a calling sequence from the main program. Prior to executing the calling sequence, an explicit exit from the Interpretive System must be performed. The calling sequence must supply:

1. the starting address of the Interpretive System,
2. the dimensions and starting address of matrix A,
3. the dimensions and starting address of matrix B, and
4. the starting memory address for the destination product matrix C.

Upon return to the main program, the calculated product matrix is in the target memory locations.

STORAGE REQUIREMENTS

Program Storage: 2 tracks, 50 sectors (178 words)
Temporary Storage: None

INPUT

The input variables are:

1. The elements of matrix A ($n \times m$), stored row-by-row in continuous sequential cells.
2. The elements of matrix B ($m \times p$), stored row-by-row in continuous sequential cells. The number of rows in matrix B must precisely equal the number of columns in matrix A.

[15] **Translator's note:** The limit of 64 is an architectural artifact of the LGP-21's memory, which is physically structured as 64 tracks of 64 sectors each. Capping dimensions at 64 allows index calculation loops to stay bound within simple track-selection logic without risking multi-track pointer overflow.

All matrices must be stored in a row-major configuration, meaning the first element of the second row immediately follows the terminal element of the first row.

The data must conform to the floating-point layout required by 6.1. Dimensions must be less than 64, and total memory overhead requires a block of $(n \times m) + (m \times p) + (n \times p)$ cells to accommodate the inputs and the trailing results workspace.

CALLING SEQUENCE

The calling sequence must provide the following information in the exact sequential order specified below:

1. F: The starting address of the Interpretive System in decimal track/sector notation.
2. A: The starting address of matrix A in decimal track/sector notation.
3. The size of matrix A, with row count n formatted as a decimal track address and column count m formatted as a decimal sector address.
4. B: The starting address of matrix B in decimal track/sector notation.
5. The size of matrix B, with row count m formatted as a decimal track address and column count p formatted as a decimal sector address.
6. C: The starting address of the destination storage area for the product matrix in decimal track/sector notation.

A template for the calling sequence is illustrated below, where L_0 denotes the starting address of this subroutine.

Location	**Instruction**	**Address**	**Description**
$\alpha - 1$	`E`	`0000`	Exit from the Interpretive System.
α	`R`	L_0	Store parameter keyword tracking address in subroutine.[16]
$\alpha + 1$	`U`	L_0	Unconditional branch to subroutine entry.
$\alpha + 2$	`Z`	F_0	Starting address of the Interpretive System.
$\alpha + 3$	`Z`	`A`	Starting address of matrix A.
$\alpha + 4$	`Z`	`nnmm`	Dimensions of matrix A: rows (`nn`) and columns (`mm`).
$\alpha + 5$	`Z`	`B`	Starting address of matrix B.
$\alpha + 6$	`Z`	`mmpp`	Dimensions of matrix B: rows (`mm`) and columns (`pp`).
$\alpha + 7$	`Z`	`C`	Starting address of destination product matrix C.
$\alpha + 8$	—	—	Return to main program (execution resumes here).

The `E` instruction at location $\alpha - 1$ is only mandatory if the preceding main program operation was an Interpretive System pseudo-instruction.

OUTPUT

The result of “Matrix Multiplication 1” is the product matrix C, structured with n rows and p columns, stored row-by-row in continuous sequential memory

cells starting at location C. Element c_{21} immediately follows element c_{1p}, and so forth.

OPERATING DETAILS

1. **Program Input:**
   The subroutine tape is distributed in two formats:
   a) arbitrarily relocatable, hexadecimal format, compatible with `J1-10.0`; and
   b) arbitrarily relocatable, decimal coding sheet format.

2. **Required Input/Output Units:**
   None.

3. **Overflow:**
   Overflow is ignored during the entry configuration phase and is not reset upon subroutine exit.

4. **Error Stops:**

Memory Location	Meaning and Remedy
`L+241`	Inner matrix dimensions mismatch: the number of columns in matrix A (m) does not equal the number of rows in matrix B (m). Mathematical multiplication is undefined. Do not attempt to resume execution; the input data dimensions must be corrected.

NOTES

The subroutine utilizes the Interpretive System internally to calculate floating-point steps. However, an exit sequence from the interpreter is automatically executed prior to returning control to the main program; control is returned in native hardware mode.

5.5 Matrix Addition and Subtraction 1

D1-14.0

PURPOSE

The program utilizes the Floating-Point Interpretive System 1 (`H1-11.0`). `D1-14.0` adds or subtracts two matrices A and B and stores the resulting product matrix in a designated memory area.

The input consists of the elements of both matrices, each having i rows and j columns. All data must be supplied in floating-point format. The matrices do not need to be square. The dimensions i and j must both be strictly less than 64. The cumulative number of elements across matrices A, B, and the output matrix must not exceed the available memory space.

"Matrix Addition and Subtraction 1" operates as a subroutine invoked via a calling sequence in the main program. Prior to executing the calling sequence, an explicit exit from the Interpretive System must be performed. The calling sequence must supply:

1. the starting address of the Interpretive System,
2. the dimensions and starting addresses of the matrices,
3. the arithmetic operation desired (addition or subtraction), and
4. the starting memory address of the storage area for the destination matrix.

Upon return to the main program, the resulting matrix resides in the designated memory cells.

STORAGE REQUIREMENTS

Program Space: 1 track (64 words)
Temporary Storage: None

INPUT

The input variables are the individual elements of the two matrices A and B, each containing i rows and j columns. The elements of both matrices must be stored identically—either both in row-major order or both in column-major order—in consecutive memory cells.[17] This means that the first element of the second row immediately follows the last element of the first row (if stored row-by-row), or the first element of the second column immediately follows the last element of the first column (if stored column-by-column).

The data must comply with the floating-point layout required by Interpretive System 1. The matrices must be of identical dimensions. The values i and j

[17] This is significantly more flexible than the Matrix Multiplication routine (`D1-13.0`). Because addition and subtraction are purely element-by-element operations, the subroutine doesn't actually care about row vs. column orientation, as long as both inputs share the exact same structure. The execution loop simply scans linearly across both matrices.

must both be strictly less than 64. Although storing the input matrix elements inherently requires only $2(ij)$ sectors, a continuous block of $3(ij)$ memory cells is required across the system to accommodate both inputs and the resulting workspace.

CALLING SEQUENCE

The following information must be supplied by the calling sequence in the exact sequential order specified below:

1. F_0: The starting address of the Interpretive System in decimal track/sector notation.
2. A: The starting address of matrix A in decimal track/sector notation.
3. The size of the matrices, with row count i formatted as a decimal track address and column count j formatted as a decimal sector address.
4. The instruction `A` or `S`, depending on whether addition or subtraction is desired. This operation symbol is written directly in the instruction field; the starting address of matrix B is specified in the address field of the exact same word.[18]
5. C: The starting address of the destination storage area for the resulting matrix in decimal track/sector notation.

A template for the calling sequence is illustrated below, where L_0 denotes the starting address of "Matrix Addition and Subtraction 1".

Location	Instruction	Address	Description
$\alpha - 1$	`E`	`0000`	Exit from the Interpretive System.
α	`R`	L_0	Store parameter tracking address in the subroutine.
$\alpha + 1$	`U`	L_0	Unconditional branch to subroutine entry.
$\alpha + 2$	`Z`	F_0	Starting address of the Interpretive System.
$\alpha + 3$	`Z`	`A`	Starting address of matrix A.
$\alpha + 4$	`Z`	`iijj`	Dimensions of the matrices: rows (`ii`) and columns (`jj`).
$\alpha + 5$	`A/S`	`B`	Operation (`A` or `S`) and starting address of matrix B.
$\alpha + 6$	`Z`	`C`	Starting address of destination area for the resulting matrix.
$\alpha + 7$	—	—	Return to main program (execution resumes here).

The `E` instruction at location $\alpha - 1$ is only mandatory if the preceding main program operation was an Interpretive System pseudo-instruction.

OUTPUT

The individual elements of the resulting matrix C_{ij} are saved in floating-point format in consecutive memory cells, beginning at address C.

[18] Another memory-saving optimization: instead of wasting an entire word on an operation flag or setting up a parsing branch, the subroutine reads this parameter word at $\alpha + 5$ and injects it directly into its internal math loop as an executable instruction; the address of the second matrix B is *also* packed into this instruction; the instruction (`A` for Add, `S` for Subtract) is executed dynamically against the current matrix address via self-modifying code.

OPERATING DETAILS

1. **Program Input:**
   The subroutine tape is distributed in two formats:

   a) arbitrarily relocatable, hexadecimal format, compatible with `J1-10.0`; and

   b) arbitrarily relocatable, decimal coding sheet format.

2. **Required Input/Output Units:**
   None.

3. **Overflow:**
   Overflow is ignored during the entry initialization phase and is not reset upon subroutine exit.

REMARKS

The subroutine relies internally on the Interpretive System to process all floating-point math steps. The subroutine internally enables and disables the interpreter prior to returning control to the main program; control is returned in native hardware mode.

Either matrix A or B may be updated in place if desired. If an independent destination area is chosen, its memory bounds must not overlap with either A or B.

5.6 Matrix Transposition 1

D1-15.0

PURPOSE

The program `D1-15.0` transposes a square matrix stored in memory and saves the transposed results into designated cells. The input consists of the elements of a square matrix whose rank is strictly less than 32. Both fixed-point and floating-point data formats are supported. The total number of elements must not exceed the available memory space.
"Matrix Transposition 1" operates as a subroutine invoked via a calling sequence in the main program. Prior to executing the calling sequence, an explicit exit from the Interpretive System must be performed. The calling sequence must supply:

1. the rank and starting address of the matrix, and
2. the destination starting address for the transposed matrix.

Upon return to the main program, the transposed matrix resides in the specified memory locations.

STORAGE REQUIREMENTS

Program Space: 1 track, 58 sectors (122 words)
Temporary Storage: None

INPUT

The input variables are the elements of a square matrix, stored either row-by-row or column-by-column in consecutive memory cells. In a row-major configuration, the first element of the second row immediately follows the last element of the first row; in a column-major configuration, the first element of the second column immediately follows the last element of the first column.
The data values may be supplied either in standard raw fixed-point fractional format or in the floating-point format dictated by Interpretive System 1 (`H1-11.0`). The rank N must satisfy $2 < N \leq 31$. Although storing the matrix elements inherently requires only N^2 sectors, a maximum workspace block of $2N^2$ memory cells is required if the destination area is separate.

CALLING SEQUENCE

The following information must be supplied by the calling sequence in the exact sequential order specified below:

1. N: The rank (dimension) of the matrix, scaled at $q = 15$.
2. M: The starting memory address of the matrix, specified in decimal track/sector notation.

3. T: The starting memory address of the destination storage area for the transposed matrix in decimal track/sector notation.

Parameters N and M are packed into a single parameter word at location $\alpha + 2$. If N is greater than 15, the packed configuration requires hexadecimal entry format.
The source address M and destination address T may be identical (in-place transposition). However, if the transposed matrix is to be written to a different memory region, that region must not overlap with the original matrix block.

Location	Instruction	Address	Description
$\alpha - 1$	`E`	`0000`	Exit from the Interpretive System.
α	`R`	`L`	Store the parameter keyword tracking address in the subroutine.
$\alpha + 1$	`U`	`L`	Unconditional branch to subroutine entry.
$\alpha + 2$	…	…	Packed parameter word: N`@15` and M`@29`.
$\alpha + 3$	`Z`	`T`	Starting address for the transposed matrix.
$\alpha + 4$	—	—	Return to main program (execution resumes here).

The `E` instruction at location $\alpha - 1$ is only mandatory if the preceding main program operation was an Interpretive System pseudo-instruction.

OUTPUT

The individual elements of the transposed matrix are saved starting at location T. If the original matrix was stored row-by-row, the columns will be laid out row-by-row in the transposed matrix. Conversely, if the original matrix was stored column-by-column, the rows will be laid out column-by-column in the destination block.

OPERATING DETAILS

1. **Program Input:**
   The subroutine tape is distributed in two formats:
   a) arbitrarily relocatable, hexadecimal format, compatible with `J1-10.0`; and
   b) arbitrarily relocatable, decimal coding sheet format.

2. **Required Input/Output Units:**
   None.

3. **Overflow:**
   Overflow is ignored during the execution initialization phase and is not reset upon subroutine exit.

REMARK

Unlike the other routines in the matrix suite, this subroutine *does not* invoke or use the Floating-Point Interpretive System internally.[19]

[19]Because matrix transposition is a purely structural index manipulation (swapping element positions (i, j) with (j, i)), the subroutine does not perform any actual arithmetic on the matrix values themselves, so it's completely unnecessary to incur the massive overhead of the floating-point interpreter `H1-11.0`. We're just moving data, so it doesn't matter if it's native fixed-point integers or multi-word floating-point interpreter variables.

5.7 Complex Matrix Operations Interpreter

5.8 Complex Operations Interpretive System

H1-10.0

PURPOSE

The program `H1-10.0` enables the user to process algebraic expressions involving complex numbers in the form $(a + bi)$ as seamlessly as if they were standard real numbers. Additionally, the system provides integrated left and right logical bit-shifting capabilities. It also allows for dynamic address modification and loop-termination testing against a target end-address without ever needing to exit the interpretive execution loop.

STORAGE REQUIREMENTS

Program Space: 3 tracks (192 words)
Temporary Storage: None

INPUT

For every complex data element, both the real and imaginary parts must be stored at the identical binary scaling factor (q-factor) in consecutive memory cells. Specifically, the real part must reside at memory cell `ttss` and the imaginary part at cell `ttss+1`.
Because each complex number occupies two discrete memory locations, the subroutine implements simulated registers to facilitate instruction execution. These architecture registers consist of a simulated pseudo-accumulator located at cell `L+33` and a simulated address register located at cell `L+219`, where `L` denotes the starting base address of the subroutine. In addition to these active components, the program records the transient state of the physical hardware accumulator at cell `L+59`.
The interpretive engine processes 15 distinct operations. For all arithmetic commands, the address field `ttss` references the memory cell containing the real component, though the internal loop automatically processes both parts of the complex value. For all other non-arithmetic commands, `ttss` functions as a standard single-word memory address. Only the instructions explicitly detailed below are structurally valid within an interpretive instruction sequence.

1. Arithmetic Instructions

`Bttss` **(Bring):** The elements at locations `ttss` and `ttss+1` replace the contents of the pseudo-accumulator. The source memory remains unchanged.

`Attss` **(Add):** The elements at locations `ttss` and `ttss+1` are added to the pseudo-accumulator. The result replaces the contents of the pseudo-accumulator.

`Sttss` (Subtract): The elements at locations `ttss` and `ttss+1` are subtracted from the pseudo-accumulator. The result replaces the contents of the pseudo-accumulator.

`Mttss` (Multiply): The pseudo-accumulator is multiplied by the complex value stored at `ttss` and `ttss+1`. The product replaces the contents of the pseudo-accumulator.[20]

`Dttss` (Divide): The pseudo-accumulator is divided by the complex value stored at `ttss` and `ttss+1`. The quotient replaces the contents of the pseudo-accumulator.

`Httss` (Hold): The contents of the pseudo-accumulator are copied into memory cells `ttss` and `ttss+1`. The pseudo-accumulator remains unchanged.

`Cttss` (Clear): The contents of the pseudo-accumulator are copied into memory cells `ttss` and `ttss+1`, and the pseudo-accumulator is subsequently reset to zero.

2. Auxiliary Instructions

`XR00nn` (Shift Right): Shifts both components of the pseudo-accumulator right by `nn` binary places ($0 \leq$ `nn` ≤ 10).

`XP00nn` (Shift Left): Shifts both components of the pseudo-accumulator left by `nn` binary places ($0 \leq$ `nn` ≤ 10).

`Uttss` (Unconditional Jump): Transfers execution control. The next pseudo-instruction to be interpreted is fetched from cell `ttss`. This jump cannot be used to exit the interpreter; `ttss` must point to a valid complex pseudo-instruction.

`XE0000` (Exit): Explicit exit from the Interpretive System. The instruction immediately following `XE0000` is executed as a native hardware instruction.

3. Address Modification Instructions

`Ettss` (Set Address Register): The address portion of the word stored at cell `ttss` is loaded into the simulated address register. Memory remains unchanged.

`Ittss` (Increment Address): Increments the current value of the simulated address register by the value `ttss`.

`Yttss` (Inject Address): Embeds the current value of the simulated address register directly into the address portion of the instruction word at cell `ttss`. The address register itself and all other memory remains unchanged.

[20] **Translator's note:** A complex multiplication $(a + bi)(c + di)$ requires computing $(ac - bd) + (ad + bc)i$. Performing this in a single interpretive step saves a massive amount of main-program setup and temporary storage management.

`Zttss` (Test and Skip): Subtracts the current value of the simulated address register from the literal value `ttss`. If the result is exactly zero, the next pseudo-instruction is skipped (execution jumps to the second following word). If the result is non-zero, execution continues normally with the immediately following instruction.

CALLING SEQUENCE

A template for entering the interpretive loop is illustrated below, where `L` denotes the starting address of the interpreter.

Location	Instruction	Address	Description
α	`R`	`L`	Store parameter tracking address inside the interpreter.
$\alpha + 1$	`U`	`L`	Unconditional branch to interpreter entry.
$\alpha + 2$	…	…	First complex pseudo-instruction to be interpreted.
⋮	⋮	⋮	Sequence of complex operations.
$\alpha + n$	`XE`	`0000`	Exit from the Interpretive System.
$\alpha + n + 1$	…	…	First native hardware instruction (execution resumes here).

The `R` instruction at location α automatically captures the address of the initial pseudo-instruction ($\alpha + 2$) and saves it internally within the interpreter's program counter tracker.

OPERATING DETAILS

1. Execution Time The following table outlines the maximum execution durations required for each individual complex operation, measured in drum rotations and milliseconds.

Instruction	Drum Rotations	Time (ms)	Instruction	Drum Rotations	Time (ms)
`B`	11	561	`P`	17	1167^{21}
`A`	14	714	`U`	8	408
`S`	14	714	`XE` (Exit)	6	306
`M`	22	1122	`E` (Set Addr)	9	459
`D`	41	2091	`I`	6	306
`H`	14	714	`Y`	8	408
`C`	14	714	`Z` ($= 0$)	11	561
`R` (Shift R)	14	714	`Z` ($\neq 0$)	13	663

2. Program Input: The subroutine tape is distributed in two formats:

a) arbitrarily relocatable, hexadecimal format, compatible with `J1-10.0`; and

b) arbitrarily relocatable, decimal coding sheet format.

3. Required Input/Output Units: None.

4. Error Stops:

Location	Meaning	Remedy
`L+122`	Negative Instruction encountered (Illegal format).	Fatal error. Do not force resum
`L+135`	Invalid `N-` Type instruction variation.	Fatal error. Do not force resum
`L+154`	Invalid `T-` Type instruction variation.	Fatal error. Do not force resum

5. Overflow: Overflow status is ignored during interpretive entry execution loops and is not explicitly reset upon subroutine exit.

REMARKS

The first instruction inside the interpretive sequence (location $\alpha + 2$) must not be an `XE0000` command, as it would cause an immediate, unintended system exit before any operation can occur.

TRANSLATOR NOTES

The inclusion of index/address pseudoinstructions (`E`, `I`, `Y`, `Z`) elevates this interpreter from a simple complex-math math library to a self-contained environment. It can handle full multi-row matrix array traversal directly inside the complex domain without ever exiting the interpreter loop to do data manipulation and control flow.

6 Floating-Point Interpretive Systems

6.1 Floating Point Interpreter 1

H1-11.0

PURPOSE

The Floating-Point Interpretive System 1 allows the programmer to prioritize floating-point calculations over standard fractional fixed-point operations. Data values are supplied to the system as 7-digit integers multiplied by a power of 10; the interpretive system automatically scales the data during calculation and delivers the final results as 7-digit integers multiplied by a power of 10.

Although this is fundamentally a single-word interpretive system, two memory words are required for each number during calculation and output operations. The mantissa and exponent are automatically separated and reassembled by the system. Consequently, a precise prior knowledge of the order of magnitude of a problem's variables is not required. The system features 33 distinct instructions, which significantly improves both programming ease and computational accuracy.

The interpretive system consists of various specialized subroutines—Data Input, Data Output, and the calculation of transcendental functions—which can be utilized either as a comprehensive program package or individually. These subroutines are invoked via specific interpretive instructions, rendering standard main-program calling sequences unnecessary. However, the interpretive system engine itself and the companion "Fixed-Point to Floating-Point Conversion (and vice versa)" routine are accessed via a traditional machine calling sequence.

DOCUMENTATION INDEX

This manual section contains comprehensive descriptions of the following subroutines and specialized features of the interpretive system:

INTERNAL NUMBER FORMAT

Every floating-point number is held within memory inside a single cell as a split mantissa and exponent. Every mantissa is maintained in binary notation at a scaling factor of $q = 0$ and consists of an algebraic sign bit followed by 24 bits of precision. The most significant bit has a fractional value of 2^{-1}, meaning that all stored mantissas are strictly normalized.
The exponent is maintained at a scaling factor of $q = 30$ and consists of an algebraic sign bit and 5 bits of precision. The exponent represents the power-of-2 base by which the mantissa must be multiplied to obtain the actual represented numeric value. Expressed mathematically, the components represent the floating-point number as:

$$Z = M \cdot 2^E$$

Where:

— Z = The represented numerical value.

— M = The normalized binary fractional mantissa ($q = 0$).

— E = The binary integer exponent ($q = 30$).

For example, the value **3.75** would be represented in a computer memory word as:

`01111100000000000000000000000100`

Where:

— Mantissa ($q = 0$): $M = 0.9375$

— Exponent ($q = 30$): $E = 2$

— Verification: $Z = 0.9375 \cdot 2^2 = 3.7500$

The mantissa and exponent are treated by the system as two entirely distinct numbers. Therefore, a negative mantissa and a positive exponent can reside within the exact same memory word safely. The component parts of a number are complemented independently of one another.

For example, the negative value **-3.75** would be represented as:

`10001000000000000000000000000100`

Where:

— Mantissa ($q = 0$): $M = -0.9375$

— Exponent ($q = 30$): $E = 2$

— Verification: $Z = -0.9375 \cdot 2^2 = -3.7500$

As a further example, the fractional value **0.46875** in floating-point form would be represented as:

```
01111100000000000000000011111110
```

Where:

— Mantissa ($q = 0$): $M = 0.9375$

— Exponent ($q = 30$): $E = -1$

— Verification: $Z = 0.9375 \cdot 2^{-1} = 0.46875$

When values are loaded inside the simulated floating-point accumulator or the multiplication register, the numbers are structured so that they occupy **two discrete memory cells**, with the mantissa shifted one place to the right. Despite this physical expansion, the structural relationship between each mantissa and its respective exponent is rigorously maintained.

INTERNAL REGISTERS

Although every cell inside standard drum memory holds both a mantissa and its exponent packed together, two full words are required to hold a number during intermediate calculations and output processing: namely, one cell for the isolated mantissa and one cell for the exponent.

To facilitate data processing from memory, the interpretive system establishes a simulated floating-point accumulator, a multiplication register, an address register, and an instruction counter register. The accumulator and multiplication register each occupy two memory cells, while the address register and instruction counter occupy only one cell each. All data remains in floating-point format throughout processing. Operands and addresses can be modified dynamically without exiting the interpretive system environment.

The initial contents of all registers are zero when the program has just been freshly loaded and not yet executed. The contents of all registers (with the sole exception of the instruction counter) are preserved upon system switches and are modified exclusively by explicit interpretive instructions. Therefore, the user can exit and re-enter the interpretive system repeatedly without corrupting or altering the values held in the registers.

The instruction counter is set upon every entry into the interpretive system and is incremented during the execution of every instruction. These specialized registers are detailed below:

1. The Floating-Point Accumulator The floating-point accumulator holds intermediate calculation results. Although the mantissa is stored in a normalized state in standard memory, it is shifted right by one binary place when held inside the floating-point accumulator. This means the mantissa sits at $q = 0$ with its most significant bit shifted one position to the right; the exponent sits at $q = 30$. Both components retain their independent algebraic sign bits. The

contents of the accumulator are altered exclusively by arithmetic operations, function evaluations, or via the instructions: “Clear”, “Swap”, “Set Sign”, or during a function calculation.

2. The Multiplication Register The multiplication register (M-register) holds the multiplicand during the execution of the “Multiply” and “Cumulative Multiplication” instructions. The mantissa sits at $q = 0$ with its most significant bit shifted one position to the right; the exponent sits at $q = 30$. Both parts are provided with their respective algebraic signs. The contents of this register are modified exclusively by the “Set” and “Swap” instructions.

3. The Address Register The address register consists of a single cell holding a track/sector value scaled at $q = 29$. This register is utilized to dynamically modify the operand address portion of standard instruction words in memory via the “Store Address” command. Additionally, this register is used during “Test for Zero” branch operations. The contents of this register can be modified exclusively by the “Set Address” or “Increment Address” instructions.

4. The Instruction Counter Register The instruction counter register tracks the memory address of the floating-point instruction currently being executed and flags the address of the next instruction to be processed. When executing a “Return” instruction, the operand address portion of the target cell is overwritten with the current contents of the instruction counter plus 2. The register is initially seeded by the main program’s calling sequence and increments by 1 with each instruction executed. This sequential operation is broken only by explicit jump instructions, which overwrite the register with a new target address.

FLOATING-POINT INSTRUCTIONS

This interpretive system operates using 33 pseudo-instructions. Each instruction consists of an operation code field and an operand address field. Certain operand addresses carry a divergent structural meaning compared to native fixed-point machine commands (e.g., null/zero addresses).
The floating-point instructions are detailed below. The placeholder notation `ttss` denotes an operand address written in decimal track and sector format. All referenced registers represent the simulated virtual registers of the interpretive engine. All instructions execute to completion without exiting the interpretive system.

1. Arithmetic Instructions

   `B ttss` **(Bring):** Copies the contents of cell `ttss` into the floating-point accumulator. The source memory remains unchanged.

`A ttss` **(Add):** Adds the contents of cell `ttss` to the current value of the accumulator. The sum replaces the contents of the accumulator. Memory remains unchanged.

`S ttss` **(Subtract):** Subtracts the contents of cell `ttss` from the current value of the accumulator. The difference replaces the contents of the accumulator. Memory remains unchanged.

`D ttss` **(Divide):** Divides the current value of the accumulator by the contents of cell `ttss`. The resulting quotient replaces the contents of the accumulator. Memory remains unchanged.

`P ttss` **(Set M-Register):** Copies the contents of cell `ttss` into the multiplication (M) register. Memory remains unchanged.

`M ttss` **(Multiply):** Multiplies the contents of the M-register by the contents of cell `ttss`. The product replaces the contents of the accumulator. The source memory and the M-register remain unchanged.

`N ttss` **(Cumulative Multiplication):** Multiplies the contents of the M-register by the contents of cell `ttss`, and automatically adds the product directly to the current accumulator value. The final sum resides in the accumulator. Memory and the M-register remain unchanged.

`D 000y` **(Shift Right):** Divides the current contents of the accumulator by 2^y. The operand address must be formatted as `000y`, where $0 \leq y \leq 9$. The accumulator contents remain strictly in floating-point format.

`M 000y` **(Shift Left):** Multiplies the current contents of the accumulator by 2^y. The operand address must be formatted as `000y`, where $0 \leq y \leq 9$. The accumulator contents remain strictly in floating-point format.

`H ttss` **(Hold):** Copies the current contents of the accumulator into memory cell `ttss`. The accumulator contents remain unchanged.

`C ttss` **(Clear):** Copies the current contents of the accumulator into memory cell `ttss`, and subsequently flushes the accumulator register entirely with zeros.

2. Jump / Branch Instructions

`U ttss` **(Unconditional Jump):** Overwrites the instruction counter register with the address `ttss`. Execution loops to fetch the next instruction from cell `ttss`. This command cannot be used to exit the interpretive system.

`T ttss` **(Test for Minus):** Overwrites the instruction counter register with address `ttss` if the current accumulator value is negative; execution then branches to `ttss`. If the accumulator is positive or zero, the branch is ignored and the next sequential instruction is executed. This command cannot be used to exit the interpretive system.

`800T ttss` **(Conditional Jump):** Overwrites the instruction counter register with address `ttss` if the accumulator value is negative OR if Program Selector Switch 7 (`PST`) is turned ON. Otherwise, execution continues sequentially. This command cannot be used to exit the interpretive system.

3. Address Modification Instructions

   `E ttss` **(Set Address):** Extracts the operand address portion of the instruction word stored at cell `ttss` and copies it into the simulated address register. Memory remains unchanged.

   `I ttss` **(Increment Address):** Increments the current numeric value of the simulated address register by the value `ttss`. This instruction can be used to decrement the register by supplying the complement of the value `ttss`.

   `Y ttss` **(Store Address):** Copies the tracking address currently held in the simulated address register into the operand address field of the instruction word at cell `ttss`. The remaining fields of cell `ttss` and the address register itself are unaltered.

   `Z ttss` **(Test for Zero):** Subtracts the literal value `ttss` from the current contents of the simulated address register. If the result is non-zero, the immediately following sequential instruction is executed. If the result is exactly zero, the next instruction is skipped and execution jumps directly to the second following word (the instruction after next).

   `R ttss` **(Return):** Computes the current value of the instruction counter register plus 2, and writes that calculated address into the operand address field of the instruction word at cell `ttss`. The instruction counter itself and the remaining fields of cell `ttss` are untouched.

4. Auxiliary Instructions The operand address field for all auxiliary commands must be exactly zero (`0000`).

   `U 0000` **(Swap):** Swaps the contents of the multiplication (M) register and the floating-point accumulator.

   `B 0000` **(Set Plus Sign):** Forces the accumulator value to be positive if it is currently negative. If the accumulator is already positive or zero, the command is skipped as a no-operation.

   `T 0000` **(Set Minus Sign):** Forces the accumulator value to be negative if it is currently positive. If the accumulator is already negative, the command is skipped as a no-operation.

   `Y 0000` **(Invert Sign):** Replaces the accumulator contents with its arithmetic complement (negating the current sign).

   `Z 0000` **(Stop):** Halts all computation loops immediately, provided that physical Break Point Switch 16 (`PS-16`) is turned OFF. Pressing the

physical "Start" button on the console resumes execution at the next sequential memory cell. If `PS-16` is turned ON, this stop command is bypassed entirely.

`E 0000` **(Exit):** Terminates execution within the interpretive engine. The instruction immediately following this command word is decoded and executed as a standard native machine instruction.

5. Input / Output Instructions The operand address field for all I/O commands must be exactly zero (`0000`).

   `I 0000` **(Input):** Reads a block of data from tape, automatically parses it into floating-point format, and stores it in sequential memory addresses for main-program access. The target destination base address is read directly from the data tape leader header.

   `P 0000` **(Output):** Prints the current contents of the floating-point accumulator to the output unit in standard floating-point notation. The accumulator state is completely preserved during the print operation.

6. Transcendental Function Evaluation Instructions The operand address field for all function evaluations must be exactly zero (`0000`).

   `R 0000` **(Square Root):** Computes the square root ($\sqrt{x}$) of the current accumulator contents. The result replaces the value in the accumulator.

   `S 0000` **(Sine):** Calculates the sine ($\sin x$) of the current accumulator contents. The result replaces the value in the accumulator. The input argument x must be supplied strictly in radians.

   `C 0000` **(Cosine):** Calculates the cosine ($\cos x$) of the current accumulator contents. The result replaces the value in the accumulator. The input argument x must be supplied strictly in radians.

   `A 0000` **(Arctangent):** Calculates the principal value of the arctangent ($\arctan x$) of the current accumulator contents. The resulting angle is returned strictly in radians.

   `N 0000` **(Natural Logarithm):** Calculates the natural logarithm ($\ln x$, base x relative to e) of the current accumulator contents.

   `H 0000` **(Exponential Function):** Computes the natural exponential value (e^x), where x represents the initial numeric value held in the accumulator.

Data Input and Output 1

Data Input

The "Data Input and Output 1" subroutine introduces data into the computer for utilization by a main program. Decimal numbers are read from tape, au-

tomatically converted into binary floating-point format, and stored into the specified memory cells in the required configuration.

This subroutine is invoked directly via an `I 0000` pseudo-instruction processed within the interpretive loop of the main program. Control automatically returns to the memory cell immediately following this input command once all data elements have been successfully processed and stored.

The input data tape must contain the following expressions in the exact order and format described below:

a) Input Keyword (Control Word) The first expression in each distinct group of data elements must be a control word that specifies both the number of decimal places P for every word within that group, and the base starting address `ttss` of the destination memory block.

   The parameter P must be supplied as a two-digit decimal integer preceded by an algebraic sign, constrained strictly within the range:

   $$-03 \leq P \leq +16$$

   The starting destination address must be written as an absolute address in decimal track and sector notation (`ttss`). A positive P value indicates that the decimal point is shifted to the left; a negative P value indicates that the decimal point is shifted to the right.

b) Numeric Data Elements The individual data entries are represented on tape as signed decimal integers containing up to 7 digits of precision. For positive values, both the explicit plus sign and any leading zeros may be omitted entirely. For negative values, however, all leading zeros between the minus sign and the first significant digit must be explicitly punched on the tape. A value of zero requires only a single stop code (`'`) to be written.

   Example: Given an input control keyword punched as `+032400'`, the fractional decimal value **16.192** would be encoded and supplied on tape simply as `16192'`.[22]

c) End-of-Block Keyword and Minus-Zero Termination The end of a specific data group is flagged on tape by an explicit minus-zero control word, formatted as `-0000000'`. This tracking word functions strictly as a structural boundary indicator and is not saved in memory by the subroutine.

   The definitive end of the entire reading operation is signaled by appending a second consecutive stop code immediately following this minus-zero control word (i.e., formatted as `-0000000''`).

   When the interpreter reads the singular minus-zero word (`-0000000'`), the program automatically transitions to read the next sequential input

[22] **Translator's Note:** Because $P = +03$, the interpreter shifts the implied decimal point 3 places to the left upon ingestion, reconstructing $16192 \times 10^{-3} = 16.192$.

control keyword on the tape, subsequently converting all following data values according to the newly defined P scaling factor and storing them starting at the new destination base address.

When the double-stop code for the absolute end of the print/read sequence (`-0000000''`) is encountered instead, the subroutine executes a carriage return and returns execution control back to the main program.

The following example illustrates the physical layout of a typical structured data block:

Input control word			Sto	Decimal numbers with sign		Sto	
±	P	Cell	p	±		p	CR
+	07	2004	’		1090000	’	☐
					120000	’	☒
					340000	’	☐
					560000	’	☒
					780000	’	☐
					900000	’	☒
+			’	–	0000000	’	☐
+	03	2010	’	–	4320001	’	☒
				–	0000021	’	☐
				–	3470	’	☒
				–		’	☐
					10	’	☒
				–	0000000	’	☐
						’	☒

Table 1: LGP-21 Floating Point Tape Input Example

The first six decimal numbers are read in and stored as follows:

Memory Cell	**Contents**
2004	109
2005	0.012
2006	0.034
2007	0.056
2008	0.078
2009	0.09

The second record group consisting of five data words would be stored as follows:

Memory Cell	Contents
2010	−4320.001
2011	−0.021
2012	3.47
2013	0
2014	0.01

The minus-zero control word `-0000000'` terminates each individual group and triggers the system to read the next record block. All mantissas are maintained at a binary scaling factor of $q = 0$ with a precision length of 24 bits. All exponents are stored within the exact same memory word as their respective mantissa, scaled at $q = 30$. The additional tracking stop code appended behind the second minus-zero control word completely terminates the data input sequence.

2. Data Output

The "Data Input and Output 1" subroutine delivers the contents of the floating-point accumulator as a signed, seven-digit decimal mantissa and a signed, two-digit decimal exponent. Following the output execution, a tabulator character (tab space) is issued, and control branches back to the memory cell immediately following the `P 0000` instruction that originally invoked the subroutine.

Output Format The printed output expression uses the following structural form:

`.XXXXXXX`± `EE`±

Where:

— `.XXXXXXX`± represents the decimal mantissa with its trailing sign.

— `EE`± represents the power-of-10 base exponent with its trailing sign.

Output Examples The following table illustrates the raw machine printout format paired with its conventional decimal representation:

Machine Printout	Conventional Representation
`.5060000- 04`	−5060.000
`.0937500- 02-`	−0.000937500
`.1000000- 06-`	−0.0000001000000
`.1000000- 10`	-0.1×10^{-10}

SINE-COSINE 3, Program B1-10.2

"Sine-Cosine 3" calculates the sine or cosine of an argument provided in floating-point radians. Upon returning to the main program, the result is stored in the floating-point accumulator. The return is executed to the cell immediately following the `S0000` or `C0000` instruction, depending on which subroutine was invoked.

ARCTANGENT 2, Program B1-11.2

"Arctangent 2" calculates the arctangent of an argument located in the floating-point accumulator. The result is placed in the floating-point accumulator in radians when control returns to the cell of the main program following the `A0000` instruction.

LOGARITHM 2, Program B3-11.1

"Logarithm 2" calculates the natural logarithm (base e) of a number located in the floating-point accumulator. The result remains in the floating-point accumulator upon return to the cell of the main program following the `N0000` instruction.

EXPONENTIAL FUNCTION 2, Program B3-10.1

"Exponential Function 2" calculates the value e^x, where x is the contents of the floating-point accumulator. The result is stored in the floating-point accumulator upon return to the cell of the main program following the `H0000` instruction.

SQUARE ROOT 2, Program B4-10.1

"Square Root 2" calculates the square root of a positive number located in the floating-point accumulator. The result is placed in the floating-point accumulator upon return to the cell of the main program following the `R0000` instruction.

RUNNING THE INTEPRETER

The "Floating-Point Interpreter System 1" is invoked via a calling sequence from the main program. Prior to branching into the system, the starting address of the floating-point instructions to be interpreted must be supplied to the system. The floating-point instructions must immediately follow the branch instruction. An example is provided below:

Cells α and $\alpha+1$ contain instructions `R` and `U`, which store the target address (α+2) into the interpreter system and execute a branch to L, the starting address of the interpreter system. Cells $\alpha+2$ through $\alpha+n$ contain the actual floating-point instructions. An explicit exit from the interpreter system is required when, as shown here at $\alpha+n+1$, fixed-point machine instructions follow.

Memory Cell	Instruction	Address	Description
α	R	L_0	Store starting address of floating-point instructions.
$\alpha + 1$	U	L_0	Entry into the interpreter system.
$\alpha + 2$	...		Floating-point instructions.
$\alpha + n$	...		
$\alpha + n + 1$	XE	0000	Exit from the interpreter system. Return to fixed-point instructions.

FIXED-POINT TO FLOATING-POINT CONVERSION (AND VICE VERSA), L3-12.0

This program serves two functions. It converts a fixed-point number into the floating-point format defined for "Floating-Point Interpreter System 1", or it converts a floating-point number back into a fixed-point number. This program is not an integral part of the interpreter system core; rather, it is a fixed-point subroutine designed to convert data intended for, or generated by, the interpreter system.

Before executing the calling sequence for this subroutine, the interpreter system must be exited. The number to be converted must reside in the hardware accumulator, and its scale factor q must be supplied by the calling sequence.

When converting back, a floating-point number must not be too large to be held at the specified q. When converting forward, a fixed-point number must not require an exponent greater than 31.

Upon return to the main program, the converted number resides in the machine accumulator. The value will either be in fixed-point form at the q provided by the calling sequence (if converted back), or in floating-point form corresponding to the specified q (if converted forward).

The respective calling sequences are explained below. The first is for converting a number from fixed-point to floating-point, and the second is for converting a number from floating-point back to fixed-point. Only one operation per number can be executed during a single call.

Conversion to floating-point

Memory Location	Instruction	Address	Description
$\alpha - 1$	B	[ ]	Load fixed-point number into accumulator.
α	R	L+25	Store address of q.
$\alpha + 1$	U	L	Entry into the subroutine.
$\alpha + 2$	Z	$089q$	Specify q of the number.
$\alpha + 3$	etc.		Return to main program.

The instruction at $\alpha - 1$ loads the fixed-point number into the accumulator (any instruction achieving this may be used). The instruction at α stores the address $(\alpha + 2)$ of the value q into the subroutine. The instruction at $\alpha + 1$ causes a branch to L, the starting address of this subroutine. The instruction at $\alpha + 2$ contains the q of the fixed-point number formatted as a decimal sector address. The return from the subroutine occurs at $\alpha + 3$.

Conversion to fixed-point

Memory Location	Instruction	Address	Description
$\alpha - 1$	B	[ ]	Load floating-point number into accumulator.
α	R	L+148	Store address of q.
$\alpha + 1$	U	L+122	Entry into the subroutine.
$\alpha + 2$	Z	$08qq$	Specify target q for the number.
$\alpha + 3$	etc.		Return to main program.

The instruction at $\alpha - 1$ loads the floating-point number into the primary hardware accumulator (any instruction achieving this is permissible). The instruction at α stores the address $(\alpha + 2)$ of the value "q" into the subroutine. The instruction at $\alpha + 1$ causes a branch to the appropriate entry cell, $L + 122$, of this subroutine. The instruction at $\alpha + 2$ contains the "q" that the converted fixed-point number should assume, formatted as a decimal sector address. The return from the subroutine occurs at $\alpha + 3$.

PROGRAM STOPS

There are two dedicated program stops within this subroutine. They are listed and explained below. L represents the starting address of this subroutine.

Memory Cell	Meaning and Remedy
$L + 152$	The number is too large for the specified "q" during backward conversion. Pressing "Start" clears the accumulator and returns control to the main program.
$L + 262$	The number requires an exponent greater than 31 during forward conversion. Pressing "Start" clears the accumulator and returns control to the main program.

Note that this subroutine is utilized independently. An input or output unit is not required. Storage requirements are detailed below.

STORAGE REQUIREMENTS

Only "Square Root 2" is natively bundled inside the actual interpreter system core. The other subroutines are provided individually. They can be loaded partially or entirely depending on application needs. The subroutines are listed below alongside their relative starting addresses and required memory footprints (where L represents the starting address of the interpreter system):

Subroutine	**Initial Filler Keyword**	**Modifier Keyword**	**Memory Required**
Interpreter System	L	L	10 Tracks
Data Input & Output 1	L+1000	L+1000	7 Tracks
Sine-Cosine 3	L+1700	L+1700	2 Tracks, 32 Sectors
Arctangent 2	L+1932	L+1932	1 Track, 32 Sectors
Logarithm 2	L+2100	L+2100	1 Track
Exponential Function 2	L+2200	L+2200	2 Tracks
Fixed/Floating Conversion & Vice Versa 1	*Anywhere*	*Anywhere*	3 Tracks

Track 63 is utilized by various parts of the system as an intermediate scratchpad memory, with the sole exception of "Fixed-Point to Floating-Point Conversion and Vice Versa 1", which requires no scratchpad storage. Therefore, no part of the system should ever be stored in Track 63.

OPERATION

1. Program Input

The "Floating-Point Interpreter System 1" is provided on two separate paper tapes. The first tape contains the program in an arbitrarily relocatable hexadecimal format, compatible with Program Input 1 (`J1-10.0`). The second tape contains the program in an arbitrarily relocatable decimal format structured in coding sheet layout. Each tape contains the interpreter system core followed by all subroutines in the exact order specified under "Storage Requirements". It is assumed that Program Input 1 starts at address `0000`.

If program selector switch PS-32 is turned ON, the relocatable hexadecimal program tape can be read continuously without manual intervention until the "Fixed-Point to Floating-Point Conversion and Vice Versa" routine is reached. If PS-32 is turned OFF, the computer will halt after each successfully read subroutine. (No stop is hardcoded between the Interpreter System core and "Data Input and Output 1".)

In either case, entering a new modifier keyword and a select keyword for the input device is strictly required before loading "Fixed-Point to Floating-Point Conversion and Vice Versa". This specific program must *never* be loaded using the same modifier as any part of the interpreter system core.

The relocatable decimal tape has a hardcoded stop programmed after the interpreter system core and after every individual subroutine. In all cases when using this tape, a new modifier and a new device selection keyword must be entered manually for each subroutine to be read.

For both tape types, whenever a stop occurs due to PS-32 OFF, the operator must press the "Start" button to return execution to Program Input 1 for subsequent data entry. If a stop occurs while PS-32 is ON, the computer will halt directly on an input command within Program Input 1.

2. Required Input/Output Units

The Flexowriter must be selected for both input and output operations.

3. Program Selector Switches

Switch	Setting	Function
PS-16	ON	Suppresses the programmatic halt at a `Z 0000` instruction.
	OFF	Permits a programmatic halt at a `Z 0000` instruction.
PS-32	ON	Forces an unconditional branch upon encountering an `-T` instruction[23]
	OFF	Forces a conditional branch upon encountering an `-T` instruction.

4. Program Stops

L represents the absolute starting address of the interpreter system.

REMARKS

The initial location of the interpreter system should ideally align with Sector `00` of a track so that the reference addresses targeting Track 63 are optimally mapped.

Remember that when the interpreter is running all instructions are pseudo-instructions! The interpreter is substituting itself for the standard fetch-execute cycle that the CPU would do. Another way of saying it is that the interpreter is an emulator for a CPU that has a floating-point instruction set, and we have an agreed-upon protocol to enter and exit emulation.

Location	Meaning	Remedy
$L + 557$	The exponent is too large for an `H` or `C` instruction, or the argument of a square root is negative. The address of the offending instruction resides in the machine accumulator.	Press "Start" to bypass. The `H` instruction, `C` instruction, or square root calculation will not be executed.
$L + 759$	Division by an illegal value (Division by Zero).	Do not proceed. The data value must be corrected.
$L + 1152$	The input value possesses an exponent that is too large.	Press "Start" to advance to the next pseudo-instruction. A value of zero is stored in place of the rejected number.
$L + 2005$	The argument for a natural logarithm is ≤ 0.	Press "Start" to proceed with a computed result of zero.
$L + 2056$	The exponent is too large for a natural logarithm operation.	Do not proceed. The data value must be corrected.

Pseudo-instructions utilizing addresses from `0000` to `0009` have special meanings. When pseudo-instructions use these addresses, they do not refer to the actual storage at these addresses, but instead invoke functions in the interpreter. The interpreter utilizes 16 such special instructions, including `D 000y` and `M 000y`. Although instructions `D 000y` and `M 000y` may carry a literal value of y between 1 and 9, standard instructions like `D ttss` and `M ttss` (while in interpreter mode) cannot reference any cell between `0000` and `0009`[24].

PROGRAMMING EXAMPLE

This example demonstrates how to evaluate a fourth-degree polynomial using the Floating-Point Interpreter System's extended instruction set. The polynomial is structured as:

$$y = Ax^4 + Bx^3 + Cx^2 + Dx + E$$

By applying Horner's Method, the equation is factored to minimize arithmetic operations, reducing it to a sequence of successive multiplications and additions:

$$y = (((Ax + B)x + C)x + D)x + E$$

The Interpreter System core is loaded at address `0300`. The main program begins at cell `4200`. The five floating-point coefficients (A, B, C, D, E) and the variable x are stored as 3-word blocks starting at cell `4306`.

[24]The addresses from `0000` to `0009`, inclusive, trigger transcendental functions, etc. *when the floating-point interpreter is running.* In normal execution mode, they're used just the same as any other location in the machine.

Loc	Op	Addr	Remarks / Description
4200	R	0300	; Set up return link pointer for interpreter loop
4201	U	0300	; Enter Interpreter Mode (Jumps to system core)
4202	B	4306	; Load coefficient A into pseudo-accumulator
4203	M	4324	; Multiply by x [Ax]
4204	A	4309	; Add coefficient B [Ax + B]
4205	M	4324	; Multiply by x [(Ax + B)x]
4206	A	4312	; Add coefficient C [(Ax + B)x + C]
4207	M	4324	; Multiply by x
4208	A	4315	; Add coefficient D
4209	M	4324	; Multiply by x
4210	A	4318	; Add coefficient E [Final polynomial value y]
4211	H	4321	; Store calculated result y into memory block
4212	XE	0000	; Exit Interpreter Mode (Return to native hardware)
4213	H	0000	; Native machine execution halt

2. Data Memory Allocation Map

Because the pseudo-accumulator manages multi-word operations, each floating-point literal spans 3 consecutive words on the magnetic drum.

Start Address	Symbol	Allocation Footprint
4306	A	Occupies cells 4306, 4307, 4308
4309	B	Occupies cells 4309, 4310, 4311
4312	C	Occupies cells 4312, 4313, 4314
4315	D	Occupies cells 4315, 4316, 4317
4318	E	Occupies cells 4318, 4319, 4320
4321	Y	Result storage (cells 4321, 4322, 4323)
4324	X	Input variable x (cells 4324, 4325, 4326)

QUICK REFERENCE INSTRUCTION SUMMARY

Operational Notation Definitions

( )	Contents of memory cell or register
→	Displaces / moves to / becomes
Acc	Floating-Point Software Accumulator
A Reg.	Address Register
M Reg.	Multiplier Register
C Reg.	Counter Register

Extended Interpreter Instruction Set

Op	Operand (ttss)	Result / Action
B	ttss	(ttss) → Acc
A	ttss	(Acc) + (ttss) → Acc
S	ttss	(Acc) - (ttss) → Acc
D	ttss	(Acc) / (ttss) → Acc
P	ttss	(ttss) → M Reg.
M	ttss	(M Reg.) × (ttss) → Acc
N	ttss	(M Reg.) × (ttss) + (Acc) → Acc
H	ttss	(Acc) → ttss
C	ttss	(Acc) → ttss; then 0 → Acc
U	ttss	Unconditional Jump: Next instruction is at ttss
T	ttss	Conditional Jump: Jump to ttss if (Acc) < 0
-T	ttss	Conditional Jump: Jump to ttss if (Acc) < 0 OR if PST is ON
E	--	Exit Interpreter Mode
I	ttss	ttss + (A Reg.) → A Reg.
Y	ttss	(Address Part of ttss) → A Reg.
Z	ttss	(A Reg.) → Address Part of ttss
R	ttss	Compare: If (A Reg.) - ttss = 0, skip next instruction; else loop

Special Behavior for Null Addresses

When the address field `ttss` of an interpreter command equals `0000` (or a designated function selector ID within Track 00), the default memory interaction is overridden to perform specialized administrative or algorithmic operations rather than referencing physical memory cell zero.

Interpreter Behavior for Null Addresses (Address = 0000)[25]

[25] Excluding extended subroutine selection commands under the format `D000y` or `M000y`. See Section 0479.

Op	Result / Action Executed (When Address Field is 0000)	
D	$(\text{Acc})^2 \rightarrow$ Acc	[Square the Accumulator]
M	(Acc) $\times$ 2 $\rightarrow$ Acc	[Double the Accumulator]
U	Swap values: (Acc) $\longleftrightarrow$ (M Reg.)	
B	Make (Acc) positive	[Absolute Value: $\lvert Acc \rvert$]
T	Make (Acc) negative	[Force Negative Sign]
Y	Invert Sign: Change sign bit of (Acc)	
Z	Conditional Program Stop (Halts execution if console switch PS-16 is OFF)	
E	Exit Interpreter Mode (Return control to native machine hardware)	
I	Read floating-point numbers from paper tape input	
P	Print/Punch floating-point numbers to output device	
R	$\sqrt{(\text{Acc})} \rightarrow$ Acc	[Square Root]
S	Sine(Acc) $\rightarrow$ Acc	
C	Cosine(Acc) $\rightarrow$ Acc	
A	Arctangent(Acc) $\rightarrow$ Acc	
N	$\ln$(Acc) $\rightarrow$ Acc	[Natural Logarithm]
H	$e^{(\text{Acc})} \rightarrow$ Acc	[Exponential Function]

6.2 Floating Point Interpreter 2

H1-11.1

1. INTRODUCTION

The "Floating-Point Interpreter System 2" permits the programming of LGP-21 problems using floating-point arithmetic. The system accepts 9-digit whole decimal numbers as input data, scaled by a two-digit decimal exponent. It automatically handles all required scaling modifications during calculation and outputs results as a 7-digit signed decimal mantissa, followed by a 2-digit decimal exponent.

Because the system autonomously processes values across an exceptionally wide dynamic range, the programmer does not require precise prior knowledge of a problem's numerical boundaries. This structural capacity guarantees a high degree of calculation and output accuracy. Furthermore, this program expands the standard 16-instruction hardware set of the computer to **33 instructions**, making operations as simple and user-friendly as possible.

The "Floating-Point Interpreter System 2" is available as a single paper tape in an arbitrarily relocatable hexadecimal format, compatible with Program Input 1 (`J1-10.0`). It bundles various subroutines—such as Data Input, Data Output, and a suite of transcendental math functions—which are detailed later in this manual. Only the specific subroutines required for a given problem need to be read into memory alongside the main "Floating-Point Interpreter System 2" core; however, the memory cross-references for the entire system must be strictly preserved during loading.

The programmer will quickly find that utilizing this system yields a massive savings of time and effort when dealing with problems where data values

fluctuate wildly in magnitude—scenarios that would otherwise demand an immense, complex programming overhead under traditional fixed-point arithmetic constraints.

Principles

The "Floating-Point Interpreter System 2" intercepts and reinterprets the elementary hardware instruction structure of the LGP-21, transforming it so that it can be programmed exactly like a natively "hardwired" floating-point computer. This illusion is achieved through:

1. **Virtual Registers:** The provision of an internal, software-driven Multiplier Register, Address Register, and a dedicated Floating-Point Accumulator.
2. **Advanced Operations:** Integrated support for cumulative multiplication (multiply-accumulate), sign inversion, and direct function-generating commands.
3. **Massive Numerical Range:** Input data consists of signed 9-digit decimal numbers followed by a base-10 exponent between +99 and −99. Output data delivers signed 7-digit decimal numbers followed by a base-10 exponent between +99 and −99.
4. **Expanded I/O framework:** A vastly modernized structure for data entry and terminal printing commands.

The entire system footprint consumes **28 Tracks** of memory. This is divided into the core interpreter loop, Data Input (7 tracks), Data Output (4 tracks), and Floating-Point Functions (6 tracks). This leaves **36 Tracks (2,304 words)** completely free on the magnetic drum for the programmer's main execution routine and data storage arrays.

2. THE REGISTERS

The floating-point format requires two consecutive memory cells to represent a single data word. Consequently, the interpreter system features two specialized registers designed for data transport, processing, and output while strictly preserving the floating-point structure. These registers—the Floating-Point Accumulator and the Multiplier Register—each occupy two memory cells.

Address modification can be executed directly within the interpreter system by utilizing a simulated, single-cell Address Register. Additionally, a single-cell Counter Register governs the operational execution flow of the interpreter system loop.

Upon initial system loading, before any operations have commenced, the contents of all virtual registers are initialized to zero. A detailed breakdown of these specialized registers follows below. (The individual instructions referenced here are explained comprehensively in Section 4, "Pseudo-Instructions").

1. The Floating-Point Accumulator

The Floating-Point Accumulator holds intermediate mathematical results and output values. Because output data must ultimately be rendered as a signed 7-digit decimal mantissa with a 2-digit decimal exponent, the hardware treats them internally as follows:

- **The Mantissa:** Maintained within the Floating-Point Accumulator with a precision of 30 bits at binary scaling factor $q = 0$. The most significant bit possesses a weight of 2^{-1}, meaning the mantissa is *always* strictly normalized.
- **The Exponent:** Positioned at binary scaling factor $q = 29$.

Both the mantissa and the exponent maintain their own independent, algebraic sign bits. The contents of the Accumulator are altered exclusively by arithmetic operations, transcendental function calls, or explicit register management commands (Clear, Swap, or Set Sign).

2. The Multiplier Register

The Multiplier Register stores the multiplicand during standard multiplication and cumulative multiplication (multiply-accumulate) routines. * The internal mantissa is mapped at $q = 0$. * The internal exponent is mapped at $q = 29$.

Like the accumulator, both components carry their respective algebraic sign bits. The contents of this register can only be modified by executing explicit "Set" or "Swap" commands.

3. The Address Register

The Address Register holds a single virtual memory address formatted in Track/Sector notation at binary scaling factor $q = 29$. This register is specifically utilized to modify the operand address field of memory words via the "Store Address" command, and to govern main program execution loops during zero-testing routines. The contents of this register are altered exclusively by an explicit "Set Address" or "Increment Address" instruction.

4. The Counter Register

The Counter Register functions as the virtual Program Counter (PC). It tracks the absolute address of the currently executing pseudo-instruction and points ahead to the address of the subsequent instruction.

When a subroutine return sequence is initiated, the current contents of this register plus 2 (PC+2) are injected into the operand address field of the targeted return command. The register is initially primed by the main program's calling sequence and automatically increments by 1 following the execution of each pseudo-instruction. This linear, step-by-step increment can only be overridden by a programmatic branch or jump instruction.

INTERNAL NUMBER FORMAT

Every floating-point number requires two consecutive memory cells for its complete representation.

- **The first cell** holds the mantissa of the number in binary representation scaled at $q = 0$. Each mantissa consists of an algebraic sign bit and 30 significant binary digits; the most significant bit (MSB) carries a value of 2^{-1}. The mantissa is therefore always maintained in a strictly normalized state.
- **The second cell** holds the base-2 exponent, which must be multiplied by the mantissa to obtain the actual value of the represented number. This exponent, maintained at $q = 29$, is a whole binary integer carrying its own algebraic sign bit.

Taken together, these two components represent a floating-point number via the following mathematical relationship:

$$Z = M \cdot 2^E$$

Where Z = The Number, M = The Mantissa, and E = The Exponent.

Example 1: Representation of $+3.75$

In floating-point memory, the decimal value 3.75 is internally normalized as $0.9375 \cdot 2^2$. Its exact binary equivalent on the magnetic drum is structured as follows:

First Cell (Mantissa at $q = 0$):

Bit 0	Bits 1--30	Bit 31
0	11110000000000000000000000000000	0
↑	↑	↑
Sign (+)	Normalized Mantissa (0.9375)	Unused

Second Cell (Exponent at $q = 29$):

Bit 0	Bits 1--29	Bits 30--31
0	00000000000000000000000000000	10
↑	↑	↑
Sign (+)	Padding Zone	Binary Exponent (2)

Example 2: Representation of a Negative Number (-3.75)

A negative number is represented by inverting the leading sign bit of the mantissa cell:

First Cell (Mantissa at $q = 0$):

Bit 0	Bits 1--30	Bit 31
1	00010000000000000000000000000000	0
↑	↑	↑
Sign (−)	Two's Complement Mantissa	Unused

Sequential Allocation in Drum Memory

The continuous storage of alternating mantissas and exponents across sequential drum tracks steps through memory in fixed increments of two:

Memory Location	Cell Contents
4304	Mantissa of Variable 1
4305	Binary Exponent of Variable 1
4306	Mantissa of Variable 2
4307	Binary Exponent of Variable 2

Operational Boundaries (Note):

1. **Input Restriction:** During decimal data entry, the input base-10 exponent is strictly bounded to the range $-99 \leq E_{10} \leq +99$.
2. **Output Restriction:** During printing operations via the paper tape typewriter, the printed base-10 exponent is likewise bounded to $-99 \leq E_{10} \leq +99$.
3. **Binary Underflow/Overflow Guard:** When executing internal calculations or storing the contents of the virtual Floating-Point Accumulator into memory, the computer will *not* halt (i.e., no hardware overflow/underflow fault is triggered), provided that the internal binary exponent satisfies the following safety threshold:

$$-333 \leq E_2 \leq +333$$

4. PSEUDO-INSTRUCTIONS

The following section details the complete catalog of the 33 virtual instructions compiled for the "Floating-Point Interpreter System 2" and explains their operational mechanics. Any reference to the "Accumulator" designates the designated two-cell memory block allocated to the software Floating-Point Accumulator.

Arithmetic Instructions

Note: `XXXX` represents the absolute starting memory address of the target floating-point number's mantissa.

Opcode	Name	Operational Action / Mechanics
BXXXX	Bring	(XXXX) → Accumulator
AXXXX	Add	(Accumulator) + (XXXX) → Accumulator
SXXXX	Subtract	(Accumulator) − (XXXX) → Accumulator
DXXXX	Divide	(Accumulator) / (XXXX) → Accumulator
PXXXX	Place	(XXXX) → M Register
MXXXX	Multiply	(M Register) × (XXXX) → Accumulator
NXXXX	Multiply Cumulatively	[(M Register) × (XXXX)] + (Accumulator) → Accumulator
XD000y	Divide by 2^y	(Accumulator) / 2^y → Accumulator $(0 \leq y \leq 9)$
XM000y	Multiply by 2^y	(Accumulator) × 2^y → Accumulator $(0 \leq y \leq 9)$
HXXXX	Hold	(Accumulator) → XXXX
CXXXX	Clear	(Accumulator) → XXXX; then $0 \rightarrow$ Accumulator

For the power-of-two scaling commands (XD000y and XM000y), the accumulator strictly maintains its internal normalized floating-point structure throughout the bit-shifting sequence.

Logical or Branch Instructions

Note: XXXX represents the target instruction address on the magnetic drum.

Opcode	Name	Operational Action / Mechanics
UXXXX	Unconditional Jump	Forces execution loop to target cell XXXX.[26]
TXXXX	Test Minus	Jumps to XXXX if (Accumulator) < 0; else executes next sequential lin
800TXXXX	Conditional Jump	Jumps to XXXX if (Accumulator) < 0 OR if console switch PST is ON.
XXXXX	Set Address	(Address Part of XXXX) → Address Register
IXXXX	Increment Address	(Address Register) + XXXX → Address Register[27]
YXXXX	Store Address	(Address Part of Address Register) → Address Part of cell XXXX
ZXXXX	Zero Test	(Address Register) − XXXX. If result $= 0$, skip next instruction; else
RXXXX	Return	Links subroutines: (PC + 2) → Address Part of target cell XXXX.

Auxiliary Instructions

Note: The address field (XXXX) for all auxiliary operations must be explicitly configured as 0000.

Opcode	Name	Operational Action / Mechanics
U0000	Swap Registers	(M Register) ⟷ (Accumulator)
B0000	Set Plus Sign	Forces (Accumulator) positive [Absolute Value: $\|Accum$
T0000	Set Minus Sign	Forces (Accumulator) negative [Inverse Absolute Value: $-\|Accum$
Y0000	Change Sign	(Accumulator) × − 1 → Accumulator [Toggle si
Z0000	Operational Stop	Halts execution if console switch PS-16 is ON. Resumes on manual START.
E0000	Exit Interpreter	Safe exit from Virtual Machine; returns to native hardware control at nex

Input/Output Instructions

Note: The address field for all input/output commands must be set to 0000.

Opcode	Name	Operational Action / Mechanics
I0000	Data Input	Diverts to tape-reader module. Parses, normalizes, and stores decimal variable
P0000	Data Output	Prints/Punches the current Accumulator value. Accumulator data remains unalter

During tape data input (I0000), sequential tracking remains under tape control until a designated physical stop-code is pulled from the reader, passing execution control back to the next sequential interpreter instruction.

Function Instructions

Note: The address field for all transcendental operations must be set to 0000. The input argument must already sit in the Accumulator.

Opcode	Name	Operational Action / Mechanics
R0000	Square Root	$\sqrt{\text{(Accumulator)}} \rightarrow$ Accumulator. Negative inputs trigger an immediate erro
S0000	Sine	$\sin$ (Accumulator) $\rightarrow$ Accumulator. Input argument must be formatted in radi
Q0000	Cosine	$\cos$ (Accumulator) $\rightarrow$ Accumulator. Input argument must be formatted in rad
A0000	Arctangent	$\arctan$ (Accumulator) $\rightarrow$ Accumulator. Resulting output is scaled in radians
N0000	Natural Logarithm	$\ln$ (Accumulator) $\rightarrow$ Accumulator.
H0000	Exponential	$e^X \rightarrow$ Accumulator, where X represents the initial value held in the Accu

System Composition Architecture Note: The core arithmetic instructions, branch/jump vectors, address register manipulation routines, auxiliary commands, and the primary Square Root (R0000) engine form the permanent hardcoded baseline footprint of the interpreter core loop.

Conversely, the individual Data Input, Data Output, and specialized transcendental math packages were developed as independent software library modules. To maximize available user space on the drum, these external extensions only need to be loaded into memory if the user's application program explicitly calls them.

5. DATA INPUT

The pseudo-input instruction (I0000) requires a data paper tape formatted precisely as follows:

1. A starting address block.
2. The raw data formatted as signed decimal integers coupled with their respective exponents.

3. A designated group-termination code at the end of each distinct cluster of numbers.

4. A final master execution stop-code at the terminal end of the data tape.

The Starting Address

Every independent data group must begin with its target starting memory location, designated in decimal Track/Sector notation. This must be an absolute address and must be immediately followed by a physical stop-code (`'`).

Example: The entry `4200'` instructs the input subroutine to begin storing the incoming converted floating-point variables sequentially starting at memory cell `4200`.

The Data Structure

Following the starting address line, one or more signed decimal integers and their exponents are fed into the reader. The subroutine converts this decimal matrix into internal binary floating-point representations and stores them in consecutive, two-cell blocks on the drum.

Each individual floating-point value must be explicitly split into *two distinct words* on the paper tape, each line capped with a hardware stop-code (`'`).

- **Negative values:** The minus sign (`-`) must immediately precede the very first digit of the first word.
- **Positive values:** A space may be inserted before the first digit, but a physical plus sign (`+`) is strictly prohibited.

The structural payload layout across the tape lines is split as follows:

- **Word 1:** Contains the algebraic sign bit and the first seven digits of the decimal integer.
- **Word 2:** Contains the final two digits of the decimal integer, followed immediately by the sign and the two digits of the base-10 exponent.

Formally, the physical tape notation maps to this exact structural syntax:

$$\begin{aligned} &\textbf{Line 1 (Word 1):} \quad \pm d_1 d_2 d_3 d_4 d_5 d_6 d'_7 \\ &\textbf{Line 2 (Word 2):} \quad d_8 d_9 \pm e_1 e'_2 \end{aligned}$$

Where $\pm d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8 d_9$ constitutes the total 9-digit signed decimal integer, and $\pm e_1 e_2$ represents the 2-digit decimal scaling exponent.

The mathematical value of the processed data variable is determined by the underlying relationship:

$$\text{Represented Value} = \text{Integer} \times 10^{E_{10}}$$

Where the exponent boundaries are hardcoded at: $-99 \leq E_{10} \leq +99$.

Input Encoding Examples

- **Example 1:** Consider the fraction 0.0900000521374209. This corresponds mathematically to the scientific notation expression: $521374209 \times 10^{-16}$. To feed this value into the interpreter, it must be punched onto the paper tape as two distinct lines:

```
5213742'
09-16'
```

- **Example 2:** Consider the large whole number 5522829640000000. This corresponds to the mathematical notation: $552282964 \times 10^{+07}$. It must be presented to the reader as:

```
5522829'
64+07'
```

Termination Stopcodes

The system defines two distinct classes of termination stop-codes. Their behavior varies fundamentally based on their context and placement on the data coding sheet:

1. **Group Termination Code:** Signifies the end of the current numerical data block. It commands the system to immediately begin reading the next subsequent address and data group on the tape.
2. **Master Input Stop-Code:** Signifies the total completion of data entry. It triggers an automatic terminal carriage return, shuts down the tape reader pipeline, and gracefully jumps execution control back to the next sequential line of the user's main program.

Input example

Table 2 demonstrates how data should be punched to be read by the `I0000` instruction. Data is entered in groups, starting with the address where the data should be stored, followed by the (signed) mantissa, a space, and the (signed) exponent, followed by a stopcode to terminate the number. The input routine will continue reading numbers until a lone stopcode is entered, ending the group. If two lone stopcodes are entered, input ends and execution resumes immediately following the `I0000` instruction.

	INPUT CONTROL WORD (Starting Address)	STOP		SIGNED DECIMAL NUMBERS Part 1 $d_1 d_2 \ldots d_7$	'	Part 2 $d_8 d_9\ e_1 e_2$		STOP	
1	5500	'†	-	3300000	'	00 -	04	'	[Group 1]
2				3141592‡	'	65 -	08	'	
3			-	0000592	'	34 -	11	'	
4					'¶				
5	2134	'		0	'			'	[Group 2]
6				30	'	50 -		'	
7				750	'	00 +		'	
8			-	0100000	'	00 +		'	
9					'§			'‖	

Table 2: Sample Data Input Format

† The stopcode (') immediately following the initial address ends the address and starts number entry for the input group.

‡ A single stopcode with no numeric input terminates the input group.

¶ Positive quantities are encoded with *no* prefix; negative values should have an explicit minus sign (-). Do not punch a plus sign (+) for positive values.

§ This single stopcode ends the second data group.

‖ This stopcode following the group terminator stopcode tells the interpreter to exit input mode and resume execution.

Reading the data in Table 2 results in the following data in memory. Group 1 is read and the floating-point numbers are built[28] starting at location 5500: 5500 -33000.0
5501 *(Exponent for* −33000.0*)*
5502 3.14159265
5503 *(Exponent for* 3.14159265*)*
5504 -0.00000059234
5505 *(Exponent for* −0.00000059234*)*

All exponents are strictly maintained at a q of @29. The physical stopcode trailing the final number of the group (located in the column immediately following Group 1) triggers the system to automatically advance and read the next starting target address of the second data group (2134[29]). The second group is

[28] Translator's note: I've preserved the original manual's representation here; suffice it to say that the values indicated are handwaving the actual values in memory.

[29] Note that these addresses don't need to be ordered. The input routine will happily build two-word floating-point numbers wherever you indicate.

processed and stored as follows: `2134` → `Zero (0.0)`
`2135` → `Zero (0.0)`
`2136` → `30.50`
`2137` → *(Exponent for* 30.50*)*
`2138` → `750000.0`
`2139` → *(Exponent for* 750000.0*)*
`2140` → `-10000000000000.0`
`2141` → *(Exponent for* −10000000000000.0*)*

The processing loop for the second data group is terminated by a single stopcode just as Group 1 was. Note the *second* stopcode, which serves to signal the interpreter that input is complete and execution may continue. **Special case:** Please note that any number meant to be input as a literal value of zero *must* be explicitly encoded and punched onto the tape as `0'` rather than as just a stopcode (`'`)[30]. Explicitly entering the zero values is necessary to allow the input routine to differentiate a true numerical zero from a structural group-termination/input-termination command.

6. Data Output

The output format works to make the numbers a consistent size on the page, and to have a consistent format. Each number has:

- a leading character for the sign. This will be a space for positive numbers, and − for negative ones,
- the mantissa of the floating-point number, in leading-decimal format,
- a separator space,
- the sign of the exponent (this is either + or −, and *always* appears),
- the two-digit exponent itself, and
- a tab character.

The `space`/− convention means that positive and negative numbers will align properly to the left down a column.

The tab character enables the user to easily create well-formatted output on the Flexowriter by setting the hardware tab stops.

Control is immediately returned to the main program loop after printing is complete. A single floating-point number is processed and output per subroutine invocation. The actual printed format of a number looks like this:

Positive	`.xxxxxxx` ±`ee[Tab]`
Negative	`-.xxxxxxx` ±`ee[Tab]`

[30]This is different from other input routines, particularly the `??` loader.

This physical teleprinter serialization format represents the standard mathematical expression:

$$\pm.\texttt{xxxxxxx} \cdot 10^{\pm \texttt{ee}}$$

Output Examples

The following examples illustrate how specific numerical values are printed:

Printed Output	Mathematical Value	Notes
-.5060000+04	-5060.0	Negative mantissa, positive exponent
.9375000-02	0.009375	Leading space for positive mantissa
-.1000000-06	-0.0000001	Extreme fractional negative
.1000000+30	1.0×10^{30}	Large positive value

Note how the leading column remains perfectly uniform; the decimal points align vertically on the page regardless of whether the value carries a negative sign or an implicit positive blank space.

Output Limits and Constraints

The formatting routine enforces the following mathematical boundaries during execution:

- **Decimal Exponent Range:** $-99 \leq ee \leq +99$.
- **Decimal Mantissa Bounds:** $0.1000000 \leq |\text{mantissa}| \leq 0.9999999$, or exactly zero.
- **Internal Binary Exponent Range:** $-333 \leq \text{exponent} \leq +333$.

An underflow condition occurs when where the internal binary exponent drops below -333; this causes the routine to automatically print a clean zero. Conversely, an overflow condition where the binary exponent exceeds $+333$ triggers a terminal range error stop, halting the interpreter loop.

7. FUNCTION SUBROUTINES

SINE-COSINE

This subroutine calculates the sine or cosine of a given value expressed in radians, using a 9th degree polynomial to approximate the value. To use, load the angle into the floating-point accumulator then execute the appropriate pseudo-instruction (`S0000` for sine or `C0000` for cosine). The result is placed in the floating-point accumulator. The following table shows the method for calculating the function values depending on the exponent of the argument.

Exponent (binary)	Sine X	Cosine X	Notes
+26 to +127	Range error	Range error	
-8 to +25	Polynomial evaluation	Polynomial evaluation	
-20 to -9	Sine X = X	Polynomial evaluation	$\sin X \approx X$ for small angles
-127 to -21	Sine X = 0	Cosine X = 1.0	Exceeds precision

ARCTANGENT

This subroutine calculates the angle that corresponds to a given tangent argument. Load the tangent value into the floating-point accumulator and execute the **A0000** pseudo-instruction. The result is returned in radians in the floating-point accumulator.

LOGARITHM AND EXPONENTIAL FUNCTION

Load the independent variable `X` into the floating-point accumulator and execute the appropriate pseudo-instruction (`N0000` for log_e and `H0000` for e^x); the resulting value is returned in the floating-point accumulator.

SQUARE ROOT

This subroutine, which is included in the interpreter system, calculates the square root of the number in the floating-point accumulator and returns it there.

---

8. OPERATION

Input is floating-point numbers loaded from tape or in memory, or from numbers in the pseudo-registers from previous operations.

Running the interpreter

Entry to the interpreter is via a subroutine call using an `R`/`U` sequence. Similar to the Floating-point Interpreter I, the `R` instruction sets the address portion of a `B` instruction at L_0, which in this case acts as an executable pointer back to the pseudo-instructions inline after the `U`. The interpreter extracts the address of the code to be interpreted from this `B` instruction. It does *not* set a return address, but instead uses the `E0000` instruction to detect the end of the interpreted pseudo-instructions, and branches back to the (fixed-point) instruction following the end of the floating-point code.

```
α          R   L0     ; Store starting address of floating-point instructions.
α + 1      U   L0     ; Entry into the interpreter system.
α + 2      ...        ; Floating-point instructions.
α + 3      ...        ;
α + n      E   0000   ; Exit from the interpreter system.
α + n + 1  ...        ; Resume fixed-point instructions.
```

Memory Requirements

If the starting address of H1-11.1 is defined as L_0, then the routines on this tape must be loaded as follows (using the corresponding input and modifier key words).

Program	Input key word	Modifier key word	Memory required
Interpreter system	L_0		11 tracks
Data input	$L_0 + 1100$	L_0	6 tracks
Data output	$L_0 + 1700$	L_0	4 tracks
Sine - Cosine	$L_0 + 2100$	L_0	2 tracks
Arctangent	$L_0 + 2300$	L_0	83 sectors
Logarithm	$L_0 + 2419$	L_0	45 sectors
Exponential function	$L_0 + 2500$	L_0	2 tracks

Scratch Storage — The program uses cells 6300 to 6363 as working storage.

Program Stops

Memory Location	Meaning	Remedy / Action
$L_0 + 0947$	Negative argument for square root	Press START. The psuedo-instruction exits with the original argument in the accumulator.
$L_0 + 0312$	Exponent 333 for Hold or Clear	Press START. Hold or Clear pseudo-instructions are bypassed.
$L_0 + 0710$	Division by zero or non-floated number	Execution cannot continue.
$L_0 + 1808$	Exponent > 99 during data input	Press START. The Flexowriter tabs to the next stop without printing, and the code exits to the core interpreter.
$L_0 + 2100$	Argument for $sin/cos \geq 2^2 6$	Press START. The S0000 or C0000 operation is skipped.
$L_0 + 2426$	Argument for $\log_e X \leq 0$	Press START. The pseudo-instruction returns zero and control returns to the main program.
$L_0 + 2629$	Argument for $e^X \geq 128$	Press START. The pseudo-instruction is skipped.

Program Control Switches

The PS-16 switch on the console can be used as a "breakpoint" facility to halt execution at selected points.

Console Switch	Setting	Action
PS-16	**ON**	Execution stops at Z0000 instructions.
	OFF	Execution continues past Z0000 instructions.

Precision and Accuracy

- **Sine-Cosine:** Maximum error is 5×10^{-8}.
- **Data Output:** The maximum error is limited to:
  - 1 unit in the last decimal place of the mantissa if $\text{EXP}_{10} \leq 30$.
  - 5 units in the last decimal place of the mantissa if $\text{EXP}_{10} \leq 99$.

Translator's note: To print a floating-point number, the data output routine has to convert a binary exponent (2^x) into a decimal exponent (10^y). Mathematically, this requires multiplying or dividing the mantissa by powers of 10 (or powers of $\log_{10}(2)$) until the binary scaling is cleared out.

1. For ≤ 30: For these exponents, the conversion can be done using exact or very tightly bounded multiplication factors that fit cleanly within a single or double-word of 31-bit precision. The rounding error is kept to a minimum of 1 unit in the last place.
2. For > 30: For very large numbers (up to 10^{99}), the output routine has to repeatedly multiply by large powers of 10. Because of the LGP-21's 31-bit word size, the intermediate multiplication constants themselves have to be rounded to fit. As the algorithm repeatedly loops and multiplies to scale down large exponents, the rounding errors in these constants also multiply.

By the time the conversion routine finishes working through a number like 10^{95}, the accumulated rounding error corrupts the bottom bits of the mantissa, producing an output that could be off by up to 5 units in the 7th decimal digit.

Notes

1. A program can exit H1-11.1 interpreter mode and return to it without the contents of the pseudo-registers being lost.
2. The starting address of the system should be at sector 00 of a track. Otherwise, many of the addresses that refer to track 63 are not optimal.

3. It's good practice to load the entirety of H1-11.1, and to then punch out the individual sub-libraries by means of J4-10.2 or J4-10.3. This enables you to load only the parts that you require using J1-10.0 or by means of J5-10.0. This allows you to verify checksums on load.

4. Instructions with zero addresses do not actually refer to location 0000, but are used to indicate special pseudo-instructions. The interpreter has 14 pseudo-instructions with zero addresses, plus the two special shift instructions `D000y` and `M000y`. These instructions set their address part to values from 0001 to 0009, meaning that the division and normal multiplication instructions must not use these addresses in interpreter mode.

INSTRUCTION	**RESULT, IF ADDRESS XXXX $\neq$ 0**
`Z`	(Address reg.) $-$ (`XXXX`) = 0? yes $\rightarrow$ skip next instruction; no $\rightarrow$ do not skip
`B`	(`XXXX`) $\rightarrow$ (Accumulator)
`Y`	(Address reg.) $\rightarrow$ address part of (`XXXX`)
`R`	Address of R plus 2 $\rightarrow$ address part of (`XXXX`)
`I`	(Address reg.) + (`XXXX`) $\rightarrow$ (Accumulator)
`D`	(Accumulator) / (`XXXX`) $\rightarrow$ (Accumulator)
`N`	(M Reg.) $\times$ (`XXXX`) + (Accumulator) $\rightarrow$ (Accumulator)
`M`	(M Reg.) $\times$ (`XXXX`) $\rightarrow$ (Accumulator)
`P`	(`XXXX`) $\rightarrow$ (M Reg.)
`E`	Address part of (`XXXX`) $\rightarrow$ address part of the (Accumulator)
`U`	Take next instruction from `XXXX`
`T`	Jump if (Accumulator) is negative
`-T`	Jump if (Accumulator) is negative or if PST is turned on
`H`	(Accumulator) $\rightarrow$ (`XXXX`)
`C`	(Accumulator) $\rightarrow$ (`XXXX`); 0 $\rightarrow$ (Accumulator)
`A`	(Accumulator) + (`XXXX`) $\rightarrow$ (Accumulator)
`S`	(Accumulator) $-$ (`XXXX`) $\rightarrow$ (Accumulator)

INSTRUCTION	RESULT, IF ADDRESS = 0 *
Z	Stop (PS-16); continue calculating after START is pressed
B	Make (Accumulator) positive
Y	Form complement of the (Accumulator)
R	$\sqrt{\texttt{(Accumulator)}} \rightarrow$ (Accumulator)
I	Input of floating-point data
D	(Accumulator) $/2^y \rightarrow$ (Accumulator) [address is not 0000, but 0001 to 0009]
N	Log_e (Accumulator) $\rightarrow$ (Accumulator)
M	(Accumulator) $\times 2^y \rightarrow$ (Accumulator) [address is not 0000, but 0001 to 0009]
P	Print (Accumulator)
E	Exit interpreter mode
U	Swap (Accumulator) and (M Reg.)
T	Make (Accumulator) negative
-T	Jump if (Accumulator) is negative or if PST switch is on
H	$e^{(\text{Accumulator})} \rightarrow$ (Accumulator)
C	Cosine (Accumulator) $\rightarrow$ (Accumulator)
A	Arctangent (Accumulator) $\rightarrow$ (Accumulator)
S	Sine (Accumulator) $\rightarrow$ (Accumulator)

6.2.1 Coding sheet for Horner's Method example

[**Translator's note:** This coding example and its data are simply oddly bunged on to the end of the chapter; best guess is that someone wanted a coding example of how to use the interpreter, and someone threw together a quick demo.]

Input keyword	STOP	Loc.	Opcode & Addr.	STOP	Address Contents	Comments
;0004200	’					Word counter to Track 04, Sector 00.
/0004200	’					Origin to Track 04, Sector 00.
		00	xR 1400	’		Set interpreted section pointer in interpreter
		01	xU 1400	’		Begin interpreting floating-point code
		02	xI 0000	’		Read coefficients from Flexowriter (see below)
		03	E 0018	’	4306	Set starting
		04	Y 0006	’	4306	address
		05	B 0013	’	Zero	
		06	A [ ]	’	a_n	This address will be overwritten
		07	xL 0002	’	$a(4306 + 2n)$	Increment address
		08	Y 0006	’	$a(4306 + 2n)$	register by 2
		09	xZ 4316	’		Check for completion
		10	U 0015	’		Not finished
		11	xP 0000	’		Print result before exit
		12	xE 0000	’		Exit interpreter
		13	xZ 0000	’		Stop machine (fixed-point opcode)
		14	U 0000	’		Swap accumulator into instruction counter
		15	xU 0000	’		Swap Accu. / M. Reg.
		16	xM 4304	’		Multipl. by x
		17	U 0006	’		Return for next term
		18	xZ 4306	’		Initial coefficient
000XXXX		19		’		

Data input sheet for Horner’s Method

This is sample data in the format that Horner’s Method demo program expects.

	INPUT CONTROL WORD (Starting Address)	S T O P	SIGNED DECIMAL NUMBERS Part 1 $d_1d_2\ldots d_7$	'	Part 2 d_8d_9	e_1e_2	S T O P
1	4304	'	1090000	'	00 +	09	'
2			0120000	'	00 -	09	'
3			0340000	'	00 -	09	'
3			0560000	'	00 -	09	'
3			0780000	'	00 -	09	'
3			0900000	'	00 -	09	'
9		'					'

Translator's Notes

Why do we have *two* floating point packages?

Modern programmers would look at this documentation and immediately start trying to figure out how to get rid of one of these two packages. Why would anyone want, or need, two separate floating point packages?

The following is speculation, based off the architectures of the two interpreters and years of experience with human nature, so take it with a grain of salt. If we ever have a real LGP-21 programmer around to ask, who actually *used* both, we can nail it down for sure. But having said that, it appears that there two different floating-point interpreters because there were two different audiences who needed floating point.

The first set of users would be people who are familiar with the LGP-21 and the fixed-point frctional architecture; who are comfortable thinking about numbers in terms of q and who have relatively lightweight requirements for floating-point operations. They will be writing code mostly in LGP-21 native mode, and occasionally dropping into floating point for a few calculations. Basic I/O of floating point works fine for them. Interpreter 1 is their choice.

The second set of users are primarily engineers, mathematicians, scientists...anyone whose native way of thinking about math is "things you do with real numbers". They want an instruction set that natively Does The Thing, using floating point numbers. Power-of-10 scaling makes perfect sense to them, but bit-twiddling and screwing around with representation is a distraction from the work they need to get done. Their attitude is effectively: *"I just want to work with actual numbers. These fixed-point fractions are too weird."*

There are tradeoffs; we can see some of them from the descriptions of the two interpreters:

- Interpreter 2 has a pad word following each mantissa-exponent pair, interpreter 1 doesn't. Is this because the all-float interpreter 2 runs *slower* than the limited interpreter 1? Or is it a brilliant speed hack that makes it run *faster*?
- Code written for interpreter 1 is *probably* faster, up to a point, but after the ratio of fixed-point to floating-point operations tilts far enough

toward floating-point, the overhead to move in and out of floating-point mode and the overhead to move *data* in and out will eventually make the interpreter 1 program larger than the same program written for interpreter 2, which doesn't have the mode-switch overhead, and eventually *may* make it slower.

- An Interpreter 2 program may be slower than an Interpreter 1 program, but it is likely to be *smaller* because the mode-switch overhead is no longer needed.

We won't know for sure until we find a programmer who used both, or two programmers who have had a 50-plus year argument about which is better.

Why interpreters?

A modern programmer's second thought is "why are the LGP-21's floating point implementations *interpreters?*" Couldn't we just write some nice libraries and call it a day?

On a more modern machine with a stack or other good support for subroutine calls, sure. (We'll pretend it doesn't have native floating point.) Let's say we're doing `fmul(fadd(a,b),fadd(c+d))`. We'd put a on the stack, call the conversion routine, push b on the stack, convert it, call the float-add function, put c on, convert, put d on, convert, call float-add, call float-multiply, done!

On the LGP-21, there are *so many* more operations. We have no stack, so everything has to go in temporary storage. We can't hold a two-word (mantissa and exponent) number in the regular accumulator, so we'll have to do something like this[31]:

Operation	**Estimated instructions**	**Description**
`fread(a)`	3	read value for `a`
`fread(b)`	3	read value for `b`
`fread(c)`	3	read value for `c`
`fread(d)`	3	read value for `d`
`fadd(a, b, t1)`	3	`a + b` to `t3`
`fadd(c, d, t2)`	3	`c + d` to `t4`
`fmul(t1, t2, t1)`	3	Final result in `t1`
`print(t1)`	3	Print it

Assuming that we can use a `B/R/U` calling sequence for each of these operations, that's 24 instructions to evaluate the expression; 4 words to store the values of `a`, `b`, `c`, and `d` that we read; and 6 for temp storage `t1`, `t2`, and `t3`; a total of 34 words. If we use an interpreter that "natively" understands floating point, we can do this:

[31] This is only the instructions needed to set up to *do* the operations; we're skipping the actual functions and the wiring necessary to do things like set up return addresses, find arguments, etc. Those are assumed to be library overhead and not something the caller codes...but that overhead exists too.

```
α          R   L0      ; Start of interpreter
α + 1      U   L0      ; Entry into the interpreter system
α + 2      I   0000    ; read a, b, c, d as float
α + 2      B   a       ; Load a
α + 3      A   b       ; Add b
α + 4      H   t1      ; Store result in t1
α + 5      B   c       ; Load c
α + 6      A   d       ; Add d
α + 7      H   t2      ; Store result in t2
α + 8      B   t1      ; Load t1
α + 9      A   t2      ; Multiply t2
α + 10     P   0000    ; Print result
α + 11     E   0000    ; Exit from the interpreter
α + n + 1  …           ; Resume fixed-point
```

To execute the exact same set of operations, the interpreter version only takes*12* (pseudo) instructions plus 10 words to store 5 floating-point numbers, 22 total, and is much simpler to understand. Fitting your program into much less storage is a serious consideration on a machine with only 4K words, especially since you'll be giving up memory to your libraries or your interpreter. (Interestingly, it should be noted that for Interpreter 2, the floating-point library functions are *smaller* than the corresponding fixed-point fractional ones.)

Coding example for Interpreter 2

This is a far better example of using all of the facilities of Interpreter 2 than the very simple comparison program above.

The Interpreter 2 Horner's Method example shows how a program would have been coded on an LGP-21 coding sheet, and then fed to one of the program loaders in 8 to make it resident in memory.

The program as designed is set up to allow the user to run the program, enter a set of exactly 5 coefficients (we're not up to handling dynamic loops here), and have them multiplied out:

$$f(x) = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

Reading an LGP-21 coding sheet

We've elected to keep the coding sheet for this particular example instead of writing out assembler code because it's useful to see how an LGP-21 program was coded by hand. Note that the locations start at 00 and go up; these line numbers act as "virtual" memory addresses, and can be referred to by other instructions by number, very much like a BASIC program. Instead of a `GOTO 200`, you have a `U 0015` to branch to the instruction on line 15.

The instructions with a leading `x` are *non-relocated*; their addresses are not modified when the program is loaded. For example, lines 0 and 1 set up Interpreter 2 by saving the address of the instruction 2 on from line 0 into the

interpreter itself, at the fixed address 1400. Additionally, the interpreter's special `I` and `P` instructions, to read and write data, have to have an address field of exactly 0000 to work properly, so those instructions are also marked as non-relocatable.

These two items together are key to understanding how the program works. A more modern[32] assembler would have had labels for the line-number branches that the coding sheet uses, but would still emit assembled code that looks more like the coding sheet than anything else.

How the Horner's Method coding sheet and data input sheet would have been used

To actually execute this on the LGP-21, an operator would have gone through the following sequence:

1. Power on the LGP-21 and ensure the Loader Utility Program is resident in memory. Load it if not.
2. Sit down at the Flexowriter and type out the coding sheet to produce a physical paper tape punched with the instruction bits and carriage-return stop codes (').
3. Take the finished source tape, walk over to the LGP-21 console, and feed it into the paper tape reader.
4. Set the console switches to execute the resident loader, and sets the input device to the paper tape reader, then tells the loader to load the tape.
5. The loader starts the tape and reads the initial tape headers (;0004200 and /0004200), targets Track 04 Sector 00, and begins shifting the relative instruction addresses to match their new home on the drum.
6. Once the program tape finishes loading, the operator removes it and inserts a Data Tape containing the actual floating-point numbers for x and the polynomial coefficients
7. The operator clears the manual controls and presses START on the physical control panel.
8. The computer starts executing the program, which starts Interpreter 2, pauses at Line 02 (xI 0000), readss in the data values from the data tape, and immediately begins crunching the numbers through the self-modifying loop.
9. The interpreter finishes the math, commands the Flexowriter to type out the final evaluation result on paper (`xP 0000`), drops out of interpretive mode, and triggers a hard system stop via Line 13 (`xZ 0000`).

[32] Or at least as modern as you're going to get for a computer built in 1963

10. If the operator wants to clear the current run and evaluate a completely fresh data tape using the same polynomial structure, they simply hit START again. The machine executes Line 14 (U 0000), which immediately vectors execution right back to the beginning of the program loop (relative line 0) to read the next input tape of x and the coefficients.

7 Format Conversion

7.1 DECIMAL-HEXADECIMAL TAPE CONVERSION

H4-10.0

PURPOSE

Program H4-10.0 converts arbitrarily relocatable, decimal-coded program tapes that are permissible for Program Input 1 (J1-10.0) into arbitrarily relocatable hexadecimal tapes, usable by 8.1. A conversion log is created with a listing of all instructions to be converted and whether or not they are relocatable, and a memory map of where the program is currently loaded.

Any corrections required in the program can be made by means of Program Input 1 8.1 before output. The resulting arbitrarily relocatable hexadecimal tape can be verified via checksum after output.

7.2 MEMORY REQUIREMENTS

STORAGE REQUIREMENTS

Program Space: 11 tracks and 35 sectors (739 words)
Temporary Storage: None The program is loaded into memory immediately following Program Input 1.

7.3 INPUT

H4-10.0 dynamically patches Program Input 1 upon execution, hooking its input routines to intercept the incoming program stream and construct the tracking tables. Once the target program has been completely read, H4-10.0 restores Program Input 1's original code to return it to normal operation. It must be resident before the program to be converted is loaded.

The decimal program to be converted should be stored at least 15 tracks above the starting address of Program Input 1, and loaded so that it could be executed, with the correct fill keyword (;) and modifier (/). All Program Input 1 input keywords work normally.
Program Input 1 as modified by H4-10.0 creates a bitmap table to track relocatable versus absolute instructions.

A second allocation bitmap tracks which memory regions contain valid data, ensuring only actually-populated sectors are eventually punched to tape. PIR's direct absolute memory write command (+) bypasses this tracking tracking mechanism and clobbers live memory cell contents directly; it does |emphnot update the instruction-patching or allocation bitmaps. Once the new program is loaded, execution can be intercepted via console switch PS-4 to hand control back to Program Input 1 for further memory patching prior to output.

When the operator resumes program execution at $L_0 + 0601$ to punch the output, the computer selects the Flexowriter for input and requests the selection keyword for the output unit. The Flexowriter's punch is selected by default;

press START to continue with this device, or enter a zero and press START to select the high-speed punch.

After the output unit is selected, the computer selects the Flexowriter again for input to read the output modifier. This is a number in decimal track/sector notation which will be subtracted from the modifiable addresses. This modifier must be four digits or smaller.

As an example, we'll assume that the program to be converted is loaded at 4000. If it is to be punched with relocatable address adjusted to be relative to 1000, one would enter an output modifier of 3000.[33]

7.4 EXECUTING THE PROGRAM

The program has the following five entry points:

$L_0 + 0600$	for reading the program to be converted
$L_0 + 0601$	for punching the converted program
$L_0 + 0602$	for restoring Program Input 1 if the tape fails verification
$L_0 + 0603$	for verifying the output
$L_0 + 0604$	for restoring Program Input 1 after successful verification

L_0 is the starting address of Program Input 1 (*not* this program!).

7.5 OUTPUT

Pressing START after entering the output modifier punches the converted program in arbitrarily relocatable hexadecimal form. Tables printed on the console indicate which portions of memory were punched and which instructions were marked as modifiable.

The converted words are punched in groups of 64 or fewer. Each group is preceded by a PIR keyword for hexadecimal fill (M), and each group is followed by a checksum. (Details of the format can be found at 8.1). After punching is complete, Program Input 1 is restarted.

After output, both the content and checksums of the output tape can be verified. To do this, place the just-punched tape into the reader and jump to $L_0 + 0603$, where L_0 is the starting address of Program Input 1. The computer selects the typewriter for input of a modifier, which must be the same as the one used when the program was punched. The computer selects the typewriter again to allow the input unit to be selected (if necessary). Press START to read and verify the tape.

The program is *not* reloaded into memory during verification; the contents of memory are used to verify that the tape matches. If a fill block on the tape fails validation, the console types out a character: `"w"` indicates a memory word does not match; `"e"` indicates that a checksum is incorrect. These error indications

[33]To relocate code to a higher memory address, the operator would input the tens-complement of the desired upward shift, forcing the subtraction routine to wrap around the address space allocation boundaries. This requires a lot of care.

apply to the specific hexadecimal fill block currently being verified; the tape reader will immediately halt, allowing the operator to inspect the block and locate the error.

8 Program input

The programs in this section are designed to make it easier to load programs into the LGP-21 than tediously typing them on the Flexowriter and depositing them word-by-word into memory. They can read blocks of data from tape, store it, and checksum the data read to ensure that it has been read correctly. They also perform the functions of a crude dynamic loader, making it possible to code relative to location 0000 and then load the code at an arbitrary address, relocating all relative addresses in the process. For 1963, this is a fairly sophisticated little pair of utilities.

`J1-10.0`, commonly known as `PIR`, can be modified by `H4-10.07` to automatically relocate tapes and repunch them as checksummed hexadecimal.

8.1 Program Loader 1

J1-10.0

PURPOSE

Program Input 1, J1-10.0 is a memory loader program. It facilitates the reading of hexadecimal tapes—whether arbitrarily relocated or not—and calculates a checksum during the reading process. It can also read and check arbitrarily relocated decimal programs.

Program J1-10.0 performs the following functions:

1. Reading, encoding, and storing instruction words to predefined memory locations.
2. Reading and storing hexadecimal or alphanumeric words to predefined memory locations without encoding.
3. Setting an address modifier in a program to a specific value.
4. Reading decimal addresses incremented by the modifier value.
5. Setting the "Start Fill" counter to a specific address value.
6. Reading specific instructions and executing them immediately.
7. Reading decimal addresses, followed by a jump either to this memory location or to the memory location specified by the address plus modifier.
8. Reading a decimal address; the contents of this memory location appear in the accumulator.
9. Reading, encoding, and storing arbitrarily relocatable decimal tapes.
10. Reading and storing hexadecimal tapes.
11. Selecting a specific input unit.

Hexadecimal tapes are usually generated by a hexadecimal dump program, such as the J4-10.0 program.

Memory usage

Hexadecimal input only	2 tracks
Combined input	3 tracks, 29 sectors
Buffer usage	None

textsf

- `MNNKHHHH` This keyword reads `NN` hexadecimal words in ascending order into consecutive memory locations, starting at memory location `HHHH` plus the modifier. The `K` is a store instruction. The input keyword contains a flag bit (bit 31); each of the NN words that are to be modified must also contain a flag bit.

  The address modifier (see below) is added to the operand address portion of each flagged word before it is stored.

  A checksum is calculated from all `NN` words as they are being stored plus the input keyword (all values are checksummed without the address modifier). If the calculated checksum does not match the checksum on the tape following these words, an error stop occurs. If the checksums match, the computer requests a new input keyword.

  Example: Let the modifier be 1200 (decimal) and the input keyword be M40K1201' (hexadecimal). The next 64 (40 hex) hexadecimal words are read and modified (if they have a flag bit) and are stored consecutively, beginning in memory location 3000.

- `VNNCHHHH` -

  The hexadecimal fill keyword for non-relocatable [absolute] tapes reads in `NN` hexadecimal words starting with memory location `HHHH`. `C` is a store instruction. The input keyword does not contain a flag bit. The most recently-entered address modifier value is added to the operand address portion of all hexadecimal fill words that have a flag bit as they are stored.

  A checksum is calculated from all `NN` words as they are being stored plus the input keyword (all values are checksummed without the address modifier). If the calculated checksum does not match the checksum on the tape following these words, an error stop occurs. If the checksums match, the computer requests a new input keyword.

- `VNNCTTSS`

  Example: The input keyword `V40C1100'` causes the following 64 (40 hex) hexadecimal words to be stored, beginning in memory location 1700.

  When reading hexadecimal tapes with `V` or `M` keywords, the Program Input Routine verifies each stored word; specifically, after a word is stored, the value in memory is compared with the read-in value. If the two values do not match, an error stop occurs.

- `/000TTSS`

  The Modifier Input Keyword sets the address modifier to the address `TTSS` contained within the input keyword.

  When entering hexadecimal tapes, this Address Modifier is added to all keywords and words that contain a flag bit.

  When entering decimal instruction words, the modifier is added to the operand addresses of all instructions except those preceded by an "X".

  The modifier remains unchanged until it is replaced by a new Modifier Input Keyword.

  Example: The input keyword `/0001200'` causes 1200 to be added to all flagged operand addresses and keywords.

- `.000TTSS'`

  The Absolute Stop and Jump Input Keyword causes the computer to stop. If the Start button is subsequently pressed, a jump occurs to cell `TTSS`. If the PS-32 switch is set to ON, execution continues at the target address.

  Example: The input keyword .0001663′ would cause the computer to jump to 1663 and halt execution (if PS-32 is off).

- `.100TTSS'`

  The Relative Stop and Jump Input Keyword causes the computer to stop. If the Start button is subsequently pressed, a jump occurs to cell `TTSS` plus the modifier. If the PS-32 switch is set to ON, the stop is suppressed.

  Example: If the modifier has been set to 1200, then the input keyword `.1001663'` would cause a jump to 2863.

- `S000TTyy'`

  The Selection Keyword selects the specific input unit designated by the decimal track address `TT`. (`yy` can be any arbitrary sector address; it is ignored, but must be supplied). A list specifying the corresponding input units for specific track addresses can be found in the LGP-21 Programming Manual.

  After entering the Selection Keyword, the computer stops. After pressing the Start button, the computer continues reading via the selected input unit.

  Restarting J1-10.0 by jumping to the start of the program will automatically reselect the primary Flexowriter for input. Reading will then continue via the Flexowriter until a new Selection Keyword selects a different unit.

  Example: If 32 refers to a second Flexowriter, then the keyword `S0003200'` selects this second typewriter for further input.

Decimal input

- `000TTSS'`

  The Decimal Fill keyword causes the input program to store decimal words in ascending order into consecutive cells, beginning with memory location `TTSS`.

  Any valid address followed by a stop code may be used. Every decimal input should be preceded by a Decimal Fill keyword. The "start fill" address is stored in the start-fill counter. This memory location of the subroutine contains the address for the decimal input. The counter is incremented by 1 each time after a word (not a keyword) is read and stored. The read-in words are stored consecutively until a new decimal fill keyword is read or the input is terminated.

  Example: The fill keyword `;0000400'` would store read-in words beginning with memory location 0400.

- `,00000NN'`

  The keyword for entering hexadecimal words causes `NN` hexadecimal words to be stored, specifically beginning in the memory cell whose address is held in the start-fill counter. `NN` must be specified in decimal and can take a value from 1 to 63. The stored words are not incremented by the address modifier. Leading zeros may be omitted for the hexadecimal words that are to be read in.

  Each hexadecimal word and keyword must be followed by a stop code. If a zero is to be entered, it is not necessary to write a `0`. A stop code alone is sufficient to indicate a zero. Each hexadecimal word must not consist of more than 8 characters.

  Example: The input keyword `,0000003'` followed by the three hexadecimal words `-2'JK73''` would cause the values `00000002`, `0000JK73`, and `00000000` to be stored into the next three consecutive cells.

- `A00000NN'`

  The keyword for entering alphanumeric words causes `NN` alphanumeric words to be stored, beginning in the memory cell whose address is held in the start-fill counter. The alphanumeric words are shifted two places to the left and stored without binary conversion. `NN` must be specified in decimal and can take a value from 1 to 63.

  Each word can contain up to five characters in 6-bit representation (if more than five characters per word are entered, only the last five are retained). The last stored word receives a group-end indicator, i.e., 1@30. All characters except tape feed (blank tape), delete code, and stop code can be entered; for example, lowercase letters can be stored.

  Example: The input keyword `A0000004'` followed by the four words `TOTAL' UNIT'S PRO'DUCED'` would cause the information `TOTAL UNITS PRODUCED`

to be stored into four consecutive memory locations without conversion into binary.

- `+00LTTSS'`

  The Instruction Input Keyword is treated by the subroutine exactly like an instruction.

  The arbitrary instruction `LTTSS` contained within the keyword—where `L` is any positive machine instruction and `TTSS` represents any valid address—is executed immediately after either the input of a subsequent hexadecimal word or the input of a stop code (if no subsequent word is required). The address portion of the input keyword is not altered by the Address Modifier.

  Each hexadecimal keyword and input word must be followed by a stop code. Left-side leading zeros within a word may be omitted. The computer returns to an input command without the execution of the instruction causing a branch. Jump instructions should be avoided.

  Note: The instruction contained within the input keyword does not alter the fill counter.

  Example: The input keyword `+00h1637'` followed by the hexadecimal word `73W08'` would cause the value `00073W08` to be stored in memory cell 1637.

- `-000TTSS'`

  The Sequential Display Input Keyword causes the contents of memory cell `TTSS` to appear in the accumulator when the computer stops. A start signal causes a return to an input command for a new keyword, provided that the `PST` switch is off. If the `PST` switch is on, the contents of `TTSS`$+i$ (where $i = 1, 2, \dots$) will appear in the accumulator.

  With each start signal, i is incremented by 1. It is also possible to obtain the address of the displayed word by switching to single-step operation and starting the machine. (The `B` instruction that fetched the information just displayed will appear in the accumulator.) `TTSS` may be any arbitrary valid address.

  Example: The input keyword `-0003263'` would directly display the contents of memory cell 3263 in the accumulator.

  Every entered word whose leftmost four bits correspond either to an "8" (negative instruction) or a zero (positive instruction) is treated as an instruction. The operand address is altered by the Address Modifier, unless the instruction is preceded by an "X". The subroutine allows the inclusion of index tags (used by some interpretive systems) immediately preceding the instructions. The inclusion of an "X" or an index tag does not alter the instruction in memory. If the leftmost four bits of a word correspond to any of the numbers 9 through 13, an error stop occurs (see the list of error stops).

Example: Let the Address Modifier be 0600. Different types of instruction words would be stored as follows:

Input	Storage
B 1234	B 1834
XB 1234	B 1234
800T 1234	800T 1834
80XT 1234	800T 1234
8B 1234	8B 1834
X8B 1234	8B 1234

The subroutine consists of two parts: hexadecimal input and decimal input. The hexadecimal input section can be used independently of the decimal input; however, the decimal input is dependent on the hexadecimal section, which must always be stored in memory.

CALLING SEQUENCE

The program is called manually by jumping to the starting address (regardless of which section is being used).

OPERATING INSTRUCTIONS

PROGRAM INPUT The tape contains the program in relocatable hexadecimal form with its own bootstrap loader. Operation is performed as follows:

a) Place the program tape into the typewriter's reader mechanism and release all levers.

b) Set the mode selector switch to `Manual Input` on the console.

c) Press `Start Read` on the Flexowriter.

d) Press `Fill Clear` on the console.

e) Press `Start Read`.

f) If the program is to be relocated, enter the starting address in hexadecimal form via the typewriter (typically the starting address is 0000); otherwise, omit this step.

g) Set the mode selector switch to `Single Step`.

h) Press `Execute` on the console.

i) Repeat steps b) through h) until "normal" is completely printed out by the computer.

j) Set the mode selector switch to `NORMAL`.

k) Press `START` on the console.

l) The program automatically continues reading in.

Note: If the PS-32 switch is turned *off*, the computer stops after reading in the hexadecimal input section. If the PS-32 switch is turned *on* or if START is pressed, the decimal input section or a hexadecimal object program tape will be read in.

PS-32 should be turned *off* before the decimal input section is completely read in; otherwise, the reader mechanism will continue to operate without tape after the read-in process is complete.

The bootstrap always occupies memory cells 0000 and 0002 . . . 0008 (absolute), while the remainder resides in the area designated for the hexadecimal input section.

INSTRUCTIONS FOR RELOADING

The bootstrap remains intact in memory if the program has not been relocated. To re-load the program without the bootstrap, proceed as follows:

a) Place the program tape into the reader, specifically aligning it at the blank tape section between the bootstrap and the program.

b) Set the mode selector switch to Manual Input on the console.

c) Enter C0000 via the typewriter.

d) Press Fill Clear on the console.

e) Enter the modifier in hexadecimal form (all 8 characters), or enter 8 zeros if the program is not to be relocated.

f) Set the mode selector switch to Single Step on the console.

g) Press Execute on the console.

h) Set the mode selector switch to Manual Input on the console.

i) Enter U0008.

j) Press Fill Clear on the console.

k) Set the mode selector switch to Single Step on the console.

l) Press Execute.

m) Set the mode selector switch to NORMAL on the console.

n) Press Start. The program will be read in automatically.

The C0000 and U0008 Instructions

In the LGP-21 instruction set, C is the Clear and Transfer instruction (storing the accumulator contents in memory), and U is the Unconditional Jump instruction. Entering U0008 manually forces the to skip the initial bootstrap block and jump straight to the start of the primary loader routine at location 0008.

REQUIRED INPUT/OUTPUT UNITS

Input units other than the standard typewriter must be specified by a selection code. Output units are not required.

8.1.1 THE EFFECT OF THE PROGRAM JUMP SWITCHES

- PS-32 (Program Switch 32)
  - OFF:
    - The computer stops before executing a jump when a Stop and Jump Input Keyword is used.
    - The computer stops after reading in the hexadecimal input section of the subroutine.
  - ON
    - No stop occurs when a Stop and Jump Input Keyword is used.
    - No stop occurs after reading in the hexadecimal or decimal section of the subroutine.
- PST (Program Switch-Typewriter)
  - OFF: Only the contents of a single, specific memory cell will appear in the accumulator.
  - ON: Consecutive memory cells can be loaded into the accumulator by repeatedly pressing the Start button.

MESSAGES

E (Input Error)

Cause:

- Hexadecimal Tape Input: This character indicates a checksum error.
- Decimal Tape Input: This character indicates an invalid input code.

Remedy:

a) Move the tape back to the last M or V input keyword.

b) Press START (the input unit does not need to be re-selected).

Remedy for an Invalid Input Code (the last word read contains the error):

a) Press Manual Input on the typewriter.

b) Set the mode selector switch to Manual Input.

c) Enter a jump to the starting address of the subroutine in hexadecimal form (i.e., U0000 or the respective starting address). If the subroutine begins in memory location 0000, steps c), e), and h) may be omitted.

d) Press Clear and Input (Fill Clear) (TR).

e) Set the mode selector switch to Single Operation (Single Step) (TR).

f) Press Execute (TR).

g) Set the mode selector switch to Normal (TR).

h) Press Start (TR). The typewriter lamp lights up.

i) Input the correct input code.

j) If the typewriter is the active input unit, release the Manual Input lever, and the tape will continue reading.

k) If another input unit is used, enter its selection code, press the Computer Start lever, then press Start (TR), and the tape will continue reading.

W (Write/Verify Error)

This printout indicates that a verification error occurred during the reading of a hexadecimal tape (i.e., the word actually stored in memory did not match the word read from the tape).

This error is remedied in the same manner as a checksum error.

Tape verification

By modifying two instructions in the subroutine, it can be used to verify hexadecimal tapes. For example, after a hexadecimal tape has been punched, alter the two instructions in the subroutine and then read the hexadecimal tape in.

The subroutine will read the tape (without storing it) and compare it word-for-word with memory. The checksums are also verified. If the comparison is unsuccessful, the program halts with the error stop "W" or "E". The tape is correct if it reads through without difficulty.

The instructions to be modified are:

Memory Location	Old Contents	New Contents
L + 24	H J	U 0025
L + 31	Y 0024	U 0032

Address Notation Legend

`Ø`	Machine instruction code, represented as a letter.
`TT`	Track portion of the address, in decimal (00–63).
`SS`	Sector portion of the address, in decimal (00–63).
`NN`	Number of words to read, in decimal.
`HHHH`	Hexadecimal memory address.

Instruction Keyword Quick Reference

Keyword Format	Basic Meaning
`ØTTSS`	Instruction – may be modified.
`XØTTSS`	Instruction – may not be modified.
`8ØØØTTSS`	Negative instruction – may be modified.
`8ØXØTTSS`	Negative instruction – may not be modified.
`IØTTSS`	Instruction with index tag – may be modified.
`XIØTTSS`	Instruction with index tag – may not be modified.

Detailed Operation Summary

Keyword	Operation & Detailed Description
MNNKHHHH	*Fill with Relocatable Words* Read NN hexadecimal words into memory, starting at location HHHH + modifier. If bit 31 is 1, operand address is modified. Checksum calculated and compared with word NN+1.
VNNCHHHH	*Fill with Non-relocatable Words* Read NN hexadecimal words into memory, starting at location HHHH. If bit 31 is 1, operand address is modified. Checksum calculated and compared with word following the NN words previously read.
/000TTSS	*Set Modifier* Sets an address modifier to TTSS. Modifier is added to all hexadecimal fields that have bit 31 set, and to the addresses of decimal instructions not preceded by an X.
.000TTSS	*Stop and Jump Absolute* Sets program counter to TTSS and stops execution. If PS-32 is ON, execution continues.
.100TTSS	*Stop and Jump Relative* Sets program counter to TTSS + modifier and stops. If PS-32 is ON, execution continues.

Keyword	Operation & Detailed Description
S000TTxx	*Device Select* Selects (decimal) input device TT (xx is ignored) and stops. To continue, press START.
;000TTSS	Decimal Fill Reads decimal instructions into memory starting at address TTSS.
,00000NN	Hexadecimal Fill Reads NN hexadecimal words into the following NN consecutive memory cells.
A00000NN	Alphanumeric Fill Reads NN alphanumeric words into the following NN consecutive memory cells.
+00ØTTSS	Execute Immediate Executes the instruction ØTTSS contained within the keyword. If the instruction has no operand, enter a stop code. Otherwise enter the operand word in hexadecimal notation an a stop code.
-000TTSS	Sequential Dump The contents of memory cell TTSS are shown in the console's accumulator display when the computer stops. After pressing START, the computer requests an command if PST is OFF. If PST is ON, pressing START causes the contents of TTSS+1 to appear in the accumulator, and so on. The address of the cell currently shown in the accumulator display appears in the accumulator if you switch to "Single Operation" and press Start.

8.2 Program Loader 2

J1-10.1

PURPOSE

Program J1-10.1 is a general-purpose memory loader and monitor utility, hereafter referred to as GPMon. It accepts arbitrarily relocatable decimal programs, hexadecimal words, and alphanumeric words. It can also read non-relocatable hexadecimal programs, calculate checksums, and verify them against the checksums on the hexadecimal tape.

Additionally, this program can load memory contents directly into the accumulator, execute instructions directly without storing them in memory, and perform manual program jumps.

Memory usage

Program storage	3 tracks
Buffer usage	None

COMMANDS

GPMon commands dictate how the utility processes incoming tape data; they are used to:

a) set the format for subsequent data words,

b) store specific words into memory, or

c) execute instructions immediately in the accumulator.

These input commands, and the formats of the words that follow them, are discussed on the following pages. (Please note that if you are entering GPMon commands from the keyboard, you must type the command followed by a stop code (') to mark the end of the input, and then press START to execute it.)

- `;000TTSS'`: Start Fill

  The Start Fill command sets the starting address `TTSS` for a series of words to be stored. This value is kept in the "fill counter" — a word within the GPMon program — and designates which cell will receive the next word. The counter is incremented every time a word other than a keyword is successfully read and stored.

  The words to be read in may be instructions, hexadecimal words, or alphanumeric words. Words are stored into consecutive locations until input ends or another fill keyword appears. The `TTSS` address must be in decimal track/sector notation, followed by a stop code. Any physical address in memory can be used. For example, the keyword `;0001600'` causes fill to start at address 1600.

- `[`*prefix*`]`∅`TTSS'` Instruction Input

  The parsing subroutine identifies a word as an instruction by inspecting its leading bits. A valid instruction must contain zeros in bit positions 1 through 3; bit 0 acts as the sign bit with a `0` for standard instructions or a `1` for negative instructions. When this bit pattern is recognized, the word is interpreted as an instruction consisting of an alphabetical command mnemonic (here represented by ∅) followed by a four-digit decimal operand address ($TTSS$)[34].

  If the accumulator is cleared prior to input, leading zeros (those preceding the instruction) may be omitted. The entire instruction is converted into its binary equivalent and stored in the cell specified by the fill counter.

  An optional prefix may be added to indicate that instructions are negative, non-relocatable, or both.

Prefix	Properties
(no prefix)	Positive, relocatable instruction
`-`	Negative, relocatable instruction
`X`	Positive, non-relocatable instruction
`-X`	Negative, non-relocatable instruction

  Non-relocatable instructions will be stored exactly as given and their address field will not be adjusted by the address modifier.

- `/000TTSS'` Set Address Modifier

  This command sets GPMon's internal address modifier to the decimal `TTSS` value specified. The modifier value is added to the operand address of all subsequent instruction words, unless the instruction is preceded by an `X`.

  The modifier remains in effect until it is changed by another Set Address Modifier command. The modifier may be set to any real address in decimal track/sector notation, followed by a stop code.

  Example: the keyword `/0001100'` sets the address modifier value to 1100. This value is added to the address field of all subsequent instructions that are not preceded by an "X":

Instruction Read	**Instruction Stored**
`B2611'`	`B3711`
`XB2611'`	`B2611`
`800Z0400'`	`800Z1500`
`80XZ0400'`	`800Z0400`

[34] Refer to the *LGP-21 Reference Manual*[9] for the defined instruction mnemonics.

- `.000TTSS'` Stop and Jump

  When this command is entered, the computer stops execution and sets the program counter to `TTSS`. If PS-32 is OFF, the computer halts. Pressing START resumes execution at `TTSS`. If PS-32 is ON, the computer jumps to location `TTSS` and continues execution immediately.

  Example: the keyword `.0002000'` with PS-32 OFF causes the computer to halt; when the START button is pressed, a jump to 2000 occurs and execution continues from there.

- `+[`*prefix*`]∅TTSS'` Direct Execute

  This keyword enables the programmer to have the computer execute an instruction directly. The instruction `∅TTSS` contained within the keyword (where ∅ represents a valid opcode symbol and `TTSS` is any real address in decimal track/sector notation) is executed following the entry of a hexadecimal word and/or a stop code. The second hexadecimal word is loaded into the accumulator.

  The address portion of the command entered is *not* modified by the address modifier; nor is any hexadecimal word entered as a secondary word. All keywords and additional hexadecimal words must be followed by a stop code. Leading zeros in the hexadecimal word may be omitted.

  The Direct Execute command allows you to immediately execute a command without writing it to memory.

  For example, if we wished to execute an `H` command to store the hexadecimal word `73W08` into location 1637, the command sequence `+00H1637'73W08'` causes the value `00073W08` to be stored in cell 1637. As a second example, the command `+00U1735''` (note the two stop codes to indicate "no second word") causes a jump to 1735.

  The keyword for the direct command does not alter the fill counter; any subsequent fill continues from the last fill location.

- `,00000NN'` Store Hexadecimal Words

  This command causes the following `NN` words to be stored as hexadecimal words in consecutive memory cells according to the current state of the fill counter. `NN` must be less than 64. The words to be stored must be in hexadecimal format and are not modified.

  For example, `;0002200',0000003',3W7'140Q''`[35] causes the memory to look like this after entry:

[35] This example string features three consecutive inputs after setting the fill address to 2200: 3W7', 140Q', and a final empty input before the last stop code. Because leading zeros can be dropped and a solo stop code represents zero, the monitor correctly shifts and pads them into full 32-bit words, parsing `''` correctly as zero, and clearing location 2202 to all zeroes.

Cell	Contents
2200	000003W7
2201	0000140Q
2202	00000000

A hexadecimal word may contain a maximum of eight characters. Any combination of the characters $0, 1, \ldots, 9$ and F,G,J,K,Q,W may be used; 00000000 is the smallest value and WWWWWWWW is the largest value the computer can accept. If a word is zero, only a stop code is required. Leading zeros may be omitted; every word must be followed by a stop code.

- VNNCHHHH' Checksummed Hexadecimal Fill

  This keyword causes the following NN hexadecimal words to be stored in consecutive locations, beginning with HHHH (a Start Fill command is not required to begin filling memory). The C is a clear instruction [sic][36]. This keyword uses hexadecimal notation and is normally generated by a hexadecimal output program, where it is found at the beginning of a block of 64 (or fewer) hexadecimal words.

  During input, a checksum is computed from the keyword and the following NN hexadecimal words. Once all NN words are stored, the program checks the PST switch. If it is OFF, the computed checksum is compared with the tape checksum word following the final word. If they are equal, reading continues; if they are unequal, an error stop occurs. See under "Error Stops." If PST is ON, the checksum comparison is skipped.

  For example, the keyword V1WC2W00' causes the next $1\mathrm{W}_{16} = 31_{10}$ hexadecimal words to be stored, beginning in location $2\mathrm{W}00_{16} = 4700_{10}$.

- A00000NN' Alphanumeric Word Fill

  This keyword permits the entry of alphanumeric information in a 6-bit format[37]. Information read in this manner can be punched back out directly by print commands without any conversion, or handled by an alphanumeric print program.

  Every word, with the exception of the last word of a block, must contain exactly 5 characters (the last word may contain fewer) and must be followed by a stop code. The quantity NN of words to be stored must be less than 64. The NN words are stored in consecutive cells (at $q = 29$), starting at the cell indicated by the current state of the fill counter.

  For example, the entry A0000005' DEPRE'SS ST'ART T'O CON'TINUE' causes the string "DEPRESS START TO CONTINUE" to be stored across five consecutive cells.

[36] It is not at all obvious *what* we are clearing; may need to run this in the simulator, snapshot memory, and compare before/after dumps to see whether this actually clears a memory location, or if it is a hijacked C instruction

[37] See table 3 in the *LGP-21 Reference Manual*[9]

- `-000TTSS'` Examine

  The Examine keyword loads words from memory into the accumulator. If the computer's console is equipped with an oscilloscope display, the programmer can view the contents of the accumulator without executing any instructions. Words are loaded sequentially, starting at `TTSS`. To examine the next word, press the START key.

  To exit[38] the Examine command and return to normal operation:

  1. Set the MODE switch to MANUAL INPUT.
  2. Press FILL CLEAR.
  3. Set the selector switch to NORMAL.
  4. Press START.

  For example, the keyword `-0002003'` loads the contents of location 2003 into the accumulator. After pressing START, the contents of 2004 appear, and so on. This process can be continued until the above sequence of operations interrupts the command.

- `S000TTss'` Input Device Select

  This command causes an alternate input device to be selected (other than the default console Flexowriter). The program reverts to console input when GPMon starts (via a branch to 0000) and following error stops. The selection codes and their corresponding units are listed on p. 30 in the *LGP-21 Reference Manual*[9].

  The selection codes must be specified in decimal track/sector notation `TTSS`, followed by a stop code (the sector portion value is irrelevant). For example, if another input device could be selected by the identification number `32`, the selection keyword `S0003200'` causes this device to be selected for this routine's input.

  Tapes to be read by the program should be capable of being typed out on the typewriter; that is, a carriage return should occur after every four to eight words[39].

  If incorrect information is read, the program selects the console typewriter[40], prints `err`, and halts. See "Error Stops".**??**

[38] Because this monitor instruction takes over the machine's primary execution cycle, stepping through and examining the contents of memory instead of executing instructions, it is necessary to force a manual reset of the input buffer via the console switches to restore standard program control.

[39] Data tapes should contain a carriage return (CR) after every four to eight words simply because of the mechanical realities of working with a hardware Flexowriter as console. If a continuous stream of data was read off a tape without embedded carriage returns, the physical typing element would reach the end of its travel at the right margin and continue to push against the margin as more characters are printed. This both imposes mechanical strain and overprints the input as an unreadable mess of characters at the edge of the log sheet.

[40] Automatically reverting control back to the primary console upon start/restart at 0000

PROGRAM EXECUTION

The following steps are necessary to load the program from tape:

1. Press the MANUAL INPUT lever on the Flexowriter.
2. Set the selector switch on the system console to MANUAL INPUT[41].
3. Press FILL AND CLEAR[42] on the console.
4. Set the selector switch to NORMAL.
5. Press the START button.

8.3 Operating Instructions

Initial Load

The program tape contains the software in two separate versions: a) a relocatable hexadecimal format with its own bootstrap routine, and b) a decimal format matching the standard coding sheet layout.

The input procedure is executed as follows:

a) Insert the tape into the typewriter's tape reader. Ensure that all typewriter levers are released.

b) Press the I/O button on the computer panel.

c) Set the MODE switch to MAN INPUT.

d) Press START READ on the typewriter.

e) Press FILL CLEAR on the computer panel.

f) Press START READ on the typewriter.

g) Set the MODE switch to ONE OPER on the computer panel.

h) Press EXECUTE on the computer panel.

i) Repeat steps c) through h) twice; then proceed to step j).

j) Set the mode selector switch to MAN INPUT on the computer panel.

---

or after an error stop is a fail-safe. If an external high-speed paper tape reader or secondary device feeds malformed data or fails mid-transfer, the system won't lock up; it immediately brings the main console back online so the operator can see the error state and intervene.

[41] This dual mechanical lockout prevented the computer from attempting to read from the console until the operator has explicitly prepared both the peripheral device and the central processing unit to accept manual keypresses.

[42] Pressing FILL AND CLEAR clears the machine's primary instruction and input registers, ensuring that when the mode switch is set to NORMAL and START is pressed, the processor begins execution from the starting address of the `J1-10.1` routine without processing residual data or garbage instructions.

k) Press START READ on the typewriter.

l) Press FILL CLEAR on the computer panel.

m) Set the MODE switch to ONE OPER on the computer panel.

n) Press EXECUTE on the computer panel.

o) Set the MODE switch to NORMAL on the computer panel.

p) Press START on the computer panel. The remainder of the tape is automatically read in, and the computer then comes to a halt.

q) Press MAN INPUT on the typewriter.

r) Press START on the computer panel.

s) The typewriter's MANUAL INPUT indicator lamp lights up, signaling that the computer is ready for manual keyboard input.

Re-reading the Tape

If the program fails to execute after step r), or if it becomes corrupted in memory after loading, the bootstrap routine may still be intact. In this case, the program can be reloaded without re-entering the bootstrap sequence by proceeding as follows:

a) Place the tape into the reader, skipping past the bootstrap section so that the bootstrap is not reread using the tape feed/leader.

b) Set the mode selector switch to "Manual Input" on the computer panel.

c) Press START READ on the typewriter.

d) Press FILL CLEAR on the computer panel.

e) Set the MODE switch to ONE OPER on the computer panel.

f) Press EXECUTE on the computer panel.

g) Set the mode selector switch to NORMAL.

h) Press START on the computer panel. The tape will be read in, and the computer will then come to a halt.

If the tape fails to read properly this time, the entire initial load bootstrap procedure must be repeated from the beginning.

Required Input/Output Units

If an input unit other than the typewriter's paper tape reader is to be selected, the selection command (S) must be used. The program selects the required output units automatically.

Meaning of the Program Switch Keys

Switch	State	Result
PS-32	OFF	The computer halts before jumping when a `Stop and Jump` command is executed.
	ON	The computer executes the `Stop and Jump` without halting.
PST	OFF	Checksums are compared when reading hexadecimal tapes.
	ON	Checksums are *not* compared when reading hexadecimal tapes.

Error Stops

Message	Meaning and Resolution
`err`	The computer has detected an error. After entering a hexadecimal fill keyword (`V`): a) Back up the tape to the last input keyword. b) Press START (on the computer). In the case of an incorrect or missing word: a) Press Manual Input (on the typewriter). b) Press START (on the computer). c) Enter the correct word. d) Release Manual Input (on the typewriter). e) Press START (on the computer). (Note: The last word read contains the error).

Note: Following an error stop, the subroutine automatically selects the typewriter for input.

REMARKS

This program cannot process data words (or constants) in decimal notation.

When a tape is read in, GPMon examines the sign and the three most significant bits of each input word[43] and jumps to the corresponding command.

A program can be broken down into logically independent sections, some of which can be reused in other programs. Examples of this include all kinds of subroutines, standard input and output routines, and programs for mathematical sub-problems. To make these sections available for use by other programs, they will need to be assigned when loaded to separate memory areas such that no two subprograms share the same memory block. To this end, these program sections should be coded relative to address 0000. By using the Start Fill (`;`) and Address Modifier (`/`) commands, the programmer can place the program

[43] In the LGP-21 machine word layout, this specific bit slice contains the bits that distinguish an executable instruction from a raw data pattern, enabling GPMon to use a fast software jump table.

into any arbitrary block of unused memory without disrupting the relationship of the instructions to one another. (Instructions in these sections which are not to be changed by the Address Modifier must be preceded by an `X`.)

On the other hand, if it is unnecessary to do so, the modification of an entire instruction group can be avoided by simply setting the address modifier to zero. There are no "reserved" blocks of memory; the programmer may use the LGP-21's memory as they see fit.

SUMMARY

Keyword	Meaning & Detailed Description
`∅TTSS`	Relocatable instruction Instruction to be altered by the address modifier.
`X∅TTSS`	Non-relocatable instruction Absolute instruction; not altered by the address modifier.
`-∅TTSS`	Relocatable negative instruction Instruction with negative sign bit or inverted logic flag; modifiable.
`-X∅TTSS`	Non-relocatable negative instruction Instruction with negative sign bit or inverted logic flag; not modified.
`;000TTSS`	Start Fill Store the following words (modifier excluded) in consecutive cells starting at `TTSS`.
`/000TTSS`	Set Address Modifier Set the address modifier base to `TTSS`. Add this value to all subsequent modifiable commands.
`.000TTSS`	Stop and Jump Halt (unless PS-32 is pressed) then jump to cell `TTSS` when the START button is pressed.
`+00∅TTSS`	Direct Command Immediately execute a command; a second (hexadecimal) word may be supplied as a data word.
`,00000NN`	Store Hexadecimal Words The next `NN` hexadecimal words are to be stored consecutively in the cells defined by the fill counter ($1 \leq NN \leq 64$).
`VNNCHHHH`	Hexadecimal Fill with Checksum Store `NN` hexadecimal words starting at `HHHH` ($1 \leq NN \leq 40_{16}$). For each group of words, including the command, a checksum is calculated. If PST is OFF, this checksum is compared with the checksum word punched on the tape just following the `NN` words. If PST is ON, no checksum comparison takes place.

Keyword	Meaning & Detailed Description
A00000NN	Store Alphanumeric Words The next NN words are to be stored in consecutive cells as alphanumeric words containing 5 six-bit characters each at $q = 29$ ($1 \leq NN \leq 63$).
-000TTSS	Examine Load the word at TTSS into the accumulator. Press START to load the word at TTSS+1 into the accumulator, and so on. Exit this mode by executing a manual jump to 0000 to restart GPMon.
S000TTxx	Input Unit Select Select the designated input unit TT for continuous input by GPMon. (The xx portion of the command may be anything and is ignored.)

Legend:

$\emptyset$	Any defined opcode
TT	Track address (decimal)
SS	Sector address (decimal)
NN	Number of words to read in (decimal)
HHHH	Address expressed in hexadecimal notation

Translator Notes

Rather than just a loader, GPMon is more of a programmer's interactive monitor.

Notably, PIR uses the Flexowriter stop code (') to terminate a command. GPMon does not. This means GPMon relies heavily on interactive execution via the Flexowriter keyboard, requiring you to physically hit the START COMP lever or START button on the computer to continue execution.

9 Data Input

9.1 Data Input 1

J2-10.0

PURPOSE

Program J2-10.0 reads data into the computer for use by a main program. The data, provided in decimal notation, is read in, converted to binary, shifted to the desired fractional bit position (q), and stored in designated memory locations. During each program execution run, the required quantity of decimal numbers can be converted and stored. All parameters regarding the type of conversion must be input alongside the data.

Data Input 1 functions as a subroutine and is called via a calling sequence in the main program. Before a jump to the subroutine occurs, the calling sequence must store the return address within the subroutine.

If an alternative input device is to be used instead of the default console typewriter, a suitable selection keyword must be included in the calling sequence. Upon return to the main program, all decimal numbers will have been converted and stored in the specified memory cells.

MEMORY USAGE

Program storage	4 tracks
Buffer usage	None

INPUT REQUIREMENTS

To process decimal numbers using J2-10.0, the following information must be entered in the specified order, with each entry followed by a stop code (`'`):

1. **Decimal Places (P):** The number of decimal places P for each subsequent data word, entered in decimal form ($0 \leq P \leq 11$).
2. **Scale Factor (Q):** The value representing the fixed-point scaling factor q at which the words should be stored. Q must consist of exactly three characters, including the sign ($-36 \leq Q \leq +46$).
3. **Starting Address:** The initial destination address of the memory allocation block, specified in decimal track/sector notation (`TTSS`).
4. **Decimal Data:** Decimal numbers consisting of a maximum of 7 digits, preceded by a sign. Negative numbers must explicitly include the minus sign and all leading zeros to the left. For positive numbers, both the sign and leading zeros may be omitted.
5. **Control Character `j`:** The letter "`j`" instructs the parser to stop reading data and prepare to read a new set of P, Q, and destination address parameters.

6. **Control Character `w`:** The letter "`w`" terminates the routine and executes a return to the main program.

REMARKS

If the decimal place count P, scale factor Q, or the target address sequence changes mid-tape, a `j` command must be injected, followed by the updated P and Q values, and the target address (any parameters that are identical must be repeated).

Example Input Stream:

```
4'+13'2318'627138'48390'j'
2'+20'3800'5134'664'-0000047'w'
```

This sequence performs the following operations:

- The decimal numbers `62.7138` and `4.8390` are converted at a scaling factor of $q = 13$ and stored in locations `2318` and `2319`, respectively.
- The control character `j` ends the block.
- The numbers `51.34`, `6.64`, and `-0.47` are then converted at a scaling factor of $q = 20$ and stored in locations `3800`, `3801`, and `3802`.
- The control character `w` stops input and triggers the return to the main program.

By default, the subroutine selects the standard console typewriter for input. A modified calling sequence to select an alternative input unit is available. When using the alternative sequence, the corresponding selection code must be loaded into the accumulator at $q = 23$ when the jump to the subroutine is executed.

CALLING SEQUENCES

Calling Sequence 1 selects the standard console typewriter for input. For alternative input devices, use Calling Sequence 2.

Calling Sequence 1 (Standard Input)

Location	Opcode	Address	Description
α	`R`	`L+138`	Set return address.
$\alpha + 1$	`U`	`L`	Enter subroutine.
$\alpha + 2$	…		Return point to main program.

The instruction at location $stores the return address (+2)$ within the subroutine structures. The instruction at $+1 executes a branch to the start of the subroutine (\texttt{L}).Location+$ 2 contains the first main program instruction to be executed after the subroutine finishes.

Calling Sequence 2 (Alternative Input Device)

Location	Opcode	Address	Description
$\alpha - 1$	`B`	`[TTSS]`	Load alternative device selection code.
α	`R`	`L+138`	Set return address.
$\alpha + 1$	`U`	`L+247`	Enter subroutine at device override entry point.
$\alpha + 2$	…		Return point to main program.
`TTSS`	`Z`	`SS00`	Input device selection code data word.

The instruction at $\alpha - 1$ loads the selection code for the new input unit into the accumulator. The instruction at α stores the return address $(\alpha + 2)$ into the subroutine. The instruction at $\alpha + 1$ executes a branch to entry point `L+247` of the subroutine. Here, the internal input device selection parameters are modified, and the subroutine runs normally until control returns to location $\alpha + 2$ via a branch instruction at `L+138`.

PROGRAM LOADING INSTRUCTIONS

The routine tape contains the program in two distinct formats:

a) A relocatable hexadecimal format compatible with loader program J1-10.0.

b) A relocatable decimal format utilizing the standard coding sheet layout.

EXECUTION TIME

The program processes and converts decimal data into binary at an average rate of 35 five-digit numbers per minute.

REQUIRED INPUT/OUTPUT UNITS

The standard console typewriter is selected for input by default. No hardware output device is required. The device assignment can be altered dynamically via Calling Sequence 2 (see above).

OVERFLOW

Any arithmetic overflow conditions occurring during data entry or base conversion are intentionally ignored. The system's overflow register flag is automatically reset to zero before control returns to the main program.

9.2 Data Input 2

J2-10.1

PURPOSE

Program J2-10.1 reads data into the computer for use by a main program. The decimal data is read, converted to binary, shifted to the desired fractional bit position (q), and, if more than one number is processed, stored in consecutive designated cells. If only a single number is read, it remains in the accumulator after being shifted to its corresponding q. Any number of decimal values can be converted and stored via a single program call. The main program must supply all configuration parameters regarding the conversion.

This program functions as a subroutine called by a sequence in the main program. Prior to branching to the subroutine, the calling sequence must store both the starting address of the **Conversion Control List** and the return address inside the subroutine structure. Upon return to the main program, all data will have been converted and either stored in memory or left in the accumulator.

MEMORY USAGE

Program storage	4 tracks, 48 sectors
Buffer usage	None

INPUT REQUIREMENTS

The data to be processed must be entered as 7-digit decimal numbers preceded by a sign. For positive numbers, both the sign and leading zeros may be omitted. Before jumping to the subroutine, the main program must have stored a **Conversion Control List** in consecutive memory locations. The structure of this list is as follows:

1. **Word 1 (N):** The number of data words to process, specified using track/sector notation (i.e., modulo 64 format).
2. **Word 2 (L):** The initial destination address for the data, specified in decimal track/sector notation.
3. **Word 3 (Q and P):** A single combined configuration word:
   (a) Q **(Track Portion):** The target fixed-point scale factor q entered as a decimal track address ($-47 \leq Q \leq +36$).
   (b) P **(Sector Portion):** The number of decimal places entered as a decimal sector address ($0 \leq P \leq 11$).

CONTROL MATRIX FLAGS

The subroutine utilizes the sign bits and specific instruction bits of the control list to determine state persistence from previous execution runs: * **Negative N:** If N is embedded in a negative instruction format (sign bit = `1`), the word count from the previous run is preserved. * **Negative L:** If L is inside a negative instruction format, the storage destination address from the previous run is reused. All standard instruction bits for the N and L command entries must otherwise be `0`. * **Negative Q:** If the combined Q/P word is negative, the value of Q (q) is treated as a negative scale factor. * **Initialization Override:** If the very first bit of the instruction cell containing Q and P is a `1`, the program reuses the exact configuration state of the previous execution run.

Special Case Values: * **$N = 0$:** The routine reads an unlimited stream of data words continuously until the block-termination character "`w`" is encountered on the tape. * **$N = 1$:** The single data word read is converted and left in the accumulator at the specified q; it is *not* written to the destination address L. * **Negative List Address Instruction:** The accumulator must hold an instruction containing the starting address of the control list when entering the subroutine. If this setup instruction is negative, the subroutine uses the exact control values from the prior run. * **Control Pre-Load Optimization:** If the first bit of the instruction containing the control list address is set to a `1`, the subroutine pre-loads and sets up the new control values **without reading any data tape**. This allows setup parameters to be configured just once for an entire sequence of subsequent operations.

Alternative Input Unit Selection: If the input hardware device is to be changed, an alternative calling sequence must be used (by default, the subroutine reverts to the console typewriter). A device change requires a **fourth word** added to the Conversion Control List. This extra word must contain the destination device identification selection code specified as a decimal track address. This assignment remains active until explicitly overwritten by a new selection word.

APPLICATION EXAMPLES

Note: Addresses enclosed in parentheses represent relative internal offsets inside the subroutine space.

Example 1: Standard Block Load

Location	Opcode	Address	Description
0500	`000Z`	`2430`	Starting address of Control List
0610	`000B`	`0500`	Load list pointer into accumulator
0611	`000R`	`(0442)`	Set return address
0612	`000U`	`(0445)`	Jump to subroutine entry point
0613	…		Main program execution resumes
2430	`000Z`	`0126`	$N = 90$ words ($01_{track} \times 64 + 26_{sector} = 90_{10}$)
2431	`000Z`	`4200`	L = Destination cell 4200
2432	`000Z`	`1205`	$Q = q = 12$, $P = 5$ decimal places

This configuration instructs the subroutine to ingest 90 decimal words from the tape, convert them to binary, scale them to $q = 12$ with 5 decimal places, and store them sequentially beginning at cell `4200`. Control then returns to main program cell `0613`.

Example 2: Indefinite Tape Stream with Reused Base Address

Location	Opcode	Address	Description
0501	`000Z`	`2433`	Starting address of Control List
0729	`000B`	`0501`	Load list pointer
0730	`000R`	`(0442)`	Set return address
0731	`000U`	`(0445)`	Jump to subroutine
0732	…		Main program execution resumes
2433	`000Z`	`0000`	$N = 0$ (Read indefinitely until `w`)
2434	`800Z`	`0000`	Negative instruction flag: Reuse destination address L from previous ru
2435	`800Z`	`0109`	Negative scale flag: $Q = q = -01$, $P = 9$ decimal places

This configuration processes an unknown length of data words until a stop code "`w`" marks the end of the block. The numbers are scaled to $q = -1$ with 9 decimal places and stored sequentially starting directly from the last memory address boundary computed during the previous subroutine invocation.

Example 3: Single Word Accumulator Read

Location	Opcode	Address	Description
0502	000Z	2436	Starting address of Control List
1925	000B	0502	Load list pointer
1926	000R	(0442)	Set return address
1927	000U	(0445)	Jump to subroutine
1928	…		Main program execution resumes
2436	000Z	0001	$N = 1$ word (Leave in Accumulator)
2437	000Z	1340	$L = 1340$ (Dummy address required by parser, but ignored)
2438	000P	0000	First bit is 1: Reuse Q (q) and P states from previous run

The subroutine reads a single decimal value from the tape, converts it using the exact scale settings leftover from the last run, and leaves the value directly in the accumulator. No data is written to cell 1340.

Example 4: Total Control List Persistence

Location	Opcode	Address	Description
0503	800Z	0000	Negative pointer instruction: Reuse prior configuration
3618	000B	0503	Load negative pointer execution command
3619	000R	(0442)	Set return address
3620	000U	(0445)	Jump to subroutine
3621	…		Main program execution resumes

Because the pointer token at 0503 has its sign bit set to negative, the subroutine bypasses parsing new parameters entirely and mirrors the execution profile of the preceding run.

Example 5: Non-Reading Control Parameter Ingestion

Location	Opcode	Address	Description
0504	000P	2439	Opcode is P (First bit is 1): Configuration initialization entry point
1231	000B	0504	Load control initialization command
1232	000R	(0442)	Set return address
1233	000U	(0413)	Jump to secondary initialization entry point
1234	…		Main program execution resumes
2439	800Z	0000	Negative instruction flag: Reuse N from previous run
2440	000Z	1500	Set new target address $L = 1500$
2441	000Z	2700	Set new scale settings: $Q = q = 27$, $P = 0$ decimal places
2442	000Z	0400	Hardware override: Select alternative input device code 04

Because the entry command at 0504 uses an initialization opcode format starting with a 1 bit pattern, and points to subroutine entry point (0413), the routine loads the new parameters, updates the hardware pointer to device 04, but

does not read any data from the tape reader. Control returns immediately to cell `1234`.

CALLING SEQUENCES

Utilize Calling Sequence 1 if the input unit selection remains unchanged. Use Calling Sequence 2 to declare a new input device structure.

Calling Sequence 1 (Retain Input Unit)

Location	Opcode	Address	Description
$\alpha - 1$	`B`	`[List]`	Load the pointer instruction containing the Control List address.
α	`R`	`L+442`	Set return address.
$\alpha + 1$	`U`	`L+445`	Jump to subroutine main entry point.
$\alpha + 2$	…		Return point to main program.

Calling Sequence 2 (Modify Input Unit)

Location	Opcode	Address	Description
$\alpha - 1$	`B`	`[List]`	Load the pointer instruction containing the Control List address.
α	`R`	`L+442`	Set return address.
$\alpha + 1$	`U`	`L+413`	Jump to device-configuration entry point.
$\alpha + 2$	…		Return point to main program.

The input hardware unit used by the system will dynamically match whichever device selection code is currently declared inside the Conversion Control List.

OPERATING INSTRUCTIONS

1. **Execution Time:** The program processes and converts decimal data into binary at an average rate of 40 five-digit numbers per minute.

2. **Program Loading Instructions:** The routine tape contains the software in two distinct formats:

   a) A relocatable hexadecimal format compatible with loader program J1-10.0.

   b) A relocatable decimal format utilizing the standard coding sheet layout.

3. **Required Input/Output Units:** The subroutine automatically selects the standard console typewriter for input, unless an alternative unit override has been explicitly established via Calling Sequence 2. No hardware output devices are required.

4. **Overflow:** Any arithmetic overflow conditions occurring during data entry or base conversion are explicitly ignored upon input. The CPU's internal overflow register flag is automatically cleared and reset to zero before the routine exits back to the main program.

TECHNICAL NOTE

To maintain maximum mathematical accuracy and prevent underflow or truncation errors during fixed-point binary conversion, the programmer should always specify the smallest practical scale factor value Q (q) necessary for the expected data range.

9.3 Data Input 3

J2-10.2

PURPOSE

Program J2-10.2 converts a single signed 7-digit decimal number into binary, or reads and converts it in a single operation.

This program functions as a subroutine called by a sequence in the main program. Prior to branching to the subroutine, the calling sequence must store the return address within the subroutine structures. Depending on the calling sequence chosen, the integer to be converted must already reside in the accumulator. Upon return to the main program, the converted binary number is left in the accumulator at @30.

MEMORY USAGE

Program storage	1 track, 9 sectors (*Note: The program must be loaded to start at sector 00 of an arbitrary track.*)
Buffer usage	None

INPUT REQUIREMENTS

The input variable is a decimal number N, which is either read directly from tape by the subroutine or supplied internally by the main program:

- **Tape Input via Subroutine:** N may consist of up to seven decimal digits plus a sign. For positive numbers, the plus sign may be omitted. For negative numbers, the minus sign must explicitly precede the seven digits.
- **Internal Input via Accumulator:** If the number is supplied by the main program, it must already be loaded into the accumulator at either @30 or @31, depending on the calling sequence used.
- **Sign Bit Positions:** The sign of the pre-loaded number is determined dynamically by specific bit slices:
  - If N is at @30, the sign is indicated by **bit position 2**.
  - If N is at @31, the sign is indicated by **bit position 3**.

  A `1` in the respective bit position designates a negative number, while a `0` designates a positive number.

CALLING SEQUENCES

Calling Sequence 1 must be used if N is to be read from tape by the subroutine. If N is supplied internally by the main program, choose Calling Sequence 2 (for N at @31) or Calling Sequence 3 (for N at @30).

Calling Sequence 1 (Tape Input)

Location	Opcode	Address	Description
α	`R`	`L+44`	Set return address.
$\alpha+1$	`U`	`L`	Enter subroutine at tape-read entry point.
$\alpha+2$	…		Return point to main program; N read from tape.

The instruction at location α stores the return address ($\alpha+2$) inside the subroutine. The instruction at $\alpha+1$ executes a branch to the initial starting address of the subroutine (`L`). Location $\alpha+2$ contains the first main program instruction executed after the subroutine runs.

Calling Sequence 2 (Accumulator Input at @31)

Location	Opcode	Address	Description
$\alpha-1$	`I`		Read/Input N into the accumulator.
α	`R`	`L+44`	Set return address.
$\alpha+1$	`U`	`L+2`	Enter subroutine at @31 handling entry point.
$\alpha+2$	…		Return point to main program.

The instruction at $\alpha-1$ reads or brings N into the accumulator. The instruction at α stores the return address ($\alpha+2$). The instruction at $\alpha+1$ executes a branch to entry offset `L+2`. Note that a `1` (negative) or `0` (positive) in bit position 3 of the accumulator dictates the sign evaluation, and N is processed at @31.

Calling Sequence 3 (Accumulator Input at @30)

Location	Opcode	Address	Description
$\alpha-1$	`B`	`[Dest]`	Bring N from memory into the accumulator.
α	`R`	`L+44`	Set return address.
$\alpha+1$	`U`	`L+58`	Enter subroutine at @30 handling entry point.
$\alpha+2$	…		Return point to main program.

The instruction at $\alpha-1$ loads N from memory into the accumulator. The instruction at α sets the return address ($\alpha+2$). The instruction at $\alpha+1$ executes a branch to entry offset `L+58`. Note that a `1` (negative) or `0` (positive) in bit position 2 of the accumulator dictates the sign evaluation, and N is processed at @30.

OUTPUT

Upon return to the main program sequence, the converted binary representation of the decimal number N resides completely within the accumulator scaled exactly to @30.

OPERATING INSTRUCTIONS

1. **Execution Time:** The program requires approximately 1.1 seconds to read and binary-convert a single 5-digit decimal number.

2. **Program Loading Instructions:** The routine tape contains the software in two distinct formats:

   a) A relocatable hexadecimal format compatible with loader program J1-10.0.

   b) A relocatable decimal format utilizing the standard coding sheet layout.

3. **Required Input/Output Units:** When executing Calling Sequence 1, the subroutine automatically selects the standard console typewriter for input. For Calling Sequences 2 and 3, device selection is purely a function of the main program logic. No hardware output devices are required.

4. **Overflow:** Arithmetic overflow states are neither evaluated nor altered by this program.

9.4 Data Input 4

J2-10.3

PURPOSE

Program J2-10.3 reads data into the computer for use by a main program. It reads a predetermined number of decimal data words, converts them to binary, aligns them according to a specified decimal point (P), and stores them in consecutive memory cells at a designated scale factor. During a single program execution run, any quantity of decimal numbers can be converted and stored. All parameters regarding the conversion must be supplied by the main program.

The program functions as a subroutine called via a calling sequence in the main program. The calling sequence must supply the starting address of the Conversion Control List as well as the return address. Upon return to the main program, all data has been converted to binary and stored.

MEMORY USAGE

Program storage	3 tracks, 52 sectors
Buffer usage	None

INPUT REQUIREMENTS

The decimal numbers to be converted can consist of an algebraic sign and up to seven digits. A negative number must explicitly consist of its minus sign and seven digits; for positive numbers, both the sign and leading zeros may be omitted.

The number of decimal values N to be read is limited only by the available memory space. The number of decimal places P must satisfy: $0 \leq P \leq 11$. The scale factor value must satisfy: $-36 \leq$ scale factor $\leq +47$.

The Conversion Control List, which occupies consecutive memory cells, is structured as follows:

1. **Word 1 (Q and P):** A single combined configuration word:
   (a) Q **(Track Portion):** The target fixed-point scale factor entered as a two-digit decimal track address.
   (b) P **(Sector Portion):** The number of decimal places entered as a two-digit decimal sector address.
2. **Word 2 (N):** The number of words to be read, specified in track/sector notation (i.e., modulo 64 format).
3. **Word 3 (Address):** The initial destination address for the data, specified in decimal track/sector notation.

The scale factor is negative if the instruction containing Q is formatted as a negative instruction (sign bit = `1`). All other instructions in the control list must be positive.

Additionally, an instruction containing the starting address of the Control List must be present. This instruction must reside in the accumulator when the jump to the subroutine is executed.

If the selection of the input unit is to be modified, a special calling sequence must be used (by default, the subroutine selects the standard console typewriter). This requires a **fourth word** in the Conversion Control List, which contains the desired device selection code as a decimal track address. This selection code remains effective until explicitly replaced by a different one.

CALLING SEQUENCES

If the standard console typewriter is to be used as the input device, Calling Sequence 1 must be used. Calling Sequence 2 is intended for variable device selection override.

Calling Sequence 1 (Standard Input)

Location	Opcode	Address	Description
$\alpha - 1$	`B`	`[List]`	Load starting address of Control List.
α	`R`	`L+351`	Set return address.
$\alpha + 1$	`U`	`L+308`	Enter subroutine at standard entry point.
$\alpha + 2$	…		Return point to main program.

The instruction at $\alpha - 1$ loads the instruction containing the starting address of the Control List into the accumulator. The instruction at α stores the return address ($\alpha+2$) within the subroutine structures. The instruction at $\alpha+1$ causes a jump to location `L+308` of the subroutine, where `L` is the base address of the subroutine. Location $\alpha + 2$ contains the first instruction of the main program to be executed after completing the subroutine.

Calling Sequence 2 (Variable Device Selection Override)

Location	Opcode	Address	Description
$\alpha - 1$	`B`	`[List]`	Load starting address of Control List.
α	`R`	`L+351`	Set return address.
$\alpha + 1$	`U`	`L+300`	Enter subroutine at device-selection entry point.
$\alpha + 2$	…		Return point to main program.

Calling Sequence 2 is identical to Calling Sequence 1, except for the instruction at $\alpha + 1$. This instruction causes a jump to `L+300`, so that the desired device selection is performed based on the selection code provided in the fourth word of the Control List.

OPERATING INSTRUCTIONS

1. **Execution Time:** The program reads and binary-converts decimal data at an average rate of 40 five-digit numbers per minute.

2. **Program Loading Instructions:** The routine tape contains the software in two distinct formats:

   a) A relocatable hexadecimal format compatible with loader program J1-10.0.

   b) A relocatable decimal format utilizing the standard coding sheet layout.

3. **Required Input/Output Units:** The subroutine automatically selects the standard console typewriter for input, unless an alternative device selection override has been established via Calling Sequence 2. No hardware output devices are required.

4. **Overflow:** Any arithmetic overflow conditions occurring during data entry or base conversion are ignored upon input, but the overflow flag is automatically reset to zero prior to returning to the main program.

EXAMPLES

The following are examples of Conversion Control Lists:

Example 1: Standard Typewriter Block Ingestion

Location	Opcode	Address	Description
0500	`Z`	`2420`	Starting address of Control List
0532	`B`	`0500`	Load list pointer into accumulator
0533	`R`	`L+0351`	Set return address
0534	`U`	`L+0308`	Jump to standard entry point
0535	…		Main program execution resumes
2420	`Z`	`1304`	$Q = @13$, $P = 4$ decimal places
2421	`Z`	`0146`	$N = 110$ words ($01_{\text{track}} \times 64 + 46_{\text{sector}} = 110_{10}$)
2422	`Z`	`1500`	Target starting address for data block

This example instructs the subroutine to read 110 decimal numbers from the tape, convert them to binary, align them for $P = 4$ decimal places, and store them in consecutive cells starting at location `1500` at scale factor `@13`. Control then returns to cell `0535` of the main program.

Example 2: Alternative Device Selection with Negative Scale Factor

Location	Opcode	Address	Description
0501	`Z`	`2423`	Starting address of Control List
0636	`B`	`0501`	Load list pointer
0637	`R`	`L+0351`	Set return address
0638	`U`	`L+0300`	Jump to device-selection override entry point
0639	…		Main program execution resumes
2423	`800Z`	`0108`	Negative instruction flag: $Q =$ @-01, $P = 8$ decimal places
2424	`Z`	`0019`	$N = 19$ words ($00_{\text{track}} \times 64 + 19_{\text{sector}} = 19_{10}$)
2425	`Z`	`1820`	Target starting address for data block
2426	`Z`	`0800`	Device selection code data word (Device code `08`)

This example instructs the subroutine to read 19 decimal numbers, convert them to binary, align them for $P = 8$ decimal places, and store them in consecutive cells starting at location `1820` at scale factor `@-01`. Input is executed via the peripheral unit corresponding to code `08`. Control then returns to cell `0639` of the main program.

TECHNICAL NOTE

To maintain maximum mathematical accuracy, the smallest practical scale factor value should be used. The program does not halt if a number cannot be properly represented or stored at the intended scale factor target; instead, it continues execution using an invalid, meaningless value.

9.5 Data Input 5

J2-10.4

PURPOSE

Program J2-10.4 reads data into the computer for use by a main program. It reads an indefinite quantity of decimal data words, converts them to binary, aligns them according to a specified decimal point (P), and stores them in consecutive memory cells at a designated scale factor. During a single program execution run, any quantity of decimal numbers can be converted and stored. All parameters controlling the conversion must be supplied by the main program.

Data Input 5 functions as a subroutine called via a calling sequence in the main program. The calling sequence must supply the subroutine with the Conversion Control List as well as the return address. Upon return to the main program, all data has been converted and stored.

MEMORY USAGE

Program storage	3 tracks, 48 sectors
Buffer usage	None

INPUT REQUIREMENTS

Data Input 5 can convert decimal numbers consisting of an algebraic sign and up to seven digits into binary. A negative number must explicitly consist of its minus sign and seven digits; for positive numbers, both the sign and leading zeros may be omitted.

The number of decimal places P must satisfy: $0 \leq P \leq 11$. The scale factor value Q must satisfy: $-36 \leq Q \leq +47$.

The Conversion Control List, which must be stored in consecutive cells before entering the subroutine, is structured as follows:

1. **Word 1 (Q and P):** A single combined configuration word:
   (a) Q **(Track Portion):** The target fixed-point scale factor entered as two decimal digits in the track address field.
   (b) P **(Sector Portion):** The number of decimal places entered as two decimal digits in the sector address field.
2. **Word 2 (Address):** The initial destination address for the data, specified in decimal track/sector notation.

The scale factor Q is negative if the instruction containing it is formatted as a negative instruction (sign bit = `1`). All other instructions in the control list must be positive.

Additionally, an instruction containing the starting address of the Control List must be present. This instruction must reside in the accumulator when the jump to the subroutine is executed.

The ingestion of decimal numbers is terminated by the specific stop code keyword `w'`.

If the selection of the input unit is to be modified, a special calling sequence can be used (by default, the subroutine selects the standard console typewriter). This requires an additional **third word** in the Conversion Control List, which follows the other two. This instruction must contain the device selection code for the desired unit in its decimal track address field. This selection code remains effective until explicitly replaced by a different selection code.

CALLING SEQUENCES

Calling Sequence 1 should be used if the standard console typewriter is chosen as the input device; otherwise, use Calling Sequence 2 to select a specific alternative peripheral unit.

Calling Sequence 1 (Standard Input)

Location	Opcode	Address	Description
$\alpha - 1$	`B`	`[List]`	Load starting address of Control List.
α	`R`	`L+347`	Set return address.
$\alpha + 1$	`U`	`L+308`	Enter subroutine at standard entry point.
$\alpha + 2$	…		Return point to main program.

The instruction at $\alpha - 1$ loads the instruction containing the starting address of the Control List into the accumulator. The instruction at α stores the return address ($\alpha + 2$) inside the subroutine structures. The instruction at $\alpha + 1$ causes a jump to location `L+308` of the subroutine, where `L` is the base address of the subroutine. Location $\alpha + 2$ contains the first instruction of the main program to be executed after completing the subroutine.

Calling Sequence 2 (Variable Device Selection Override)

Location	Opcode	Address	Description
$\alpha - 1$	`B`	`[List]`	Load starting address of Control List.
α	`R`	`L+347`	Set return address.
$\alpha + 1$	`U`	`L+300`	Enter subroutine at device-selection entry point.
$\alpha + 2$	…		Return point to main program.

Calling Sequence 2 differs from Calling Sequence 1 only by the instruction at $\alpha + 1$. This instruction causes a jump to `L+300` of the subroutine, so that the corresponding peripheral unit is selected based on the selection code provided in the Control List.

OPERATING INSTRUCTIONS

1. **Execution Time:** The program reads and binary-converts data at an average rate of 40 five-digit numbers per minute.

2. **Program Loading Instructions:** The routine tape contains the software in two distinct formats:

   a) A relocatable hexadecimal format compatible with loader program J1-10.0.

   b) A relocatable decimal format utilizing the standard coding sheet layout.

3. **Required Input/Output Units:** The subroutine automatically selects the standard console typewriter for input. This assignment can be altered dynamically via alternative Calling Sequence 2 (see above). No hardware output devices are required.

4. **Overflow:** Any arithmetic overflow conditions occurring during data entry or base conversion are ignored upon input, but the overflow register flag is automatically reset to zero prior to returning to the main program.

EXAMPLES

The following examples illustrate the application of the Conversion Control List:

Example 1: Indefinite Stream via Standard Typewriter

Location	Opcode	Address	Description
1700	`Z`	`2900`	Starting address of Control List
2000	`B`	`1700`	Load list pointer into accumulator
2001	`R`	`L+0347`	Set return address
2002	`U`	`L+0308`	Jump to standard entry point
2003	…		Main program execution resumes
2900	`Z`	`0100`	$Q =$ @01, $P = 0$ decimal places
2901	`Z`	`1500`	Target starting address for data block

This example instructs the subroutine to read an indefinite quantity of data words from the tape, convert them to binary, align them for $P = 0$ decimal places, and store them in consecutive cells starting at location `1500` at scale factor `@01`. Control then returns to cell `2003` of the main program.

Example 2: Indefinite Stream via Alternative Unit Override

Location	Opcode	Address	Description
1701	Z	2902	Starting address of Control List
3000	B	1701	Load list pointer
3001	R	L+0347	Set return address
3002	U	L+0300	Jump to device-selection override entry point
3003	…		Main program execution resumes
2902	800Z	0209	Negative instruction flag: Q = @-02, $P = 9$ decimal places
2903	Z	1200	Target starting address for data block
2904	Z	0400	Device selection code data word (Device code 04)

This example instructs the subroutine to read an indefinite number of data words via the peripheral device corresponding to selection code 04, convert them to binary, align them for $P = 9$ decimal places, and store them in consecutive memory cells starting at location 1200 at scale factor @-02. Control then returns to cell 3003 of the main program.

TECHNICAL NOTE

To maintain maximum mathematical accuracy, the smallest practical scale factor value should be chosen. The program does not halt if a number cannot be properly represented or stored at the specified scale factor target; instead, it continues execution using an invalid, meaningless value.

10 Data Output

10.1 Data Output 1

J3-10.0

PURPOSE

Program J3-10.0 converts binary numbers stored in memory into decimal format, including an algebraic sign and decimal point. The scale factor of the binary number can range between `@0` and `@30`. The maximum number of decimal places printed is mathematically bounded by:

$$\frac{30 - \text{scale factor}}{3}$$

A maximum of 10 digits can be output. Leading zeros are automatically suppressed.

The data output routine operates as a subroutine and is invoked via a calling sequence in the main program. Prior to jumping to the subroutine, the binary number to be printed must reside in the accumulator. The required scale factor of the binary number must be stored in the memory cell immediately following the jump instruction. Upon return to the main program, the decimal equivalent of the binary number has already been printed. The main program can be structured to repeatedly invoke Data Output 1 to print an entire list of decimal values sequentially.

MEMORY USAGE

Program storage	4 tracks
Buffer usage	None

INPUT REQUIREMENTS

The input variables required for Data Output 1 are:

1. The binary number N, whose decimal equivalent is to be computed.
2. The value Q, which represents the scale factor position of the binary number.

When jumping to the subroutine, N must reside in the accumulator, and Q must be stored in the cell immediately following the branch instruction.

N may be any arbitrary binary number. Q must be a positive integer and can never be greater than 30. Q can be supplied either as the sector address portion of a `Z` instruction or as a raw hexadecimal constant at bit position 29.

The number of fractional digits (F) printed after the decimal point is determined directly by the value of the scale factor q according to the formula:

$$F = \left\lfloor \frac{30 - q}{3} \right\rfloor$$

(The evaluated value of F is rounded down to the nearest integer).

Binary Point (q)	**Fractional Digits (F)**	**Integer Digits (I)**
30	0*	10
28–29	0	9
25–27	1	8
22–24	2	7
19–21	3	6
16–18	4	5
13–15	5	4
10–12	6	3
7–9	7	2
4–6	8	1
1–3	9	0
0	10	0

**Note: For $q = 30$, either 9 or 10 digits are written depending on the actual magnitude of the number.*

CALLING SEQUENCE

Location	**Opcode**	**Address**	**Description**
α	`R`	`L+4`	Store the address of Q into the subroutine.
$\alpha + 1$	`U`	`L`	Jump to subroutine entry point.
$\alpha + 2$	`Z`	`00qq`	Scale factor Q written as a decimal sector address at @29.
$\alpha + 3$	…		Return point to main program.

In the calling sequence shown above, the instruction at memory location α copies the absolute tracking address ($\alpha + 2$) of the `Z` instruction containing parameter Q directly into the operand address field of the modified instruction located inside subroutine cell `L+4` (where `L` represents the base execution address of the subroutine).

The instruction at $\alpha + 1$ triggers a branch to entry location `L`. Memory cell $\alpha+2$ holds the structural value of Q encoded at @29. Finally, cell $\alpha+3$ contains the first instruction of the main program sequence to resume execution after the output parsing completes.

Important Setup Note: The binary number N must be loaded directly into the accumulator immediately prior to executing this calling sequence.

OUTPUT CHARACTERISTICS

Each time a branch executes from the main program to Data Output 1, the raw binary integer N held inside the accumulator is parsed, converted according to scale parameter Q, and printed as a formatted decimal string.

To guarantee the highest possible printing precision, the programmer should choose the absolute smallest practical scale factor value Q. When properly configured, the transformation error remains bounded within ± 1 unit of the least significant decimal digit.

The computer automatically selects the console typewriter hardware for the output operation. A maximum of 10 characters are printed per number, and all leading zeros are completely suppressed. Every completed value printed is automatically followed by a typewriter `tab` instruction.

OPERATING INSTRUCTIONS

1. **Execution Time:** Output processing runs at approximately 30 formatted numbers per minute via the Flexowriter peripheral system.
2. **Mathematical Accuracy:** The final precision of the printed text remains fundamentally bounded by the initial bit resolution defined by the scale factor Q.
3. **Program Loading Instructions:** The routine tape contains the software in two distinct formats:
    a) A relocatable hexadecimal format compatible with loader program J1-10.0.
    b) A relocatable decimal format utilizing the standard coding sheet layout.
4. **Required Input/Output Units:** No hardware input devices are used during execution. The subroutine automatically selects the standard console typewriter for output processing.
5. **Overflow:** Any arithmetic overflow conditions that occurred prior to entering the subroutine are explicitly ignored upon entry. The CPU's internal overflow register flag is automatically cleared and reset to zero before the routine exits back to the main program.
6. **Error Halts:**

Memory Location	Meaning and Corrective Action
`L + 361`	Q is either negative or greater than 30. After pressing the *Start* key, the subroutine exits immediately back to the main program without performing any data output.

REMARKS

If a peripheral output device other than the standard console typewriter is required, the programmer must manually modify the following output print instructions located within the subroutine body.

Location	Instruction	Location	Instruction
L + 035	XP [ ]	L + 235	80XP [ ]
L + 037	XP [ ]	L + 238	80XP [ ]
L + 054	XP [ ]	L + 241	80XP [ ]
L + 101	XP [ ]	L + 244	80XP [ ]
L + 119	XP [ ]	L + 247	80XP [ ]
L + 136	XP [ ]	L + 250	80XP [ ]
L + 209	XP [ ]	L + 253	80XP [ ]
L + 211	XP [ ]	L + 256	80XP [ ]
		L + 259	80XP [ ]

By default, fractional values are not mathematically rounded during base conversion. However, rounding can be implemented by inserting a specific add instruction (A) at location L + 128. The algebraic effect of this added rounding value depends entirely on the target scale factor at which the number is to be printed.

Alternatively, rounding can be achieved without inserting an additional instruction by directly reducing the constant value stored at memory cell L + 352. Here too, the operational impact depends on the specific scale factor of the number being printed. Integer-only output values are automatically rounded when the scale factor is configured to @28 or @29, and are calculated exactly when the scale factor is precisely @30. By default, fractional values are not mathematically rounded during base conversion. However, rounding can be implemented by inserting a specific add instruction (A) at location L + 128. The algebraic effect of this added rounding value depends entirely on the target scale factor at which the number is to be printed.

Alternatively, rounding can be achieved without inserting an additional instruction by directly reducing the constant value stored at memory cell L + 352. Here too, the operational impact depends on the specific scale factor of the number being printed. Integer-only output values are automatically rounded when the scale factor is configured to @28 or @29, and are calculated exactly when the scale factor is precisely @30.

10.2 Data Output 2

J3-10.1

PURPOSE

Program J3-10.1 prints the contents of consecutive memory cells in decimal notation. Each printed number comprises up to 8 digits plus an algebraic sign and a decimal point. The numbers to be printed must be stored as integers scaled to @30. For positive numbers, all leading zeros and the plus sign are suppressed and replaced by spaces.

The decimal point is only printed if fractional places are explicitly specified. Words sharing the same formatting parameters can be grouped together. Consecutive numbers form a block, and identical word formats form a group within that block. Any number of blocks, and any number of groups within those blocks, are permissible.

Data Output 2 operates as a subroutine invoked by a calling sequence in the main program. The calling sequence must supply:

1. The total number of words to be output within the block.
2. The number of consecutive words in each format group.
3. The total field width for each number.
4. The number of decimal places for each format group.

Upon return to the main program, the specified values have been printed.

MEMORY USAGE

Program storage	5 tracks, 48 sectors (*Note: The program must be loaded to start at sector 00 of an arbitrary track.*)
Buffer usage	None

INPUT REQUIREMENTS

The input variables for Data Output 2 consist of two types of control words: Block Parameters (N) and Group Parameters (N_1). These parameters must be provided by the calling sequence in the following strict order:

- **Block Parameter (N):** This control word must be negative (sign bit = `1`). It contains the total number of consecutive words to be output scaled to @15, and the memory address of the first data word scaled to @29.
- **Group Parameter (N_1):** This control word must be positive. It encodes the following formatting specifications:

- (N_1)**:** The number of consecutive words requiring identical P and D formatting. N_1 is located in the operation field of the instruction word.
- (W_1)**:** The total field width of each printed number or group. W_1 is located in the track address field of the word.
- (D_1)**:** The number of decimal places for each word in this group. D_1 is located in the sector address field of the word.

The numbers to be printed must be stored at @30 and must be strictly less than $\pm 1 \times 10^7$. The structural boundaries for N, N_1, W_1, and D_1 are defined as:

$$0 \leq D_1 \leq 8$$

$128 \geq W_1 \geq (2{+}D_1)$ (ensuring sufficient space for the decimal point and sign)

$$1 \leq (N_1 \text{ and } N) \leq 15$$

$$\sum N_1 = N_t \quad \text{(for each individual block)}$$

For integer-only numbers (no decimal places), no space needs to be budgeted for a decimal point. If the required boundaries for W_1 and D_1 are violated, or if the specified total field width is too narrow to hold the magnitude of the number, the output will be corrupted. If the control values N_1 and N fall between 10 and 15, they can be specified using the hexadecimal characters `F`, `G`, `J`, `K`, `Q`, and `W`.

CALLING SEQUENCES

Two calling sequences are provided. Calling Sequence 1 automatically selects the console typewriter as the output unit. Calling Sequence 2 is used exclusively when an alternative output peripheral is required or when changing the active output device channel.

Calling Sequence 1 (Standard Typewriter Output Block)

Location	Opcode	Address	Description
α	`R`	`L+22`	Store address of first control word into subroutine.
$\alpha + 1$	`U`	`L`	Jump to subroutine entry point.
$\alpha + 2$	`80X`	`ttss`	**Block Parameter** N**:** Negative flag, count at @15, start address at @2
$\alpha + 3$	`[N1]`	`W1 D1`	**Group 1 Format:** N_1 in Opcode, Field Width W_1, Decimals D_1.
$\alpha + 4$	`[N2]`	`W2 D2`	**Group 2 Format:** N_2 in Opcode, Field Width W_2, Decimals D_2.
$\ldots$	…	…	…
$\alpha + n$	`[Ni]`	`Wi Di`	**Group** i **Format specifications.**
$\alpha + n + 1$	…		Begin next block block parameter, or resume main program.

Where `L` is the base starting address of the subroutine. Memory cell $x + n + 1$ contains either the negative block parameter for the subsequent data block or the next executable instruction of the main program.

Calling Sequence 2 (Device Selection Initialization)

Location	Opcode	Address	Description
$\alpha - 1$	`B`	`[Code]`	Load device selection control word into accumulator.
α	`R`	`L+421`	Set return address.
$\alpha + 1$	`U`	`L+532`	Jump to device selection entry point.
$\alpha + 2$	…		Return point to main program (no data printed yet).

The instruction at $x - 1$ loads the specific device selection control word into the accumulator (any instruction achieving this result can be used). This sequence merely primes the subroutine configuration; it executes no data printing operations directly.

OUTPUT CHARACTERISTICS

The program prints the decimal equivalent of each word according to the group formatting specifications. Leading zeros are suppressed. The sign is appended to the end of the printed number: a space for positive numbers and a minus sign for negative numbers. The decimal point is written only if fractional places ($D_1 > 0$) are requested. All planned spaces between numbers are trailing spaces. No tabulator or carriage return commands are issued automatically.

OPERATING INSTRUCTIONS

1. **Execution Time:** In a standard benchmark test utilizing the console typewriter, printing 100 data words with a field width of $W_1 = 9$ required 155 seconds. This total includes overhead for exits, carriage returns, and address modifications performed after every 10 words.

EXAMPLE

Suppose 10 numbers are stored across two memory regions: `3600` to `3605` and `3630` to `3633`. The numbers are to be printed in rows according to the following layout template:

```
1         2         3         4         5         6         7
+XXX.XX   +XXX.XX   XX.XXX++  XX.XXX++  XXXX.XX++ XXXX.XX++ XXXX.XX++
8         9         10
XXXXX+    XXXXX+    XXXXX+...
```

Formatting Breakdown:

- **Word 1:** 2 integer digits, 3 decimal places → (*Note: Typo in original description mismatch; adjusted to align with the Group word values below*).
- **Words 2–4:** 2 integer digits, 3 decimal places, 2 trailing spaces.
- **Words 5–6:** 4 integer digits, 2 decimal places, 2 trailing spaces.

- **Words 7–10:** 5 integer digits, 0 decimal places, 3 trailing spaces.

The subroutine is loaded at base address `1600`. The calling sequence begins at location `2243` and is structured as follows:

Location	Opcode	Address	Instruction Value	Comments
2243	`R`	`1622`		Set Control List link pointer.
2244	`U`	`1600`		Jump to standard execution entry.
2245	`80X`	`06 3600`	`80x6 3600`	**Block 1:** $N = 6$ words; starts at `3600`.
2246	`X`	`07 03`	`x1 0703`	**Group 1:** $N_1 = 1$; $W_1 = 7$ $[(I = 2) + (D = 3) +$
2247	`X`	`09 03`	`x3 0903`	**Group 2:** $N_1 = 3$; $W_1 = 9$ $[(I = 2) + (D = 3) +$
2248	`X`	`10 02`	`x2 1002`	**Group 3:** $N_1 = 2$; $W_1 = 10$ $[(I = 4) + (D = 2) +$
2249	`80X`	`04 3630`	`80x4 3630`	**Block 2:** $N = 4$ words; starts at `3630`.
2250	`X`	`09 00`	`x4 0900`	**Group 1:** $N_1 = 4$; $W_1 = 9$ $[(I = 5) + (S = 3) + 1$

The comments column illustrates the value for N and the derivation of the formatting values. Note that $\sum N_1$ within each block matches its total block count parameter N. Furthermore, when adding up the integer digits (I), decimal places (D), and trailing spaces (S), a constant of `2` must be added to account for the decimal point and sign characters.

In cell `2250`, this formatting constant is only `1` because no decimal point is required for integer-only output.

10.3 Data Output 3

J3-10.2

PURPOSE

Program J3-10.2 prints the contents of consecutive memory cells in decimal notation. Each printed number can consist of up to 7 digits, an algebraic sign, a decimal point, and leading zeros. The numbers to be printed must be stored as integers scaled to @30. Leading zeros and the plus sign are suppressed and replaced by spaces. The decimal point is written only if fractional places are explicitly specified.

Values that are to be printed using identical formatting parameters can be combined into groups. Any number of blocks of consecutive cells, and any number of groups of identical word formats within those blocks, are permissible.

The program functions as a subroutine invoked by a calling sequence in the main program. The calling sequence must supply:

1. The total number of words to be output within the block.
2. The number of consecutive words in each format group.
3. The total field width for each number.
4. The number of decimal places for each value.

Upon return to the main program, the specified values have been printed.

MEMORY USAGE

Program storage	5 tracks, 13 sectors (*Note: Reduces to 5 tracks and 7 sectors if the alternate device-selection entry point is bypassed.*
Buffer usage	None

INPUT REQUIREMENTS

The input variables consist of two types of control words: Block Parameters (N_t) and Group Parameters (N_i). These control words must be provided by the calling sequence in the following strict order:

- **Block Parameter (N_t):** This control word must be negative (sign bit = `1`). It contains the total number of consecutive words to be output scaled to @15, and the memory address of the first data word scaled to @29.
- **Group Parameter (N_i):** This control word must be positive. It encodes the following formatting specifications:
  - (N_i)**:** The number of consecutive words requiring identical P and D formatting. N_i is located in the operation field of the instruction word.

– (W_i)**:** The total field width of each printed number in the group. W_i is located in the track address field of the word.

– (D_i)**:** The number of decimal places for each word in this group. D_i is located in the sector address field of the word.

The numbers to be printed must be stored at @30 and must be strictly less than $\pm 1 \times 10^7$. The structural boundaries for N_t, N_i, W_i, and D_i are defined as:

$$0 \leq D_i \leq 7$$

$128 \geq W_i \geq (2+D_i)$ (ensuring sufficient space for the decimal point and sign)

$$1 \leq (N_i \text{ and } N_t) \leq 15$$

$$\sum N_i = N_t \quad \text{(for each individual block)}$$

CALLING SEQUENCE

Location	**Opcode**	**Address**	**Description**
α	`R`	`L+22`	Set the address of the first control word into the subroutine.
$\alpha+1$	`U`	`L`	Entry into the subroutine (`L+7` is equivalent as an entry).
$\alpha+2$	`80X`(N_t)	`ttss`	N at @15 and starting address of the data at @29.
$\alpha+3$	`[`N_1`]`	W_1D_1	Format specifications
$\alpha+4$	`[`N_2`]`	W_2D_2	Format specifications
$\cdots$	…	…	
$\alpha+n$	N_i	W_iD_i	
$\alpha+n+1$	…		Beginning of the next block or return to the main program (return instruction must be positive).

`L` is the starting address of the subroutine. Cell $\alpha+n+1$ contains either the N control word for the next block or the next instruction of the main program.

Location	**Opcode**	**Address**	**Description**
$\alpha-1$	`B`	`[ ]`	Bring selection code
α	`R`	`L+512`	Set return address
$\alpha+1$	`U`	`L+220`	Entry into the subroutine
$\alpha+2$	…		Main program

The instruction in $\alpha-1$ brings the selection code for the output unit into the accumulator (any instruction that does this is permissible). `L` is the starting address of the subroutine. This calling sequence only prepares the subroutine; no output results from it.

OUTPUT CHARACTERISTICS

The program prints the decimal equivalent of each word according to the group formatting specifications. Leading zeros are suppressed. The sign is appended to the end of the printed number: a space for positive numbers and a minus sign for negative numbers. The decimal point is written only if fractional places ($D_1 > 0$) are requested. All planned spaces between numbers are trailing spaces. No tabulator or carriage return commands are issued automatically.

OPERATING INSTRUCTIONS

1. **Execution Time:** In a standard benchmark test utilizing the console typewriter, printing 100 data words with a field width of $W_1 = 9$ required 155 seconds. This total includes overhead for exits, carriage returns, and address modifications performed after every 10 words.

EXAMPLE

Suppose 10 numbers are stored across two memory regions: `3600` to `3605` and `3630` to `3633`. The numbers are to be printed in rows according to the following layout template:

```
1      2          3          4         5          6         7
XXX.XX+XXX.XX+    XX.XXX+    XX.XXX+   XXXX.XX+   XXXX.XX+  XXXX.XX+...
8          9          10
XXXXX+     XXXXX+     XXXXX+...
```

Formatting Breakdown:

- **Word 1:** 2 integer digits, 3 decimal places → (*Note: Adjusted to map the Group parameters below*).
- **Words 2–4:** 2 integer digits, 3 decimal places, 2 trailing spaces.
- **Words 5–6:** 4 integer digits, 2 decimal places, 2 trailing spaces.
- **Words 7–10:** 5 integer digits, 0 decimal places, 3 trailing spaces.

The subroutine is loaded at base address `1600`. The calling sequence begins at location `2243` and is structured as follows:

Location	Opcode	Address	Instruction Value	Technical Notes
2243	`R`	`1622`		Set Control List link pointer.
2244	`U`	`1600`		Jump to standard execution entry.
2245	`80X`	`06 3600`	`80x6 3600`	**Block 1:** $N = 6$ words; starts at `3600`.
2246	`X`	`07 03`	`x1 0703`	**Group 1:** $N_1 = 1$; $W_1 = 7$ $[(I = 2) + (D = 3) +$
2247	`X`	`09 03`	`x3 0903`	**Group 2:** $N_1 = 3$; $W_1 = 9$ $[(I = 2) + (D = 3) +$
2248	`X`	`10 02`	`x2 1002`	**Group 3:** $N_1 = 2$; $W_1 = 10$ $[(I = 4) + (D = 2) +$
2249	`80X`	`04 3630`	`80x4 3630`	**Block 2:** $N = 4$ words; starts at `3630`.
2250	`X`	`09 00`	`x4 0900`	**Group 1:** $N_1 = 4$; $W_1 = 9$ $[(I = 5) + (S = 3) +$

The notes column illustrates the value for N and the derivation of the formatting values. Note that $\sum N_1$ within each block matches its total block count parameter N. Furthermore, when adding up the integer digits (I), decimal places (D), and trailing spaces (S), a constant of `2` must be added to account for the decimal point and sign characters.

In cell `2250`, this formatting constant is only `1` because no decimal point is required for integer-only output.

TECHNICAL REMARK

This program works identically and satisfactorily with both the Mod 18 and Mod 9 print mechanism variants of the console typewriter.

10.4 Data Output 4

J3-10.3

PURPOSE

The program prints the contents of the accumulator. Up to nine decimal places plus an algebraic sign, a decimal point, and a tabulator command are output. Leading zeros are suppressed, and the output is printed correctly. The data to be output may be positioned at any one of 10 predetermined q scale factors.

The program operates as a subroutine invoked by a calling sequence in the main program. The calling sequence must supply:

1. The number of digits C that can stand before the decimal point.
2. The number of digits D to be printed after the decimal point (q is determined by C).

MEMORY USAGE

Program storage	3 tracks
Buffer usage	None

INPUT REQUIREMENTS

The input consists of the data word to be printed and the control keyword for C and D.

CALLING SEQUENCE

Location	Opcode	Address	Description
$x-1$	`B`	`[ ]`	(Any instruction sequence that supplies data). Loads data word into the accumulator.
x	`R`	`Lo+12`	Set return link pointer.
$x+1$	`U`	`Lo`	Entry into the subroutine.
$x+2$	`Z`	`0D0C`	D encoded at @23 and C encoded at @29.
$x+3$	…		Main program.

KEYWORD SPECIFICATIONS

D may be any integer between 0 and 9, provided that $C + D \leq 9$.

C may be any integer between 0 and 9, provided that $C + D \leq 9$. The scale factor q of the value to be printed is supplied by C, as shown in the following table:

C	q	C	q
0	0	5	17
1	4	6	20
2	7	7	24
3	10	8	27
4	14	9	30

OUTPUT CHARACTERISTICS

The value in the accumulator is printed with C digits before the decimal point (leading zeros are suppressed) and D digits after the decimal point (properly rounded up), followed by a minus sign or a space, and a tabulator jump.

OPERATING INSTRUCTIONS

1. **Execution Time:** Variable, depending on C and D. Output speed is approx. 6 characters/second.
2. **Accuracy:** If nine digits are printed, an error of 1 unit may be present in the ninth decimal place.
3. **Program Loading Instructions:** The tape contains the program in two versions:
   a) Relocatable hexadecimal format compatible with Program Input 1.
   b) Decimal format utilizing the coding sheet layout.
4. **Required Input/Output Units:** Output is sent via the Model 121 typewriter hardware. Input units are not required.

ERROR HALTS

Memory Location	Meaning and Corrective Action
`Lo + 120`	$C + D$ is greater than 9. After pressing *Start*, `"fld"` is printed and execution returns to the main program.
`Lo + 120`	The number to be output cannot be printed within C integer digits. After pressing *Start*, `"big"` is printed and execution returns to the main program.

EXAMPLE

A number with the formatting layout template `XXXXX.XX+` is to be printed.

Location	Opcode	Address	Technical Notes
1345	`B`	`[ ]`	Target data word scaled to @17.
1346	`R`	`2312`	Link pointer mapped to `Lo + 12`.
1347	`U`	`2300`	Jump to entry point `Lo`.
1348	`XZ`	`0205`	Keyword control word ($D = 2$, $C = 5$).
1349	…		Main program resumes.

This example assumes that Data Output 4 is located at base address `2300`. The calling sequence begins at memory location `1345`.

10.5 Data Output 5

J3-10.4

PURPOSE

Program J3-10.4 outputs a nine-digit decimal number plus an algebraic sign and a decimal point. For a negative number, a minus sign is printed; a space appears in place of a plus sign.

The program functions as a subroutine invoked by a calling sequence in the main program. The calling sequence must supply a keyword in cell $x + 2$ that specifies the number of digits in the integer portion of the value.

MEMORY USAGE

Program storage	120 sectors
Buffer usage	None

INPUT REQUIREMENTS

The input consists of the number to be printed and the identification digit C, which specifies the number of digits in the integer portion.

CALLING SEQUENCE

Location	Opcode	Address	Description
$x - 1$	`B`	`L(N)`	Load data word into the accumulator.
x	`R`	`Lo+12`	Set address of the keyword into the subroutine.
$x + 1$	`U`	`Lo`	Entry into the subroutine.
$x + 2$	`Z`	`000C`	Identification code C, which corresponds to q.
$x + 3$	…		Main program.

Memory cell $x - 1$ does not strictly require a load (`B`) instruction. Any instruction that leaves the argument inside the accumulator is permissible. C is an identification digit bounded between 0 and 9.

OUTPUT CHARACTERISTICS

The output consists of: nine decimal digits, an algebraic sign (or a space if positive), and the decimal point. Leading zeros are replaced by spaces.

KEYWORD SPECIFICATIONS

C	q	**Output Format**
0	0	`.XXXXXXXXX`
1	4	`X.XXXXXXXX`
2	7	`XX.XXXXXXX`
3	10	`XXX.XXXXXX`
4	14	`XXXX.XXXXX`
5	17	`XXXXX.XXXX`
6	20	`XXXXXX.XXX`
7	24	`XXXXXXX.XX`
8	27	`XXXXXXXX.X`
9	30	`XXXXXXXXX.`

Note: Every binary number is scaled and converted as a fraction. The identification digit is used by the subroutine as a column placement counter. Every completed number print is followed by a tabulator command. Execution then returns to location $x + 3$ of the main program.

OPERATING INSTRUCTIONS

1. **Execution Time:** 3.0 seconds per number.
2. **Accuracy:** The maximum error is bounded within 1 unit at the ninth decimal digit position.
3. **Program Loading Instructions:** The program is available in two versions:
   a) Relocatable hexadecimal format, readable via Program Input 1.
   b) Decimal format utilizing the standard coding sheet layout.
4. **Required Input/Output Units:** The program selects the paper tape typewriter channel for output. Hardware input units are not required.

PRINT INSTRUCTIONS TO BE PATCHED

Location	**Instruction**
`Lo + 0048`	`XPC 0200`
`Lo + 0053`	`XPC 0200`
`Lo + 0111`	`XPC 0200`
`Lo + 0117`	`XPC 0200`
`Lo + 0132`	`80XPC 0200`

ERROR HALTS

Memory Location	Meaning and Corrective Action
`Lo + 46`	The argument is $\geq 10^9$. The computer halts. After pressing *Start*, the routine exits without printing.

10.6 Data Output 6

J3-10.5

PURPOSE

Program J3-10.5 prints one or more groups of numbers in decimal notation, each of which can contain one or more individual values. The numbers within a group must be stored in consecutive cells scaled to the same scale factor q. Leading zeros are replaced by spaces; a space or a minus sign follows immediately behind the last decimal place.

MEMORY USAGE

Program storage	224 cells (3 1/2 tracks)
Buffer usage	Sectors 42 and 43 of Track 63 are utilized as scratchpads.

CALLING SEQUENCE

Location	Opcode	Address	Description
x	`R`	`Lo+3`	Set return link pointer.
$x+1$	`U`	`Lo`	Entry into subroutine.
$x+2$	`Z`	`Loc1`	Destination address of the first data word in Group 1.
$x+3$	`Z`	`N1 q1`	Total words N_1 and scale factor q_1 for Group 1.
$x+4$	`Z`	`Loc2`	Destination address of the first data word in Group 2.
$x+5$	`Z`	`N2 q2`	Total words N_2 and scale factor q_2 for Group 2.
...	…	…	…
$x+2i$	`Z`	`Loci`	Destination address of the first data word in Group i.
$x+2i+1$	`Z`	`Ni qi`	Total words N_i and scale factor q_i for Group i.
$x+2i+2$	…		Exit marker (This instruction field must NOT be a `Z` opcode).

Where `Loc`$_i$ represents the starting memory address of the first data value in the i-th group; N represents the absolute quantity of data values in the i-th group; q is the binary scale factor of all data points within the i-th group. The values N and q each occupy 2 decimal digits, corresponding directly to the track and sector layout fields respectively.

OUTPUT CHARACTERISTICS

Each output value consists of 8 to 10 decimal digits, a decimal point, a space or minus sign, and a tabulator command. Two distinct `Z` instructions are required to cleanly define each data group block. After printing the final data group, execution exits to the instruction immediately following the calling sequence array; this exit instruction must not use a `Z` opcode.

OUTPUT FORMAT LAYOUTS

Scale Factor Range (q)	Output Format Template
0	`.XXXXXXXX+`
$0-4$	`X.XXXXXXX+`
$4-7$	`XX.XXXXXX+`
$7-10$	`XXX.XXXXX+`
$10-14$	`XXXX.XXXX+`
$14-17$	`XXXXX.XXX+`
$17-20$	`XXXXXX.XX+`
$20-24$	`XXXXXXX.X+`
$24-27$	`XXXXXXXX+`
$27-30$	`XXXXXXXXX.+`
$30-31$	`XXXXXXXXXX.+`

EXAMPLE

Suppose 10 values are stored sequentially. The calling sequence starts at memory address x and points to locations `2100` and `2110`:

Location	Opcode	Address	Instruction Value	Technical Notes
x	`R`	`Lo+3`		Set return link pointer.
$x+1$	`U`	`Lo`		Jump to entry point `Lo`.
$x+2$	`Z`	`2100`	`xZ 2100`	$\texttt{Loc}_1 = \texttt{2100}$
$x+3$	`Z`	`0407`	`xZ 0407`	$N_1 = 4$ words; scale factor $q_1 = 7$.
$x+4$	`Z`	`2110`	`xZ 2110`	$\texttt{Loc}_2 = \texttt{2110}$
$x+5$	`Z`	`1115`	`xZ 1115`	$N_2 = 11$ words; scale factor $q_2 = 15$.
$x+6$	`B`	`[ ]`	`xB [ ]`	Main program exit (Non-`Z` instruction).

The calling sequence above executes the following parsing stages:

1. Prints the content of locations `2100`, `2101`, `2102`, and `2103` using the template `XX.XXXXXX+` for values whose absolute magnitude is strictly less than 100.00000, or using the template `XXX.XXXXX+` for values scaling above this boundary (see $q = 7$ range rules inside the format guide).

2. Prints the content of locations `2110` through `2120` using the layout template `XXXXX.XXX+`.

3. Branches out to position $x + 6$, where the non-`Z` operational code safely terminates the subroutine parser sequence loop.

FEHLERSTOPS

Memory Location	Meaning and Corrective Action
`Lo + 207`	$N \leq 1$. After pressing *Start*, execution exits back to the main program loop without executing any prints.

TECHNICAL SPECS

1. **Accuracy:** Output calculations carry a potential maximum mathematical rounding error of 1 unit at the eighth printed decimal place position.

2. **Execution Time:** Processing throughput clocks at approximately 25 words/minute.

3. **Program Loading Instructions:** The tape contains the routines in two standard alternative distributions: relocatable hexadecimal format, and decimal formatting using the standard coding sheet structures. Both tapes are natively read using Program Input 1.

11 Hexadecimal Output

The programs in this section are mostly utilities for taking existing data and reformatting it for use with one of the program loaders.

- J4-10.1 is the equivalent of a "delinker". It takes a block of memory, subtracts a relocation constant from a set of memory blocks defined by its input, and punches that out to tape. This allows programs which had been loaded and relocated with PIR or GPMod to be relocated back to zero-based addressing for loading again at a different address later.
- J4-10.2 is a straight dump-memory-to-tape backup program. Given a starting track, and an ending track, the program dumps the memory in 64-word blocks with a checksum.
- J4-10.3 is a slightly more refined dump program, taking a starting track and sector and an ending track and sector, The output has a carriage return inserted every 8 words; this makes the output a useful, readable hex dump as well as a way to dump that memory to tape.

[There is no Hexadecimal Output 1 (which would have been J4-10.0) program in this document.]

11.1 Hexadecimal Output 2

J4-10.1

PURPOSE

Program J4-10.1 punches the contents of a specified memory region onto paper tape in a relocatable hexadecimal format. For every block of 64 words (or fewer), a hexadecimal loader control word (Füllschlüsselwort) and a checksum are punched along with the data. The contents are output in ascending order of memory addresses.

The program tape generated by this routine can be read back into the computer using Program Input 1 (J1-10.0).

MEMORY USAGE

Program storage	6 tracks
Buffer usage	Storage table for the hexadecimal regions (see "Input Requirements").

INPUT REQUIREMENTS

J4-10.1 subtracts a specified modifier value from all instruction words (with the exception of instructions containing an `I` or `P` operation code). Therefore, all instructions (or hexadecimal words configured to look like instructions) that must not be modified during relocation must be declared inside the hexadecimal

regions table. The memory area allocated for this table, along with its specific entries, is supplied to the computer following the print prompts `"tble"` and `"hex"`.

Upon branching to the program's starting location, the computer prompts the operator with the following query-and-response routine:

`Tble` Enter the boundaries for the hexadecimal region storage table using decimal track/sector notation. If no boundaries are input, the computer automatically defaults to Track 63. When specifying a custom area, the start and end addresses must be entered together as a single word (e.g., `0300 0463'` allocates Tracks 03 and 04 for the table). The table accommodates up to $(N-1)$ entries, where N represents the absolute number of memory cells allocated to the table region. For instance, if two tracks are assigned, up to 63 entry lines can be stored.

`Hex` This prompt functions as a multi-line sequential input entry; the entire table of hexadecimal regions is managed as a single logical entity. Enter the boundaries—using decimal track/sector notation—for all memory zones that are to be punched using absolute addresses, regardless of how the words appear. The starting and ending address of each contiguous block of memory cells must be entered as a single word (e.g., `0300 0301'`, `0402 0432'`, `0533 0533'`, etc.).

Any word containing a `P` or `I` opcode is automatically output with an absolute address. Similarly, any word that contains a `1` bit set outside the standard sign, opcode, or address fields is automatically treated as absolute. These words do not need to be manually declared via the `"Hex"` prompt.

`Pnch.` Enter the boundaries of the memory region whose contents are to be output, using decimal track/sector notation. Provide the start and end address inside a single input word (e.g., `0300 0663'`).

`Modi` Enter the modifier value—in decimal track/sector notation—that must be subtracted from each instruction word so that it gets converted into a relative address format (e.g., `0300'`).

The memory cells selected for output must reside in a continuous, ascending sequence. Any arbitrary quantity of cells may be specified.

CALLING SEQUENCE

The program is executed by branching directly to its designated starting address.

OUTPUT CHARACTERISTICS

Program J4-10.1 punches the contents of the specified memory range onto tape in relocatable hexadecimal format. The data is structured into discrete blocks

of 64 words each (the final block may contain fewer). Each block is preceded by a hexadecimal loader control word and followed by a calculated verification checksum. A carriage return is automatically punched after a maximum of 10 words. Blank feed spaces are punched between individual data blocks.

Output processing proceeds sequentially through ascending memory addresses. The actual contents of the source memory cells remain unaltered by the program. Once the requested data stream is completely punched, the computer executes a halt instruction. Pressing the *Start* key triggers a branch back to the beginning of the program loop.

OPERATING INSTRUCTIONS

1. **Program Loading Instructions:** The routine tape contains the software in two alternate distributions:

   a) A relocatable hexadecimal format compatible with Program Input 1.

   b) A relocatable decimal format using the standard coding sheet layout.

2. **Required Input/Output Units:** The standard console typewriter is automatically selected by the program logic to handle all parameter entry prompts and data tape punching operations.

11.2 Hexadecimal Output 3

J4-10.2

PURPOSE

Program J4-10.2 punches the contents of a specified memory region onto paper tape in the format required by Program Input 3 (J5-10.0). The requested information is punched in hexadecimal notation in structural blocks of 64 words. Blank feed spaces are punched between individual data words within a block. Each block comprises an entire, unbroken track. Every block is accompanied by a hexadecimal loader control word (Eingabeschlüsselwort) and a verification checksum. Any arbitrary number of tracks can be punched in a single execution pass, provided they reside in ascending memory sequence order.

MEMORY USAGE

Program storage	4 tracks; or 4 tracks and 17 sectors, depending on the selected output unit configuration (see "Program Loading Instructions").
Buffer usage	None

INPUT REQUIREMENTS

Upon executing a branch to the program's starting address, the computer selects the console typewriter for input parameters and requests the following boundaries:

1. **L1:** The address of the first track to be punched, entered as a 2-digit track number.
2. **L:** The address of the last track to be punched, entered as a 2-digit track number.

When the computer halts to request these output boundaries, and before the operator enters the final track address (**L**), any number of negative words may be input to be punched directly onto the tape. These words are interpreted by Hexadecimal Input 3 as absolute execution jump instructions and must strictly conform to the specifications of that input program.

CALLING SEQUENCE

The program is invoked by branching directly to its designated starting address. Program distributions featuring an integrated bootstrap loader terminate with an automatic branch to the starting address; other distributions end with a halt-and-transfer sequence (see "Program Loading Instructions").

OUTPUT CHARACTERISTICS

The program punches the contents of consecutive tracks onto paper tape in hexadecimal notation. Processing advances sequentially from the specified starting track address to the ending track address. Only complete tracks are output; partial track dumping is not supported.

The hexadecimal words are arranged on the tape in 64-word blocks. Each block covers a distinct track, and the words within a block follow a specific layout pattern separated by space characters. The physical sector tracking sequence within each output block is ordered exactly as follows:

```
56,  58,  60,  63,  01,  03,  05,  62,  00,  02,  04,  06,  08,  10,  12,  07,
09,  11,  13,  15,  17,  19,  14,  16,  18,  20,  22,  24,  26,  21,  23,  25,
27,  29,  31,  33,  28,  30,  32,  34,  36,  38,  40,  35,  37,  39,  41,  43,
45,  47,  42,  44,  46,  48,  50,  52,  54,  49,  51,  53,  55,  57,  59,  61.
```

Each block is preceded by a hexadecimal loader control word and followed by a calculated verification checksum. All negative jump control keywords are punched to tape immediately upon manual entry. Following the completion of the final track (**L**), the program automatically execution-branches back to the starting address so that new output boundaries may be entered.

OPERATING INSTRUCTIONS

1. **Execution Time:** Approximately 26 seconds are required per track when outputting via the Model 151 high-speed paper tape punch peripheral.

2. **Program Loading Instructions:** The program tape contains the software in four alternate distributions:

   a) Relocatable hexadecimal format with a Track 00 bootstrap loader.

   b) Relocatable hexadecimal format with a Track 63 bootstrap loader.

   c) Relocatable hexadecimal format compatible with Program Input 1 (J1-10.0).

   d) Relocatable decimal format using the standard coding sheet structures.

   The exact entry procedure varies depending on the selected distribution and the active input peripheral:

   Relocatable Hexadecimal Format with Bootstrap Relocatable hexadecimal programs paired with an independent bootstrap loader allow the operator to enter the routine via either the Model 121 console typewriter or the Model 141 high-speed tape reader, as a dedicated bootstrap block is provided for each device. They are structured sequentially on the tape in the following order:

   (a) Track 00 bootstrap for program input via the Model 141 tape reader.

(b) Track 00 bootstrap for program input via the Model 121 console typewriter.

(c) Program J4-10.2 software body.

(d) Track 63 bootstrap for program input via the Model 141 tape reader.

(e) Track 63 bootstrap for program input via the Model 121 console typewriter.

(f) Program J4-10.2 software body.

The operator must select the specific bootstrap block that matches both the target destination track and the desired input hardware, then feed it according to the standard bootstrap entry procedure described in the *Program Input 1* documentation.

The first instruction word loaded by the bootstrap is the modifier, which defines the absolute starting address where the program is to be stored. The tape features the following predefined (and manually alterable) modifier flags:

- Track 00 bootstraps utilize a modifier value of `0000`.
- Track 63 bootstraps utilize a modifier value of `6000`.
- After the bootstrap block finishes reading, 4 program tracks are loaded automatically, and the computer executes a halt.

If the program is intended to run utilizing the Model 121 typewriter as the active output device, a manual jump to the program starting address must be executed at this stop point (see "Manual Branching Procedure").

If the operator instead intends to route output to the Model 151 high-speed tape punch, they should press the *START* switch instead of executing the manual jump when the machine halts. The computer will then ingest and store 14 additional instructions that overwrite the default print commands, reconfiguring the program to target the Model 151 high-speed punch channel. Following the ingestion of these 14 patching instructions, the computer halts. To execute a jump to the program's starting address, the operator only needs to press the *START* switch.

Relocatable Hexadecimal Format (Program Input 1 Variant) This distribution is loaded using the standard loading routine detailed in the *Program Input 1* reference guide. The loader ingests the 4 tracks of the program body and then halts.

If the console typewriter is to be used as the output device, set the control lever to *Manual Input* (Eingabe von Hand) and press the *START* switch. The typewriter is now selected for entry. Manually enter a stop-and-transfer command pointing to the starting address of the program, set the control toggle to *START COMP*, and press the *START* switch.

If the operator wishes to preserve arbitrary peripheral device assignment options at runtime, execute the following sequence precisely when the initial loading phase halt occurs:

(a) Set the control lever to *MANUAL INPUT* (Eingabe von Hand).

(b) Press the *START* switch. (The console typewriter is selected for input parameters).

(c) Type the standard Program Input 1 device assignment keyword matching the physical peripheral device through which the tape is being fed (e.g., `S000 0000` for the Model 141 high-speed reader).

(d) Set the control toggle to *START COMP*.

(e) Release the *MANUAL INPUT* control lever.

(f) Press the *START* switch.

The remainder of the tape profile (17 patching instructions) is ingested, and the computer executes a halt. (An automatic jump-and-transfer instruction pointing to the starting address of the dumper program is located at the tail end of this block. Pressing the *START* switch at this point begins program execution, defaulting the typewriter channel for input and output).

To override this and explicitly lock down a custom output unit channel, execute a manual execution branch to address `L + 0400`, where `L` represents the base starting address of the active input loader program. The routine selects the typewriter for input. Enter the device selection keyword for the desired output hardware peripheral as a standard 2-digit track address token. The routine dynamically overwrites the internal print commands to target the selected device channel and begins runtime operations immediately (i.e., requesting the start and end track parameters) without executing an intermediate halt.

3. **MANUAL BRANCHING PROCEDURE (Das "Springen von Hand")**

a) Set the master mode selection dial to *MANUAL INPUT* (Eingabe von Hand).

b) Type an unconditional execution transfer instruction (`U`) targeting the desired memory cell address using hexadecimal notation (e.g., to target memory location `6000`, type `000 U3J00`).

c) Press the *FILL/CLEAR* (Füllen/Löschen) button.

d) Set the master mode selection dial to *SINGLE STEP* (Einzeloperation).

e) Press the *EXECUTE* (Ausführen) button.

f) Return the master mode selection dial to *NORMAL*.

g) Press the *START* switch.

4. **REQUIRED INPUT/OUTPUT UNITS** The standard console typewriter hardware is automatically designated to handle all manual boundary input parameters. The destination output unit is assigned by the loader block processing sequence based entirely on the specific configuration selections executed by the operator at load time.

11.3 Hexadecimal Output 4

J4-10.3

PURPOSE

Program J4-10.3 punches blocks of memory in hexadecimal format. The resulting paper tapes can be read back into the computer using Program Input 1 or 2 (J1-10.0 or J1-10.1). The active output peripheral unit can be designated by the user at runtime as desired.

MEMORY USAGE

Program storage	3 tracks
Buffer usage	None

INPUT REQUIREMENTS

The input parameters consist of the decimal device selection code, the starting and ending addresses of the memory block to be punched, and an execution address for an absolute "Halt and Jump" control word (`.000XXXX`). All of these parameters must be entered using decimal track/sector notation.

OUTPUT CHARACTERISTICS

The output stream is structured into blocks of up to 64 words in hexadecimal format. Each block is preceded by a loader control word (`VNNCHHHH`; see reference documentation for Program Input 1 or 2), and each block is followed by a calculated verification checksum that incorporates both the control keyword and every individual data word contained within the punched block.

A carriage return command is automatically punched after every eight words. To optimize tape space, data words that are completely equal to zero have all eight hexadecimal zero characters entirely suppressed. Words containing three leading zeros (as seen in the vast majority of standard instruction words) have those three leading zeros suppressed. All other data words are punched as a full sequence of eight hexadecimal characters.

OPERATING TIMES

Console Typewriter Tape Punch	Approx. 68 seconds / track
High-Speed Tape Punch	Approx. 34 seconds / track

OPERATING INSTRUCTIONS

1. **Program Loading Instructions:** The tape contains the program in two alternate distributions:

   a) Relocatable hexadecimal format compatible with Program Input 1 (J1-10.0).

   b) Relocatable decimal format using the standard coding sheet layout.

2. **Program Execution Steps:**

   (a) Execute a branch to the starting address of the program.

   (b) Type the decimal device selection code: `02` for the console typewriter tape punch or `06` for the high-speed paper tape punch peripheral.

   (c) Type the starting address of the memory block to be punched using decimal track/sector notation.

   (d) Type the ending address of the memory block to be punched using decimal track/sector notation. (*Note: The ending address must be greater than or equal to the starting address*).

   (e) Type the target execution address that is to be encoded into the final "Halt and Jump" control word. This causes the block sequence terminator `.000TTSS` to be punched at the tail end of the operation. If no "Halt and Jump" control keyword is desired after the block, the operator can simply press the typewriter's *START COMP* (Rechnen Start) lever without entering prior characters. (If an absolute stop code of `.0000000` is explicitly desired, at least one zero character must be typed before pressing the *START COMP* lever).

   (f) Once the punching routine completes, the program loops back to step 3 and waits for a new starting address input.

3. **Required Input/Output Units:** The program automatically selects the console typewriter hardware to handle all manual entry parameters. The active output peripheral destination is determined dynamically by the decimal selection code entered at runtime (`02` for the typewriter punch; `06` for the high-speed punch).

TECHNICAL REMARK

The program automatically clears the hardware overflow flag. The overflow status register is guaranteed to be reset following the completion of each punched block sequence.

12 Alphanumeric Output

12.1 Alphanumeric Output 1

J4-11.0

PURPOSE

Program J4-11.0 translates keywords and handles their output as alphabetic and/or numeric text. The alphanumeric text must be stored by the main program in a 4-bit character encoding format. It consists of a group of words composed of special codes corresponding to the characters and functions of the typewriter. The amount of data to be stored for J4-11.0 is limited only by the available memory space. The information must be stored in ascending sequence in consecutive memory cells.

J4-11.0 operates as a subroutine. It is called by a calling sequence in the main program. Before jumping to the subroutine, the starting address of the data block to be output must be stored within the subroutine. The memory locations following the jump instruction must contain the control keywords. Upon return to the main program, the output operation has already been executed.

MEMORY USAGE

Program storage	56 sectors
Buffer usage	None

INPUT REQUIREMENTS

The starting address of the keyword list must be stored before jumping to the subroutine. The data block must consist of consecutive words located immediately following the jump instruction of the calling sequence. Each word consists of four keyword symbols (encoded as pairs of characters), which correspond to alphabetic characters as well as the control functions of the typewriter.[44]

The last word of the block is designated by the special termination symbol `7Q`. Since every word must contain four keyword symbols (representing eight total hexadecimal digits per word), two to six trailing zeros are required to fill out the final word. The number of words in a block is limited only by available memory space. In the main program, every block of 63 or fewer keywords must be preceded by a hexadecimal loader control word, as required by Program Input 1.

[44]This subroutine essentially encodes the Flexowriter's 6-bit characters as 8-bit ones, so that each character takes up two hex digits. Instead of having to bit-slice characters in one's head and attempt to pack them 5 to a word, this subroutine lets you pack them 4 to a word on hex-digit boundaries and not lose your marbles.

ALPHANUMERIC KEYWORD SYMBOL TABLE

[45]

Keycap	Symbol	Keycap	Symbol
)0	04	Bb	0F
L1	0J	Cc	6F
*2	14	Dd	2F
"3	1J	Ee	4F
Δ4	24	Ff	54
%5	2J	Gg	5J
$6	34	Hh	62
π7	3J	Ii	22
Σ8	44	Jj	64
(9	4J	Kk	6J
Space	06	Ll	0J
_-	0A	Mm	3F
=+	16	Nn	32
:;	1A	Oo	46
?/	26	Pp	42
].	2A	Qq	74
[,	36	Rr	1F
Tab	30	Ss	7F
Lower	08	Tt	5F
Upper	10	Uu	52
Color	18	Vv	3A
CR	20	Ww	7J
Backspace	28	Xx	4A
'	40	Yy	12
Terminator	7Q	Zz	02
Aa	72		

[Translator's note: Readers should note that the table includes "A" as a hex code, which in LGP-21 parlance is nonsense; the hex digits are F G J K Q W. I have no documentation that specifically explains why this is "A" here. Best guess is that this was touch-typed on an AZERTY keyboard by someone used to a QWERTY keyboard, which would swap A for Q.]

EXAMPLE

The following example illustrates how to compile a data group for "Alphanumeric Output 1":

[45]This table is transcribed accurately from the original typed documnentation; I have preserved the errors noted in the translator's note. If it turns out we can test this routine at a later date, we can verify if those values are Q's or not and correct this table.

Loader Word	Location	Opcode	Address	Description
	x	R	L	Set data pointer.
	$x+1$	U	L	Branch to subroutine.
.0000004	$x+2$	20	42 09 72	Alphanumeric raw hex data tokens.
	$x+3$	32	32 22 32	Alphanumeric raw hex data tokens.
	$x+4$	5J	06 42 72	Alphanumeric raw hex data tokens.
	$x+5$	12	7F 7Q 00	Alphanumeric raw hex data tokens with end marker.
	$x+6$	…		Main program resumes.

This example issues an automatic carriage return command, prints the text "PLANNING PAYS", and cleanly branches execution back to location $x+6$ of the main program logic sequence. The token 7Q inside the final memory word marks the structural end code statement; the remaining trailing zero characters are solely included to fill out the word boundary layout.

CALLING SEQUENCES

There are two distinct calling sequences available for invoking "Alphanumeric Output 1". Calling Sequence 1 is utilized exclusively when routing output to the standard console typewriter. Calling Sequence 2 is used when an alternative hardware output peripheral unit is required.

Calling Sequence 1 (Standard Typewriter Output)

Location	Opcode	Address	Description
x	R	L	Transfer starting address of data block into subroutine cell L.
$x+1$	U	L	Jump to subroutine standard execution entry point.
$x+2$	…		(First control keyword string token word).
…	…		Alphanumeric string control words.
$x+n$	…		(Final control keyword string token word containing 7Q).
$x+n+1$	…		Main program execution resumes.

The instruction at location x stores the entry pointer of the data block into the subroutine body. The command at $x+1$ triggers the execution branch. Cells $x+2$ through $x+n$ contain the n string keyword words. Location $x+n+1$ holds the next instruction of the main program to be executed immediately following subroutine exit.

Calling Sequence 2 (Custom Peripheral Routing)

Location	Opcode	Address	Description
$x - 1$	`B`	`[Code]`	Load custom device selection control word into accumulator.
x	`R`	`L`	Store starting address of keywords into subroutine body.
$x + 1$	`U`	`L+50`	Jump to alternative device-selection entry point.
$x + 2$	…		(First control keyword string token word).
$\dots$	…		Alphanumeric string control words.
$x + n$	…		(Final control keyword string token word containing `7Q`).
$x + n + 1$	…		Main program execution resumes.

This configuration differs from the standard entry pattern by exactly two operational instructions. The command at $x - 1$ stages the peripheral selection mask inside the accumulator register. This selection code can be encapsulated within the track address operand field of a standard non-operative instruction word (e.g., `xZ tt00`, where `tt` represents the physical target peripheral selector code). The command at $x + 1$ branches into the alternative subroutine device configuration entry sequence at offset address `L+50`.

OUTPUT CHARACTERISTICS

Upon execution return to the main program context, the alphanumeric string array text generation sequence has been completely processed and sent to the printer hardware buffer.

OPERATING INSTRUCTIONS

1. **Execution Speed:** Throughput clocks at approximately 660 characters per minute.

2. **Program Loading Instructions:** The routine tape distribution features the software in two alternate styles:

   a) Relocatable hexadecimal format compatible with Program Input 1.

   b) Relocatable decimal format mapped to the standard coding sheet structure.

3. **Required Input/Output Units:** Hardware data input units are not required. If an alternative hardware peripheral unit other than the primary console typewriter is designated for text routing, the absolute track operational address entries inside memory cells `L+06`, `L+14`, `L+22`, and `L+30` of the subroutine body must be manually patched to match the system configuration constants (consult the *LGP-21 Programming Manual* peripheral channel assignment tables for absolute unit selector byte lists).

4. **Hardware Overflow Flag Status:** The register status is neither read nor impacted by this program's internal loop execution paths.

ALPHANUMERIC OUTPUT 2 (J4-11.1)

Purpose

The program J4-11.1 outputs alphabetic and/or numeric information that has been stored without binarization (i.e., stored as raw character codes). The alphanumeric data must be stored as **five-character-long words in 6-bit format**. A data block for J4-11.1 must not exceed 63 cells. The data must be stored in consecutive memory cells.

The program operates as a subroutine called by a calling sequence from the main program. The starting address of the data block must be supplied to the subroutine before entry. Upon return to the main program, the specified information has already been printed.

Storage Requirements

- **Program Space:** 24 sectors
- **Temporary Storage:** None

Input

Before jumping to the subroutine, the calling sequence must insert into the first cell of the subroutine the address of the `Z` instruction which contains the starting address of the alphanumeric data block to be printed. This makes it possible to store the data block at any arbitrary location in memory, instead of forcing it to immediately follow the calling sequence as is typical. This `Z` instruction must not be omitted.

The data table consists of consecutive words of exactly 5 characters or typewriter functions each (the final word may contain fewer). The number of words in a single block must not exceed 63. The data must be stored in ascending order in consecutive cells.

During input storage, the data must be preceded by the constant keyword of **J1-10.0** so that it is not binarized. (The input keyword also automatically places an end-of-block marker "into the final word").

Example

Program Input Keyword	Memory Location	Instruction	Address	Description
	α	`R`	`L`	Store return address
	+1	`U`	`L`	Entry to subroutine
	+2	`Z`	`AA`	Pointer to data block
	+3	u.s.w.		Return to main program
`A0000003 A`	`AA`	`PLANN`		First data word
	+1	`ING P`		Second data word
	+2	`AYS`		Final data word

This example causes the printout of `PLANNING PAYS` and returns execution to +3 of the main program. The end-of-block marker is automatically stored by the program input keyword. The alphanumeric data can reside anywhere in memory.

Calling Sequence (J4-11.1)

Memory Location	Instruction	Address	Description
α	`R`	`L`	Insert the address of the `Z` instruction (+2) into `L`.
+1	`U`	`L`	Entry into the subroutine.
+2	`Z`	`A`	Starting address of the data block (A).
+3	u.s.w.		Return to the main program.

The instruction at α stores the address (+2) of the instruction containing the data block's starting address into the subroutine. The instruction at +1 executes a jump to the starting address (L) of the subroutine. Cell +2 contains the `Z` instruction with A, the starting address of the data block. Cell $\alpha + 3$ contains the instruction of the main program that will be executed first after exiting the subroutine.

Please note: The `Z` instruction must always reside directly within the calling sequence, even if the data block immediately follows the calling sequence.

Output

Upon return to the main program, the desired data block has already been output to the terminal.

Operation

1. **Time Required:** Approximately 190 characters per minute.
2. **Program Input:** The tape contains the program in two versions:
   - **a)** Arbitrarily relocatable hexadecimal (valid for Program Input 1).
   - **b)** Arbitrarily relocatable decimal (in coding sheet format).
3. **Required Input/Output Units:** No input units are required. The standard typewriter is selected for output by default. If a different output unit is to be used, the track addresses in cells `L+03`, `L+05`, `L+07`, `L+09`, and `L+11` of the subroutine must be replaced with the corresponding selection identifier (see the *LGP-21 Programming Manual*, where selection identifiers and their associated units are listed).
4. **Overflow:** Neither tested nor altered by the program.

Remarks

The program should ideally begin at **Sector 00** to ensure optimal drum execution timing.

13 Hexadecimal Input

HEXADECIMAL INPUT 3 J5-10.0

PURPOSE

The program J5-10.0 reads and stores hexadecimal tapes that were produced by the program Hexadecimal Output 3 (J4-10.2). Only full tracks can be stored, up to the limit of storage space available. The program is optimized for high-speed reading.

STORAGE REQUIREMENTS

Memory used: 3 tracks

Temporary storage: none

INPUT

After a jump to the starting address of the program, the computer selects the typewriter and requests the identification code for the input unit to be used by the program. Use (`0000` for the high-speed reader or `0200` for the Flexowriter reader). Enter the corresponding word in hexadecimal notation and press START.

The program reads tapes in a special hexadecimal word format. The words must be arranged in groups of 64, with each group corresponding to one track. Each group must be preceded by a four-character hexadecimal input command of the form `tt00`, whereby `tt` represents the track number. Each group must also be followed by a checksum of all the words in the group.

The words must be in the specific order described below, with the words separated by a single space. This optimizes the read/write speed of the program at the expense of pre-shuffling the words from 00, 01, 02,. . . , 64 into the indicated order.

> 56, 58, 60, 63, 01, 03, 05, 62, 00, 02, 04, 06, 08, 10, 12, 07, 09, 11,
> 13, 15, 17, 19, 14, 16, 18, 20, 22, 24, 26, 21, 23, 25, 27, 29, 31, 33,
> 28, 30, 32, 34, 36, 38, 40, 35, 37, 39, 41, 43, 45, 47, 42, 44, 46, 48,
> 50, 52, 54, 49, 51, 53, 55, 57, 59, 61.

The subroutine is capable of accepting a Stop and Jump" command of the form `8000ttss`, whereby `ttss` is a track/sector address in hexadecimal notation. The key word must always *follow* a checksum to avoid it being stored as part of the input. If this command is read after a checksum, the computer stops, and when STARTis pressed, a jump occurs to the address specified in the command and execution continues.

J5-10.0 performs a check sum of the input as it is read. After each full track, this checksum is compared with the one on the tape following the block. If

the checksums match, reading continues. If not, the program prints a carriage return and `e` is printed, and stops (see error stops).

CALLING SEQUENCE

The program is run by executing by a jump to its starting address. If the program has only just been read in, pressing START runs it.

OUTPUT

The program to be read in is stored on the specified tracks in the correct sequence. No other output.

OPERATION

- **Time requirement:** approx. 13 sec. for the input of one track.
- **Program input:** The tape contains the program in four versions:
  a) Arbitrarily relocatable hexadecimal with bootstrap on track 00.
  b) Arbitrarily relocatable hexadecimal with bootstrap on track 63.
  c) Arbitrarily relocatable hexadecimal permissible for Program Input 1 (J1-10.0).
  d) Arbitrarily relocatable decimal in coding sheet format.

  The input procedure varies depending on the version used:

Arbitrarily relocatable hexadecimal with bootstrap

Arbitrarily relocatable hexadecimal programs with their own bootstrap permit the operator to enter the program either via the paper tape typewriter or via the high-speed reader, since a special bootstrap is provided for each input unit. They appear on the tape in the following sequence:

1. Track-00-bootstrap for input of the program via the high-speed reader.
2. Track-00-bootstrap for input of the program via the paper tape typewriter.
3. Program J5-10.0.
4. Track-63-bootstrap for input of the program via the high-speed reader.
5. Track-63-bootstrap for input of the program via the paper tape typewriter.
6. Program J5-10.0.

One chooses a bootstrap corresponding to the desired track and input unit, and enters it according to the bootstrap procedure (described in Program Input 1).

The first instruction stored in the bootstrap is the adress modifier, which sets the starting address of the program to be stored. The following modifiers are preset on the tape, and can be changed as desired:

- Track-00-bootstraps have the modifier `0000`.
- Track-63-bootstraps have the modifier `6100`.

Arbitrarily relocatable hexadecimal, permissible for Program Input 1

This version is read in according to the procedure described in Program Input 1.

Required input/output units:

The input unit is selected by a suitable selection code at the beginning of the operation of the subroutine (see Input). An output unit is not required.

Error stops:

Printout	Meaning and Remedy
`e`	The checksums do not match. Rewind the tape to the beginning of the last group and restart.

REMARK

The program should be loaded in an even-numbered track, so that the reading-in occurs optimally.

ADDENDUM

The tape "Hexadecimal Input 3" in arbitrarily relocatable hexadecimal version with Track-00- or Track-63-bootstrap, can be patched so that it can be stored at the beginning of a program tape. After the following alterations, the program will automatically select the same input unit it was read on for its own input.

```
'c01gj'8'u02g4'j's01k0'10'c0068'30'u0034'64'u0068'w4'b00k.j'w8'
u02g8'70'u0074'78'u007j'80'u0084'88'c0118'8j'b0100'
1q0'a0000'1q4'a0000'1q8'a0000'1qj'a0000'1w0'a0000'1w4'a0000'
1w8'a0000'200'80010000'204'18'2f4'kj'2f8'q4'2fj''c00w4''
```

The alteration adds four additional instructions in the first line and modifies the last word in the last line. The rest of the program, including bootstrap, remains unaltered.

14 Specialized Input/Output

14.1 Angle or Direction I/O

DIRECTION OR ANGLE INPUT AND OUTPUT 1 J9-11.0

PURPOSE

The program J9-11.0 reads or prints a positive or negative angle, or a direction, consisting of the specification of the quadrant and a positive angle. All angles must be smaller than 512°, and direction angles must not be greater than 90° [sic][46]. The entered angle or peak circle [vertex circle] of the direction is binarized and placed into the accumulator.

The printing of an angle occurs in decimal notation in degrees and minutes, each followed by a dash, and in seconds, accurate up to one-tenth of a second. For a direction, the angle is printed in the same way, except that a character `N` (North) or `S` (South) precedes it and a character `E` (East) or `W` (West) follows it.

The program operates as a subroutine that is called by a calling sequence of the main program. Before the jump into the subroutine, the calling sequence must store the return address. In the case of the output function, the angle or the direction must stand in the accumulator before the entry into the subroutine occurs. Upon return to the main program, the angle or the direction is read and placed into the accumulator if the input function was called, or the angle or the direction is printed if the output function was called.

STORAGE REQUIREMENTS

Storage cells used: 2 tracks, 61 sectors

Temporary storage: sectors 2, 5, 15, 17, 48, 51, 52, 56, and 60 of track 63

INPUT

The input for the program consists of one of the two following possibilities:

1. A positive angle in decimal notation in degrees, minutes, and seconds.
2. A direction, consisting of the characters `S` or `N`, a positive angle in decimal notation in degrees, minutes, seconds, and the characters `E` or `W`.

[46]Architectural Note: The German text states ‘Richtungswinkel dürfen nicht größer als 600 sein” (direction angles must not be greater than 600) in the Purpose section, but explicitly contradicts this later under Input, stating ‘Die Winkelangaben bei Richtungen dürfen nicht größer als 90° sein” (The angle specifications for directions must not be greater than 90°). In LGP-21 quadrant geometry, 90° is the correct architectural limit. The ‘600’ is a typographical error in the original manual, likely a misprint of 90° or 60° on a standard typewriter.

Each value must be specified as one word; e.g.: `1422038'` to specify an angle of 142°20′38″; `S204010W` to specify the direction SW 20°40′10″; or `N000342E` to specify the direction NE 0°3′42″ [sic][47].

The angle specification can be at most seven digits; leading zeros on the left may be omitted. All angles must be smaller than 512°. Every direction specification must consist of eight characters; for vanishing quantities, the zeros must be written out completely; algebraic signs are not specified. The angle specifications for directions must not be greater than 90°.

CALLING SEQUENCE

In the subroutine, three calling sequences are provided: 1. for input, 2. for angle output, 3. for direction output. In every case, L_0 is the starting address of the subroutine.

1. Input:

Intended for reading in an angle or a direction.

Memory Location	Instruction	Address	Meaning
α	`R`	$L_0 + 47$	Store return address in the subroutine.
$+1$	`U`	L_0	Entry into the subroutine.
$+2$		*etc.*	Return to the main program.

2. Angle Output:

Intended for printing an angle. The angle must reside in the accumulator at $q = 9$ before the jump into the subroutine occurs.

Memory Location	Instruction	Address	Meaning
$\alpha - 1$	`B`	Γ [sic][48]	Bring the angle into the accumulator.
α	`R`	$L_0 + 249$	Store return address in the subroutine.
$+1$	`U`	$L_0 + 100$	Entry into the subroutine.
$+2$		*etc.*	Return to the main program.

3. Direction Output:

Intended for printing a direction. The peak circle of the direction must reside in the accumulator at $q = 9$ when the jump into the subroutine occurs.

[47] Architectural Note: The text contains minor typos in the symbols used for arcminutes and arcseconds (using various configurations of apostrophes such as 38′ for seconds or 5120 instead of 512°). These are structural transcription discrepancies native to the German source manuscript type-setting.

Memory Location	Instruction	Address	Meaning
$\alpha - 1$	`B`	`C3`	Bring peak circle into the accumulator.
α	`R`	$L_0 + 219$	Store return address in the subroutine.
$+1$	π [sic][49]	$L_0 + 200$	Entry into the subroutine.
$+2$		*etc.*	Return to the main program.

In the calling sequences 2 and 3, the instruction in $\alpha - 1$ does not need to be a "Bring" instruction. Any instruction sequence that brings the quantity to be outputted at $q = 9$ into the accumulator is permissible. In all calling sequences, the instruction in α stores the return address $(\alpha + 2)$ into the subroutine. The instruction in $\alpha+1$ effects a jump to the corresponding location of the subroutine. In $\alpha + 2$ stands the instruction of the main program that is executed first after running through the subroutine.

OUTPUT

The program binarizes every entered angle or the peak circle of every direction specification and places them at $q = 9$ into the accumulator.

Each angle printout occurs in decimal form in degrees, minutes, seconds, and tenths of a second, whereby the degrees and minutes are followed by a separation dash; e.g., an angle of $80°37'01''$ would be printed out as `80-37-01.0`.

Upon output of a direction, the angle is written in the same manner, however one of the letters `N` or `S` precedes the angle specification, and one of the letters `E` or `W` follows it; i.e., a direction of NW $42°1'28''$ would be printed as `N 42-01-28.0W`.

OPERATION

1. **Accuracy:** max. error: ± 1 second.
2. **Time requirement:** approx. 1.5 seconds to read or write a quantity.
3. **Program input:** The tape contains the program in two ways:
   a) arbitrarily relocatable, hexadecimal, permissible for Program Input 1, and
   b) arbitrarily relocatable, decimal, in coding sheet format.
4. **Required input/output units:** The subroutine selects the paper tape typewriter both for the input and for the output. The selection can be changed by changing the following memory contents:
5. **Overflow:**
   Is not tested upon entry, but is however reset upon output.

Input (`80XItt00`)	**Output (`80XPtt00`)**	**Output (`XPtt00`)**
$L_0 + 1$	$L_0 + 105$	$L_0 + 116$
$L_0 + 113$	$L_0 + 121$	$L_0 + 126$
$L_0 + 123$	$L_0 + 133$	$L_0 + 136$
$L_0 + 131$	$L_0 + 142$	$L_0 + 210$
$L_0 + 140$	$L_0 + 216$	
	$L_0 + 218$	

REMARK

After each outputted value, a tabulator jump [tab command] is executed.

To suppress the printing of the period and the last decimal place, change $L_0 + 137$ from `E0252` to `U0247`.

15 Tracing

TRACE 1 K1-10.0

PURPOSE

The program K1-10.0 facilitates the debugging of a fixed-point program residing in memory by providing a detailed printout. Any or all addresses, instructions, and/or the results of their execution can be printed. Addresses are written in decimal track/sector notation; calculation results are written in hexadecimal form.

STORAGE REQUIREMENTS

Storage cells used: 3 tracks, 51 sectors

Temporary storage: none

INPUT

This subroutine stablishes simulated counters, instruction registers, an accumulator, and a PST [Program Sense Switch] in memory.

When the program is executed, the computer selects the Flexowriter, prints a carriage return, and requests the address of the main program which is to be traced in decimal track/sector notation.

When this address has been entered, or when PST has been turned ON, the computer selects the typewriter, executes a carriage return, and requests an output formatting command. A list of command characters and their functions follows. The registers mentioned in these commands -— accumulator, counter register, and instruction register -— are the registers simulated by this program.

Command	Function
T	Turns on simulated PST. This "switch" has the same function while tracing the tested program as the real PST would during normal operation of the traced program.
C	Print the contents of the simulated counter register in decimal track/sector notation after the execution of each instruction of the traced program.
R	Print the contents of the instruction register in instruction format after execution of each instruction of the traced program. The instruction is preceded by a space character or a minus sign.
A	Print the contents of the accumulator in hexadecimal notation after the execution of each instruction of the traced program.

Note: Any combination of the above command characters can be entered and the corresponding commands remain in effect until the next input. Entering a

new command disables all functions first and then re-enables the ones entered. These characters may be entered in any order.

Command	Function
X	Restart the program and enter a new address for tracing.
O	Continue tracing from the point where execution was stopped, without printout.
(none)	Pressing START without entering T, C, R, or A causes tracing to continue from the point where execution was stopped; the previous key words remain effective.

Note: The accumulator is never re-initialized by "Trace 1". The program under test must take care of this housekeeping itself.

CALLING SEQUENCE

Start execution via a jump to the starting address.

OUTPUT

The program prints the content of the simulated registers and/or the accumulator depending on the key word specification after each instruction of the program under test has been executed.

The instructions of the program under test are in fact executed exactly as they would be during normal program flow. Any output from the traced program is written on the same line as the content of the simulated registers. When the program traces without printing, any output that occurs is executed exactly where the P instruction of the traced program has it.

If the physical PST switch is turned ON during tracing, the computer print a carriage return and the contents of the simulated registers according to the currently-specified command characters. The computer then stops and requests the entry of a command. Continuously pressing START in this state causes the execution of the respective next instruction of the traced program, exactly as if the computer were in ONE OPER mode. This continues until PST is turned OFF.

Note that tracing without output can cause confusion if the traced program executes an I instruction. If the PST in ON, the computer prints a carriage return, the address of the last traced instruction, and requests a new command. In this state, pressing START allows execution to continue, but if the following instruction requests input again, the computer stops again -— but this time Any input will be routed to the program under test. This state can be recognized by the fact that there was no address printout before the computer stopped.

OPERATION

1. **Program input:**
   The tape contains the program in two versions:

   a) arbitrarily relocatable, hexadecimal, permissible for J1-10.0, and

   b) arbitrarily relocatable, decimal, in coding sheet format.

2. **Required input/output units:**
   The subroutine selects the paper tape typewriter for input and output.

3. **Program sense switches:**

Switch	Position	Function K1-10.0
PST	OFF	No effect.
	ON	Jump to an input instruction of the program to enter new commands.

4. **Error stops:**
   The subroutine executes all `Z` instructions of the program under test as stops. The only stop instructions in the trace program itself are before the initial output formatting command and before the entry of a new trace address.

REMARK

If the traced program requests input during checking, the least significant bit of the entered word is lost.

16 Memory Dumps

MEMORY PRINT 1 K2-10.0

PURPOSE

The program K2-10.0 outputs the contents of a specified memory region. There are three possible types of output:

a) in coding sheet format,

b) arbitrarily relocatable decimal, permissible for J1-10.0,

c) both: a and b.

The words can be output as instructions or as hexadecimal constants, depending on appearance or according to the specifications of the user. If hexadecimal words are punched, they are preceded by the input key word for hexadecimal constants. Output occurs cell by cell in ascending sequence.

STORAGE REQUIREMENTS

Storage cells used: 9 tracks

Temporary storage: none

INPUT

After a jump to the starting address of K2-10.0, the computer requests the following information:

Question	Answer
`lins`	Number of lines per page as a two-digit decimal number.
`locs`	Number of memory addresses to be printed on a page as two decimal digits.
`wrds`	The number of words to be printed in one line (only becomes effective if PS-4 is ON) as two decimal digits.
`skip`	The number of blank lines between two pages as a two-digit decimal number.

Note: The above answers must consist of two digits: zero must be entered as "`00`". The blank lines between pages are output when either the full number of memory addresses has been printed, or when the number of outputted lines plus "`skip`" equals "`lins`".
Note: In all address specifications, the second address must be higher than the first.

Question	Answer
`md a`	The boundaries of the memory region in which the modifier is to become effective (see below) in one word in decimal track/sector notation (i.e., `TTSSTTSS`).
`modi`	The value that is to be subtracted from the address part of the instructions in the modification region, in decimal track/sector notation.
`hx a`	The boundaries of the region that is to be output in the form of hexadecimal words, regardless of content, as one word per region in decimal track/sector notation (i.e., `TTSSTTSS`). The termination of input is designated by pressing START twice after the last address.
`pnch`	The boundaries of the output region as one word in decimal track/sector notation (i.e., `TTSSTTSS`).

If PS-32 is ON when the program starts, the computer skips the first four questions and uses the values from the previous run or the defaults: `lins=51`, `locs=32`, `wrds=08`, and `skip=08`.

The program *does not stop* after the answer to "`pnch`". The typewriter lever PUNCH ON should already be pressed when START is pressed.

CALLING SEQUENCE

The program is run by a jump to the starting address.

OUTPUT

The output occurs in one of two versions, depending on the setting of PS-4.

aON) Each line begins with an address (absolute minus modifier) followed by a space character and one instruction per line. Input key words for hexadecimal words are printed out one to a line and are counted toward the number of lines per page.

OFF) Each line begins with the address (absolute minus modifier) of the first word of that line, followed by a space character and as many words as specified for "`wrds`". Input key words for hexadecimal words are not counted toward the number of words per line.

For both formats, eight characters are output per word. Hexadecimal words contain leading zeros where applicable. Instruction words replace leading zeros with spaces.

All instructions outside the modification region are preceded by an "`X`" during output. `P` and `I` instructions are never modified and are always provided

with an `X`[50]. Hexadecimal words are output in groups of at most 63 words; each group is preceded by a command for hexadecimal input (see J1-10.0 regarding the command).

Every word that contains a "1" bit outside of the sign position of the operation part and the operand address field is output as a hexadecimal word. If a word looks like an instruction but is in reality a hexadecimal word, this must be specified during the entry of `hx a`. If instructions in the modification region are not designated as hexadecimal words, or if they are not `P` or `I` instructions, the modifier is subtracted from the operand addresses and they are not dumped with an `X`.

Memory is dumped in ascending location order. The program sense switches can be used as noted below to control the output type and format. Once the whole of the selected region has been output, the computer stops; pressing START restarts the program.

OPERATION

1. **Program input:**
   The tape contains the program in two versions:
   a) arbitrarily relocatable, hexadecimal, permissible for Program Input 1, and
   b) arbitrarily relocatable, decimal, in coding sheet format.
2. **Required input/output units:**
   The subroutine selects the paper tape typewriter for input and output.
3. **Program sense switches:**

REMARK

Optimal operation is independent of the starting sector number.

If only a single memory dump is needed, Memory Print 2, K2-10.1 is preferable.

ADDENDUM

Suppression of the address printout:
During the generation of a decimal tape, an address is output at the beginning of each line. This tape can be read in via the typewriter using Program Input 1, but not via the high-speed reader.

K2-10.0 can be patched to suppress the output of the memory addresses; this produces a tape that can be entered via the high-speed reader.
L_0 is the starting address of K2-10.0.

[50] Relocating unit constants is a bad idea.

Switch	Position	Function
PS-4	ON	Prints an address (minus modifier) and a predetermined number of words per line. Key words for hexadecimal input are not counted.
	OFF	Prints an address and its content per line. A key word for hexadecimal input is counted as one line.
PS-8	ON	Jump to the starting address of the subroutine and request new parameters when the next word is output.
	OFF	No effect.
PS-16	ON	Halt after the output of each word. Continuation after pressing "Start".
	OFF	No effect.
PS-32	ON	The computer uses the page format from the previous run, or if the program was only just read in, the page format native to the original program. (i.e., the first four questions and answers are omitted).
	OFF	Computer requests page format specifications.

In Cell	Replace	By
`B[`L_0 `+ 0242]` [sic][51]	L_0 `+ 0234`	`U[`L_0 `+ 0241]`

16.1 Memory Dump 2

K2-10.1

PURPOSE

K2-10.1 dumps the contents of consecutive locations in ascending sequence. The words are printed in hexadecimal or in instruction format, depending on the content or the state of the PST.

STORAGE REQUIREMENTS

Memory used: 2 tracks

Temporary storage: none

INPUT

After jumping to the starting address, the computer requests the following output parameters:

1. L_s Starting address of the region to be output in decimal track/sector notation.
2. L_e Ending address of the region to be output in decimal track/sector notation.

The ending address of a region must in each case be higher than the starting address. If L_e is numerically smaller than L_s, the computer prints L_e and a carriage return, and requests new parameters.

After the dump the specified memory region, the computer requests new output parameters.

CALLING SEQUENCE

The program is executed by a jump to its starting address.

OUTPUT

The program prints one address and eight words per line until the content of the last address has been printed out. Hexadecimal words are printed if there are "1" bits outside of the sign, operation part, and operand address part, or if the PST is ON. Instruction words are printed if the above conditions do not apply. Negative instructions are preceded by a minus sign. The contents of consecutive cells are output in ascending sequence.

OPERATION

1. **Program input:**
   The tape contains the program in two versions:
   a) arbitrarily relocatable, hexadecimal, permissible for Program Input 1, and
   b) arbitrarily relocatable, decimal, in coding sheet format.
2. **Required input/output units:**
   The subroutine selects the paper tape typewriter for input and output.
3. **Program sense switches:**

Switch	Position	Function
PST	ON	Print in hexadecimal word format.
	OFF	Print in the format defined by the memory content.

REMARK

This program is intended as a simple tool for memory investigation. If an output useful for Program Input 1 is desired, one should use K2-10.0.

17 Sorting

17.1 SORTING 1

L1-10.0

PURPOSE

The program L1-10.0 sorts a list of numbers within memory by magnitude (smallest value to largest value) according to a specified mask. The sorted numbers are stored in the same memory region as the unsorted ones, so no additional memory space is required. The mask determines which parts of the words are compared.

The program operates as a subroutine called by a calling sequence from the main program. Upon jumping to the subroutine, the starting address of the numbers to be sorted must reside in the accumulator, and the address of the mask must have been stored in the subroutine. The two cells following the jump instruction must contain the mask and the ending address of the list. Upon returning to the main program, the numbers stand sorted by magnitude (smallest to largest) in the original memory region of the list.

STORAGE REQUIREMENTS

Storage cells used: 1 track, 32 sectors

Temporary storage: sectors 54, 55, 62, and 63 of track 63

INPUT

Sequence of input variables:

1. L_1 Starting address of the numbers to be sorted in decimal track/sector notation.
2. Arbitrary mask controlling the sorting process, in hexadecimal notation.
3. L_2 Ending address of the numbers to be sorted in decimal track/sector notation.

The mask must not contain a "1" in the sign position, as the subroutine ignores the sign of the numbers being sorted. L_1 and L_2 are inclusive.

Upon jumping to the subroutine, L_1 at $q = 29$ must reside in the accumulator. The cells following the jump instruction must contain the mask and L_2. The numbers to be sorted must reside in consecutive cells.

CALLING SEQUENCE

The instruction at $\alpha - 1$ brings L_1 into the accumulator @ 29. (Any instruction that achieves this may be used.) The instruction at α stores the address of the mask $(\alpha + 2)$ in the subroutine. The instruction at $\alpha + 1$ effects a jump to

Memory Cell	Instruction	Address	Meaning
$\alpha - 1$	`B`	$\langle \ldots \rangle$	Bring L_1 @ 29.
α	`R`	$L_0 + 3$	Store the address of the mask in the subroutine.
+1	`U`	L_0	Jump to the subroutine.
+2	`,0000001'`		Mask.
+3	`Z`	L_2	Specification of L_2.
+4		*etc.*	Return to the main program.

L_0, the starting address of the subroutine. Cell $\alpha + 2$ contains the hexadecimal mask. The instruction at $\alpha + 3$ contains L_2. Cell $\alpha + 4$ contains the instruction of the main program that is executed first after the subroutine runs.

OPERATION

1. **Execution time:**
   Average values:
   - 32 words – 114 sec.
   - 64 words – 6 min.
   - 128 words – 27 min.
2. **Program entry:**
   The tape contains the program in two versions:
   a) arbitrarily relocatable, hexadecimal, permissible for Program Input 1, and
   b) arbitrarily relocatable, decimal, in coding sheet format.
3. **Required input/output units:**
   No input or output units are required.
4. **Overflow:**
   Ignored upon entry and not reset upon exit.

EXAMPLE

Suppose numbers are stored in cells 1500 to 1731, which are to be sorted according to their content in bit positions 7 to 11 and 24 to 29. Let "Sorting 1" be stored at 700 to 832, and the main program occupies cells 900 to 1215. The calling sequence would look as follows:

The instruction in 1011 brings 1500 @ 29 into the accumulator; the instruction in 1012 stores the address of the mask to 0703; and the instruction in 1013 effects a jump to the starting address of "Sorting 1". Cell 1014 contains the sorting mask. Note that only those bit positions that correspond to "1" bits in the mask are compared. The instruction in 1015 defines the ending address of

Memory Location	Instruction	Address	Meaning
1011	B	1211	Bring L_1 @ 29.
1012	R	0703	Set the address of the mask.
1013	U	0700	Jump to the subroutine.
1014	,0000001' 01WO OOWJ		Mask.
1015	Z	1731	L_2.
1016		*etc.*	Return to main program.
1211	Z	1500	L_1.

the list. Cell 1016 contains the main program instruction to which execution returns.

17.2 SORTING 2

L1-10.1

PURPOSE

The program L1-10.1 sorts one-word or multi-word "intervals" [records] within a contiguous memory region by magnitude (smallest value to largest value) according to a designated "sorting key word" located within each respective record. Each record can be 1 to 64 words long. An additional memory region is required to temporarily buffer record contents during the sorting process. Sorting occurs relative to an arbitrary mask.

"Sorting 2" operates as a subroutine called by a calling sequence from the main program. Upon jumping to the subroutine, a key word specifying the number of records to process and the starting address of the first record must reside in the accumulator. Additionally, the calling sequence must supply the "sorting key word" position and the mask. Upon returning to the main program, the records stand sorted in the original memory region.

STORAGE REQUIREMENTS

Storage cells used: 4 tracks

Temporary storage: 2 tracks ($L_0 + 0400$ to $L_0 + 0563$, where L_0 is the starting address of the subroutine).

INPUT

The input consists of three hexadecimal key words supplied by the calling sequence and the record contents to be sorted. The key words are:

1. R, the number of records at $q = 15$; and L_1, the starting address of the first record at $q = 29$.

2. N, the number of words in each record at $q = 15$; and K, the position of the sorting key word within a record at $q = 29$. (For example, $K = 3$ at $q = 29$ specifies that the third word of the record is the sorting key).

3. A mask indicating the bit positions to be compared during sorting.

The number of records sortable in a single run is limited only by available memory space. For N (the number of words per record), the following applies: $1 \leq N \leq 64$. The mask should not contain a "1" in the sign position, as the subroutine ignores the sign of the words during sorting.

The records to be sorted must reside in consecutive cells.

Memory Location	Instruction	Address	Meaning
$\alpha - 1$	`B`	$\langle \ldots \rangle$	Bring the word containing R and L_1.
α	`R`	$L_0 + 20$	Store the address of the control data.
+1	`U`	L_0	Entry to subroutine.
+2	`,0000002' N K`		N @ 15; K @ 29.
+3	`Mask`		Mask.
+4		*etc.*	Return to main program.

CALLING SEQUENCE

The instruction at $\alpha - 1$ brings R @ 15 and L_1 @ 29 into the accumulator. (Any instruction that accomplishes this is permissible.) The instruction at α stores the address $(\alpha + 2)$ of the instruction containing N and K into the subroutine. The instruction at $\alpha + 1$ effects a jump to L_0, the starting address of the subroutine. Cell $\alpha + 2$ contains N @ 15 and K @ 29. Cell $\alpha + 3$ contains the mask. After $\alpha + 4$, execution returns from the subroutine.

OPERATION

1. **Execution time:**
   An average of 3 seconds per word for small examples.

2. **Program entry:**
   The tape contains the program in two versions:

   a) arbitrarily relocatable, hexadecimal, permissible for Program Input 1, and

   b) arbitrarily relocatable, decimal, in coding sheet format.

3. **Required input/output units:**
   No input or output units are required.

4. **Overflow:**
   Ignored upon entry and not reset upon exit.

EXAMPLES

Let the subroutine begin at 300; the main program occupies tracks 10 to 15.

Example 1

Sort 37 words, stored in 4900 to 4936, treated as 37 single-word records, considering only bits 15 to 30.

Note that during sorting, the subroutine only compares word components that match "1" bits in the mask.

Memory Location	Instruction	Address	Meaning
1010	B	1559	Bring R and L_1.
1011	R	0320	Store the N-K word address.
1012	U	0300	Jump to "Sorting 2".
1013	,0000002' 0001	0004	$N = 1$ @ 15; $K = 1$ @ 29.
1014	0001 wwwQ		Mask for bits 15 to 30.
1015		*etc.*	Return to main program.
1559	,0000001' 0025	3100	$r = 37$ @ 15; $L_1 = 4900$.

Example 2

Sort 32 words, stored in 3200 to 3231, treated as eight 4-word records. They are to be sorted by magnitude according to the value of the second word of each record (smallest value to largest value; sorting the full records based on the magnitude of their respective second words). The comparison should only occur at bit positions 4 to 11 and 16 to 20.

Memory Location	Instruction	Address	Meaning
1401	B	1563	Bring R and L_1.
1402	R	0320	Store the N-K word address.
1403	U	0300	Jump to "Sorting 2".
1404	,0000002' 0004	0008	$N = 4$ @ 15; $K = 2$ @ 29.
1405	OwwO W800		Mask for bits 4–11 and 16–20.
1406		*etc.*	Return to main program.
1563	,0000001' 0008	2000	$R = 8$ @ 15; $L_1 = 3200$.

Sample printouts follow, showing what both calling sequences achieve.

SAMPLE PRINTOUT

Example 1

Location	Unsorted
4900	`Owj00004'`
4901	`g0000010'`
4902	`00jj0018'`
4903	`Okqw0026'`
4904	`00qj001j'`
4905	`7wqj002f'`
4906	`Ofg00008'`
4907	`Ow000030'`
4908	`0k00000f'`
4909	`0q00003j'`
4910	`0g000032'`
4911	`Ow00000j'`
4912	`0q00003w'`
4913	`0f00001f'`
4914	`qj00003j'`
4915	`00000014'`
4916	`w000012j'`
4917	`0w000006'`
4918	`0q00000b'`
4919	`qk000020'`
4920	`3j000064'`
4921	`fg000190'`
4922	`kf000002'`

Location	Sorted
4900	`kf000002'`
4901	`Owj00004'`
4902	`0w000006'`
4903	`Ofg00008'`
4904	`0k00000f'`
4905	`0w00000j'`
4906	`0q00000b'`
4907	`g0000010'`
4908	`0k000012'`
4909	`00000014'`
4910	`0j000016'`
4911	`00jj0018'`
4912	`0f00001f'`
4913	`00qj001j'`
4914	`0000001g'`
4915	`qk000020'`
4916	`0k000022'`
4917	`0j000024'`
4918	`0k000028'`
4919	`7wqj002f'`
4920	`0w000030'`
4921	`0g000032'`
4922	`0q000034'`

Location	Unsorted
4923	`0k000012'`
4924	`00000036'`
4925	`0k000022'`
4926	`0j000016'`
4927	`0q000038'`
4928	`Ow000258'`
4929	`0f00003w'`
4930	`0j000024'`
4931	`0000001w'`
4932	`0g00003f'`
4933	`0f0000j8'`
4934	`0g000034'`
4935	`0k000028'`
4936	`Ow0001w2'`

Location	Sorted
4923	`00000036'`
4924	`0q000038'`
4925	`0g00003f'`
4926	`0q00003j'`
4927	`qj00003j'`
4928	`0g00003q'`
4929	`0f00003q'`
4930	`3j000064'`
4931	`0f0000j8'`
4932	`w000012j'`
4933	`fg000190'`
4934	`Ow0001w2'`
4935	`Ow000258'`
4936	`Okqw0026'`

Example 2

Location	Unsorted
3200	080403jO'
3201	02q05000'
3202	00001000'
3203	00g00005'
3204	0404003g'
3205	03g0q800'
3206	00048000'
3207	02q0w800'
3208	030j03g0'
3209	02q0w800'
3210	03g00000'
3211	0010j0wO'
3212	040403gO'
3213	0160q000'
3214	00j0q000'
3215	000gq000'
3216	040403jO'
3217	03g0f800'
3218	OwwOw800'
3219	0q801000'
3220	090w03jo'
3221	03g0f800'
3222	016j8000'
3223	03g0f800'
3224	010303jo'
3225	01607000'
3226	Ofg09800'
3227	005080jo'
3228	0f0803jO'
3229	0080f800'
3230	000f0800'
3231	01606800'

Location	Sorted
3200	0f0803jO'
3201	0080f800'
3202	000f0800'
3203	01606800'
3204	01050330'
3205	01607000'
3206	Ofg09800'
3207	00j080jO'
3208	040403g0'
3209	0160q000'
3210	00j0q000'
3211	000gq000'
3212	080403jO'
3213	02g05000'
3214	00001000'
3215	00g00004'
3216	030j03gO'
3217	02q0w800'
3218	03g00000'
3219	0010jOwO'
3220	040403j0'
3221	03g0f800'
3222	OwwOw800'
3223	0q801000'
3224	090w03j0'
3225	03g0f800'
3226	016j8000'
3227	03g0f800'
3228	0404003f'
3229	03g0q800'
3230	00048000'
3231	02q0w800'

17.3 SORTING 3

L1-10.2

PURPOSE

The program L1-10.2 sorts a list of numbers within memory by magnitude (smallest value to largest value) according to a specified mask. Although the

sorted numbers are written back to their original memory region, additional buffer cells are required. The mask determines which parts of the words are compared.

The program operates as a subroutine called by a calling sequence from the main program. Upon jumping to the subroutine, the number of words in the list must reside in the accumulator. The cells following the jump instruction must contain the starting address of the list and the mask. Upon returning to the main program, the numbers stand sorted by magnitude (smallest to largest) in the original memory region of the list.

STORAGE REQUIREMENTS

Storage cells used: 3 tracks

Temporary storage: none

INPUT

The input variables for "Sorting 3" are listed below in the sequence of their appearance in the calling sequence:

1. N, the number of values to be sorted in decimal track/sector notation [sic][52].
2. S_0, the starting address of the words in decimal track/sector notation.
3. The arbitrary mask directing the sorting process, in hexadecimal notation.

The numbers to be sorted must be positive and reside in consecutive cells.

CALLING SEQUENCE

Memory Location	Instruction	Address	Meaning
$\alpha - 1$	`B`	$\langle \dots \rangle$	Bring N @ 29.
α	`R`	$L_0 + 63$	Store the address of the first list word in the subroutine.
$+1$	`U`	L_0	Jump to the subroutine.
$+2$	`Z`	S_0	Starting address of the list.
$+3$	`,0000001' Mask`		Mask.
$+4$		*etc.*	Return to main program.

The instruction at $\alpha - 1$ brings N into the accumulator. (Any instruction that achieves this is permissible.) The instruction at α stores the address $(\alpha+2)$ of the first list word in the subroutine. The instruction at $\alpha + 1$ effects a jump

[52]Architectural Note: Although the text states "in decimal track/sector notation," the subsequent programming example shows that N is loaded as a simple integer scalar value (96 words). The address representation inside the memory word maps to the standard data scaling format used by the LGP-21.

to L_0, the starting address of the subroutine. Cell $\alpha+2$ contains the address S_0 of the first list word, and cell $\alpha+3$ contains the hexadecimal word representing the mask. After $\alpha+4$, control returns from the subroutine.

OPERATION

1. **Execution time:**
   Average values:
   - 32 words – 1.95 min.
   - 64 words – 5.50 min.
   - 128 words – 12.87 min.
2. **Program entry:**
   The tape contains the program in two versions:
   a) arbitrarily relocatable, hexadecimal, permissible for Program Input 1, and
   b) arbitrarily relocatable, decimal, in coding sheet format.
3. **Required input/output units:**
   No input or output units are required.
4. **Overflow:**
   Ignored upon entry and not reset upon exit.

EXAMPLE

The contents of cells 1514 to 1645 are to be sorted by magnitude, considering only bit positions 3 to 9 and 18 to 23. Let "Sorting 3" occupy 0500 to 0763, and the main program occupies 3600 to 4228.

Memory Location	Instruction	Address	Meaning
3715	B	4221	Bring N (96 words).
3716	R	0563	Set the list address.
3717	U	0500	Entry to subroutine.
3718	Z	1514	Starting address of the list.
3719	,0000001' 1WJO	3WOO	Mask.
3720		*etc.*	
4221	Z	0132	$N = 1$ track, 32 sectors.

Please note that during sorting, only those bit positions that correspond to "1" bits in the mask are compared.

17.4 SORTING 4

L1-10.3

SORTING 4 L1-10.3

PURPOSE

The program L1-10.3 sorts single-word or multi-word "intervals" [records]. In a list of single-word records, every word is treated as a comparison word [key]. In a list of multi-word records, any arbitrary word at the same position across all records can be designated as the comparison key word.

The sorting process can be executed using any of the three following methods:

Method I: Sorts the comparison key words by magnitude (smallest value to largest value) and generates a contiguous list of the addresses where these key words are located; i.e., the first cell of the new list contains the address of the key word with the smallest numerical value from the original list.

Method II: Sorts the comparison key words by magnitude (smallest value to largest value) and creates a contiguous list containing only the sorted key words themselves.

Method III: Sorts the comparison key words by magnitude (smallest value to largest value) and generates a new list containing the fully sorted records. With this method, the new sorted list has the exact same length as the original list.

Note: If two comparison key words are identical in magnitude, the first one encountered is treated as though it has the smaller value.

The program is invoked via a calling sequence from the main program. Upon jumping to "Sorting 4", the starting address of the parameter list must reside in the accumulator. Upon returning to the main program, the list stands sorted and stored in accordance with the specifications in the parameter list.

STORAGE REQUIREMENTS

Memory used: 3 tracks

Temporary storage: none

PROGRAM VARIABLES

The seven program parameters must be specified in the following exact sequence:

1. `bbbb`, the starting address of the unsorted list.
2. `aaaa`, the starting address of the memory region where the sorted list is to be stored.

3. `nnnn`, the number of records [intervals] in the list, in decimal track/sector notation.

4. `wwww`, the number of words per record [interval], in decimal track/sector notation.

5. `kkkk`, specifies which word within each record is to be used as the comparison key word.

6. `t`, the selection code for the sorting method: 1 for Method I, 2 for Method II, or 3 for Method III.

7. `mmmm mmmm`, the mask, in hexadecimal notation.

Note: The mask must not contain a "1" in the sign position, as the program ignores the sign of the key words.

CALLING SEQUENCE

The starting address of the parameter list is designated as β. This address β must reside in the accumulator when the jump to the subroutine occurs.

Memory Location	Instruction	Address	Meaning
α	`B`	L	Bring the starting address of the parameter list (β) into the accumulator.
$+1$	`R`	$L_0 + 1$	Store the return address.
$+2$	`U`	L_0	Entry to the subroutine.
$+3$		*etc.*	Return point in the main program.

The parameter block list for the subroutine is defined as follows:

Memory Location	Instruction	Address	Meaning
β	`XZ`	`bbbb`	Starting address of the unsorted list.
$+1$	`XZ`	`aaaa`	Starting address of the sorted destination region.
$+2$	`XZ`	`nnnn`	Number of records [intervals] per list.
$+3$	`XZ`	`wwww`	Number of words per record [interval].
$+4$	`XZ`	`kkkk`	Position index of the comparison key word within all records.
$+5$	`XZ`	$000t$	Selection code for the sorting method ($t = 1, 2$, or 3).
$+6$	`mmmmmmmmm`		Comparison mask.

OPERATION

1. **Execution time:**
   Examples:
   - 64 single-word entries: 27 min. 15 sec.
   - 32 two-word entries: 7 min. 25 sec.
2. **Program entry:**
   The tape contains the program in two versions:
   a) arbitrarily relocatable, hexadecimal, permissible for Program Input 1, and
   b) arbitrarily relocatable, decimal, in coding sheet format.
3. **Required input/output units:**
   No input or output units are required.
4. **Overflow:**
   Ignored upon entry and not reset upon exit.

EXAMPLE

The example demonstrates the three available sorting methods. Sample calling sequence for "Sorting 4" stored starting at 4100:

Memory Location	Instruction	Address	Meaning
3013	`B`	3129	Bring parameter word address.
3014	`R`	4100	Set subroutine linkage.
3015	`U`	4201	Jump to subroutine.
3016		*etc.*	Return point.
3129	`XZ`	3236	Address of parameter block.
3236	`XZ`	1000	L_1 (Starting address of unsorted list).
3237	`XZ`	2020	L_3 (Starting address for sorted destination list).
3238	`XZ`	0005	R (Number of records/intervals to sort).
3239	`XZ`	0003	N (Number of words per record/interval).
3240	`XZ`	0002	K (Position of the sorting key word within a record).
3241	`XZ`	000t	Method selection code ($t = 1, 2,$ or 3).
3242	`000000WQ`		Sorting mask.

The calling sequence above defines a list of five 3-word records [intervals], stored starting at memory location 1000. The list is to be sorted based on the contents of the second word ($K = 2$) of each record, using the hexadecimal mask `000000WQ`. The resulting sorted list is to be stored starting at memory location 2020.

Original List Data Address Mapping	**Value**	**Method I Location**	**Method I Value**	**Method II Value**	**Method III Value**
1000 → 2020	7658	1010	614	3846	136
1001 → 2021	136	1004	428	614	428
1002 → 2022	3429	1001	136	4948	346
1003 → 2023	1876	1013	346	1876	170
1004 → 2024	428	1007	170	428	4470
1005 → 2025	4470			4470	7658
1006 → 2026	18208			7658	136
1007 → 2027	170			136	3429
1008 → 2028	7720			3429	5650
1009 → 2029	3846			5650	346
1010 → 2030	614			346	6312
1011 → 2031	4948			6312	18208
1012 → 2032	5650			18208	170
1013 → 2033	346				7720
1014 → 2034	6312				

18 Conversion to Binary

18.1 Binary Conversion of Whole Numbers 1

PURPOSE

The program L3-10.0 converts a positive 8-digit decimal number into its binary equivalent. Leading zeros must be explicitly included.

The program operates as a subroutine invoked by a calling sequence from the main program. Upon jumping to the subroutine, the return address must be stored, and the argument must reside in the accumulator. Upon returning to the main program, the binarized number is held in the accumulator.

STORAGE REQUIREMENTS

Memroy used: 58 sectors (starting at sector 00)

Temporary storage: none

INPUT

The input variable – a positive 8-digit decimal number scaled at $q = 31$ – must reside in the accumulator when the jump to the subroutine occurs. Leading zeros must be explicitly specified.

CALLING SEQUENCE

Memory Location	Instruction	Address	Meaning
$\alpha - 1$	`B`	$\langle \dots \rangle$	Load the input argument @ 31.
α	`R`	$L_0 + 42$	Store the return address in the subroutine.
$+1$	`U`	L_0	Jump to the subroutine starting address.
$+2$		*etc.*	Return point in the main program.

The instruction at $\alpha - 1$ brings the argument scaled at $q = 31$ into the accumulator. (Any instruction that achieves this may be used.) The instruction at α stores the return address $(\alpha + 2)$ into the subroutine. The instruction at $\alpha + 1$ effects a jump to L_0, the starting address of the subroutine. Control returns to location $\alpha + 2$ of the main program.

EXAMPLE

Suppose the 8-digit decimal value to be converted is 00024576, stored in memory location 2500. Let the subroutine "Binarization 1" be loaded at location 0800, and the main program calling sequence occupies locations 1200 to 1203.

Memory Location	Instruction	Address	Meaning
1200	`B`	2500	Bring decimal number 00024576 @ 31 into the accumulator.
1201	`R`	0842	Set return address linkage inside the subroutine.
1202	`U`	0800	Jump to the subroutine entry point.
1203		*etc.*	Execution resumes here with the binary result held in the accumulator @ 30.

The instruction at 1200 loads the data word containing the decimal integer representation. The instruction at 1201 sets the return path to cell 1203 inside the subroutine's exit tracking line (at $0800 + 42$). Cell 1202 triggers the jump to the routine. Upon return to cell 1203, the accumulator contains the true binary value equivalent to `0x6000` (decimal 24576) correctly shifted to $q = 30$.

OUTPUT

Upon returning to the main program, the binarized number scaled at $q = 30$ stands in the accumulator.

OPERATION

1. **Execution time:**
   Average of 530 milliseconds.

2. **Program entry:**
   The tape contains the program in two versions:

   a) arbitrarily relocatable, hexadecimal, permissible for Program Input 1, and

   b) arbitrarily relocatable, decimal, in coding sheet format.

3. **Required input/output units:**
   No input or output units are required.

4. **Overflow:**
   Ignored upon entry and not reset upon exit.

19 Backup Utilities

19.1 Backup 1

19.2 Tape Duplicator 1

20 Binary Searching

20.1 Table Search 1 (Binary Search)

21 Programming Utilities

21.1 Loop Driver Utility

22 Device Support

22.1 High-speed Punch Output

23 Bibliography

References

[1] J. von Neumann, "First Draft of a Report on the EDVAC," Moore School of Electrical Engineering, University of Pennsylvania, Philadelphia, PA, Tech. Rep., Jun. 1945.

[2] A. W. Burks, H. H. Goldstine, and J. von Neumann, "Preliminary discussion of the logical design of an electronic computing instrument," Institute for Advanced Study, Princeton, NJ, Tech. Rep., Jun. 1946.

[3] International Business Machines Corporation, *Principles of Operation, Electronic Data Processing Machines Type 701*, New York, NY, 1953.

[4] Digital Computer Laboratory, *Electronic Digital Computer: Internal Organization, Programming, and Coding (ILLIAC I)*, University of Illinois, Urbana, IL, 1954.

[5] Elliott Brothers (London) Ltd., *The Elliott 405 Electronic Data Processing System: Reference Manual*, London, UK, 1956.

[6] Royal McBee Computer Corporation, *LGP-30 Programming Manual*, Port Chester, NY, 1957.

[7] General Precision Electronic Computer Company, *LGP-21 Maintenance and Training Manual*, Burbank, CA, 1963

[8] Eurocomp GmbH, *LGP-21 Unterprogramm-Handbuch*, Minden, Westphalia, 1963

[9] General Precision Electronic Computer Company, *LGP-21 Computer System Reference Manual*, Burbank, CA, 1963

[10] General Precision Electronic Computer Company, *RPC-4000 Programmers Reference Manual*, Burbank, CA, 1960.

[11] E. Nigh, "The Story of Mel," first posted to Usenet newsgroup `net.followup`, May 1983. [Online; accessed June 2026]. Available: `https://users.cs.utah.edu/~elb/folklore/mel.html`